

# Rust

## Variabili e tipi

Una variabile lega un valore ad un nome (**binding**). Viene introdotta dalla parola chiave **let**, che definisce un comportamento **immutabile**. Rust infatti, favorisce l'immutabilità. Di **default** una variabile può essere legata ad **un solo valore** per tutta la sua esistenza. E' possibile indicare che la variabile potrà essere legata, in futuro, ad un altro valore (dello stesso tipo) aggiungendo alla parole chiave `let` il modificatore **mut**.

Ad ogni variabile è associato **staticamente** (ovvero per tutta la durata del programma) un tipo. Il tipo definisce l'insieme dei valori che possono essere memorizzati in una variabile, così come l'insieme delle operazioni che possono essere effettuate su tali valori. Il tipo associato ad una variabile può essere esplicitamente definito nella clausola `let` che introduce la variabile. Il motore di inferenza dei tipi consente, nella maggior parte dei casi, di dedurre il tipo di una variabile dal valore con cui è stata **inizializzata**, rendendo opzionale la dichiarazione esplicita

```
let v: i32 = 123;           // v è immutabile e ha tipo i32
v = -5;                    // ERRORE non è possibile riassegnare il valore

let mut w = v;             // w può essere riassegnata, ha lo stesso tipo di v
w = -5;                    // Ora w vale -5

let x = 1.3276              // x è immutabile di tipo f64

let y = 1.3278f32           // y è immutabile di tipo f32

let one_million = 1_000_000 // si possono usare gli _ per separare le cifre
```

Per stampare sul flusso di stdout: **print!(...)** oppure **println!(...)**. Il primo parametro è una stringa di formato: per ciascun *placeholder* `{}` presente al suo interno deve essere indicato un parametro successivo, il cui contenuto sarà inserito al posto delle graffe. Tutto quello che non sta nelle graffe viene stampato così com'è:

```
println!("Hello, {}!", "world");    // Hello, world!
```

Per stampare sul flusso stderr: **eprint!(...)** oppure **eprintln!(...)**.

In Rust `print!()` e `println!()` sono **macro** espanse a livello di codice sorgente dal compilatore prima di diventare codice oggetto.

```
fn main() {
    let i:i32 = -12
    println!("Numero {i}!");

    let i = "ciao mamma";          // La presenza della seconda variabile i nasconde
                                    // la prima
    println!("Stringa {i}");
}
```

Con lo **shadowing** si crea una nuova variabile che nasconde la copia precedente, ma non la distrugge:

```
fn main() {
    let mut x = 5;
    x = x + 1;

    {
        let x = x * 2;
        println!("The value of x in the inner scope is: {x}");    // 12
    }

    println!("The value of x is: {x}");                          // 6
}
```

Con lo shadowing si può cambiare tipo, mentre con `mut` non si può cambiare.

Un'**espressione** è un costrutto sintattico la cui esecuzione produce un valore di un dato tipo. Le espressioni sono valutate **bottom-up**, si possono usare le parentesi tonde per modificare l'ordine con cui un'espressione viene valutata. Una variabile è un modo comodo per catturare il risultato di un'espressione attribuendogli un nome, per poterlo usare in seguito.

Ad una variabile **dobbiamo** assegnare un **valore iniziale** che può essere immediato o il risultato di un'espressione. Possiamo non assegnare un valore iniziale ad una variabile, ma, ovviamente, non possiamo leggere una variabile a cui non è stato assegnato ancora un valore. Sulla base del valore che verrà poi assegnato alla variabile il compilatore inferisce il tipo.

Tutte le espressioni producono un valore. A differenza di quanto avviene in C, la maggior parte delle istruzioni vengono viste come espressioni. Ma, al contrario, alcuni costrutti che in C restituiscono un valore, in Rust hanno tipo **()**, che corrisponde al tipo `void` del C. Le **()** sono chiamate tipo **Unit**, non ha una rappresentazione in memoria, esiste solo a livello di sistema dei tipi. Il tipo `Unit` è un placeholder che indica *operazione completata senza un risultato da restituire*.

```
fn main() {
    let x = 5;

    // Istruzione if restituisce un valore
    let y = if x > 3 { 10 } else { 20 };

    println!("y = {}", y);                                // Stampa "y = 10"
}
```

```
fn main() {
    let risultato = {
        let a = 5;
        let b = 10;

        // L'ultima espressione di un blocco è il valore restituito
        a + b // Senza ;
    };

    println!("Risultato: {}", risultato);
}
```

In C l'**assegnazione** è vista come una funzione e restituisce un valore. In Rust non è possibile perché vincola al principio di transitività tra tipi che non è detto che sia valido.

Rust offre un insieme di **tipi** predefiniti. E' possibile definire tipi ulteriori sotto la forma di struct, enum, funzioni e chiusure. A differenza di quanto succede in altri linguaggi, i tipi **non** sono organizzati in una gerarchia di ereditarietà, ma sono tutti **indipendenti** tra loro. Di seguito un elenco di tipi:

- **Numeri interi con segno:** `i8`, `i16`, `i32`, `i64`, `i128`, `isize` (`isize` e `usize` sono grandi quanto il parallelismo del processore)
- **Numerici interi senza segno:** `u8`, `u16`, `u32`, `u64`, `u128`, `usize`
- **Logici:** `bool`
- **Caratteri (32 bit, Unicode Scalar Value):** `char` (un `char` può contenere qualunque simbolo Unicode)
- **Unit:** `()`. Rappresenta una tupla di 0 elementi, `()` indica sia il tipo che il suo unico valore possibile. Serve per descrivere il comportamento di quelle funzioni che non ritornano un valore esplicito e corrisponde al tipo `void` in C
- **Never:** il tipo `Never` è un tipo speciale che indica che una funzione o un'espressione non ritorna mai un valore (cioè non **termina mai**) oppure che causa un **panic**

**Panic!()** è un meccanismo di gestione degli errori utilizzato quando il programma incontra una situazione **irrecuperabile** dalla quale non può continuare l'esecuzione in modo sicuro. Quando viene eseguito un `panic!()` il programma si **arresta** e produce un messaggio di errore che può includere informazioni utili. Questo tipo di errore è utilizzato per situazioni in cui il programma non può continuare in modo sicuro e non esiste un modo per recuperare (**fatal errors**). Quando si verifica un panic, Rust entra in un processo di **unwinding**, liberando tutte le risorse allocate nello stack e invocando eventuali funzioni di distruzione (`drop`) sugli oggetti che vanno distrutti

```
fn main() {
    let x = -1;

    if x < 0 {
        panic!("Valore negativo non supportato: {}", x);
    }

    println!("Il valore di x è: {}", x);
}
```

Il compilatore è in grado di capire a volte se il codice entrerà in stato di panic, ma qualche volta potrebbe non esserne capace.

```
fn main() {
    let mut u1: u8 = 1;
    let mut u2: u8 = 2;

    u2 *= 2;
    u1 += u2;
    u1 -= 10;
}
```

// Un valore negativo genera panic

Il tipo **char** non è equivalente ai numeri e non ci sono 2 alla 32 simboli diversi definiti nell'Unicode (codifica qualche milione di simboli).

Il compilatore lega ai tipi delle proprietà descritte attraverso i **tratti**. Un **tratto** descrive un insieme di comportamenti (**metodi**) di un dato **tipo implementa**. Il compilatore utilizza l'informazione che un certo tipo implementa un dato tratto per governare il codice che viene generato manipolando un'espressione che consuma o genera quel particolare tipo.

Mentre i tipi sono isolati e non hanno relazione di parentela tra di loro, i tratti possono avere dipendenze reciproche e dunque fanno sì che certe cose possono essere condivise tra diversi tipi di oggetti. Qualsiasi tipo, predefinito o meno, può implementare zero o più tratti.

Ad esempio il tipo `i32`:

- Gode del tratto `Copy`, significa che permette di effettuare una copia di un dato di tipo intero a fronte di un'assegnazione.
- Gode del tratto `Eq`, significa che un dato intero è confrontabile con un altro intero e posso applicare l'operatore `==`.
- Gode del tratto `Ord`, significa che posso valutare la relazione d'ordine e applicare gli operatori relazioni `<`, `>`, `<=`, `>=`.
- Non gode del tratto `Drop`, utilizzato per altri tipi per definire le azioni svolte quando il valore esce dallo scope e viene distrutto. Questo perché un valore di tipo `i32` è di un tipo

elementare che viene automaticamente gestito dal sistema di gestione della memoria di Rust senza bisogno di definire azioni particolari.

I tratti sono legati tra di loro in una relazione di **dipendenza** che può essere **negativa**, ad esempio chi gode del tratto `Drop` non può avere il tratto `Copy` e viceversa, o **positiva**, ad esempio chi ha il tratto `Copy` deve avere il tratto `Clone`.

Esistono anche tratti privi di metodi (**tratti marker**) per definire una proprietà del tipo. Ad esempio i tratti della programmazione concorrente `Sync` e `Send` sono tratti privi di metodi e i tipi possono godere del tratto `Sync` oppure `Send` oppure di entrambi oppure di nessuno dei due.

Poiché tipi diversi possono implementare tratti comuni, si viene a creare una forma di **parentela** alquanto articolata tra i tipi. Rust introduce una ventina di tratti predefiniti, cui il compilatore associa un particolare significato, e permette al programmatore di aggiungerne altri a piacere, al fine di estendere tale comportamento.

Le **Tuple** rappresentano collezioni ordinate di valori eterogenei. Vengono costruite racchiudendo i valori in parentesi tonde. Una tupla ha tipo `(T1, T2, ..., Tn)` dove `T1`, `T2`, ..., `Tn` sono i tipi dei singoli valori. Se dichiaro una tupla come mutabile, rendo mutabile tutte le sue parti. Si accede al contenuto di una tupla utilizzando la notazione `.0`, `.1`, .... Una tupla può contenere un numero arbitrario di valori.

```
let t: (i32, bool) = (123, false);           // t è una tupla formata da un intero
                                              // e da un booleano
let mut u = (3.14, 2.71);                   // u è una tupla riassegnabile formata
                                              // da due double
let i = t.0;                                // i contiene il valore 123
u.1 = 0.0;                                   // adesso u contiene (3.14, 0.0)
```

Rust offre vari modi per rappresentare indirizzi di memoria: riferimenti, box e puntatori nativi. Diversi tipi di puntatori con prerogative e limitazioni particolari. A differenza di quanto capita in C, l'uso dei puntatori è abbondantemente semplificato grazie alle garanzie offerte dal compilatore del linguaggio, che verifica il **possesso** ed il **tempo di vita** delle variabili e garantisce che possano avvenire solo accessi che è possibile dimostrare essere leciti, oppure forza il programmatore ad assumersi la responsabilità della correttezza del proprio operatore racchiudendo il codice in un blocco `unsafe{...}`

L'espressione `let r1 = &v;` dove `v` è un qualsiasi valore o espressione, definisce ed inizializza il **riferimento immutabile** `r1`. La variabile `r1` prende in **prestito (borrows)** il valore `v` e potrà

accedervi (**in sola lettura**) con l'espressione **\*r1**. Un riferimento viene rappresentato internamente come un blocco di memoria contenente l'indirizzo di memoria in cui il valore è memorizzato. Un dato il cui indirizzo è in prestito **non può essere modificato**, ma può essere prestato molte volte. I riferimento in sola lettura **possono essere copiati** assegnandogli un'altra variabile o passandogli come parametro ad una funzione: ma fino a che esiste almeno un riferimento ed è in uso, il valore originale non è modificabile.

L'espressione **let r2 = &mut v;** definisce ed inizializza **il riferimento mutabile r2**. La variabile **r2** prende in prestito, **in modo esclusivo**, il valore **v** e permette di modificarlo. Prestito esclusivo è consentito **1 solo alla volta**. Finché un riferimento esiste ed è in uso, non è possibile né creare altri riferimenti al valore originale, né accedere in alcun modo al valore originale (**mutuamente esclusivo**), neanche essere letto.

Non può esistere un riferimento nullo, perché si riferisce sempre ad un valore o espressione esistente

**Ref (&)** e **RefMut (&mut)** sono puntatori **privi di possesso** ossia, chi li riceve non ha la responsabilità del rilascio che rimane al possesso originale. Se voglio operare sul dato puntato posso invocare un **metodo** senza necessariamente dereferenziare il Ref: il compilatore va a vedere che cosa c'è prima del punto e se è un Ref lo dereferenzia automaticamente:

```
fn main() {
    let v = 32;
    let p = &v;
    let pp = &p;
    let ppp = &pp;

    let str = ppp.to_string();
    println!("{}", str);
}
```

Sebbene possano apparire simili ai puntatori in C, i riferimenti Rust non possono mai essere nulli, né tantomeno contenere l'indirizzo di un valore che è stato già rilasciato o non è stato inizializzato. Se un valore svanisce, il riferimento deve svanire prima di lui. I riferimenti implementano una logica **single writer or multiple readers**. Il compilatore, attraverso un apposito modulo detto **borrow checker** si fa carico di garantire questa regola che costituisce la base delle regole di sanità all'interno di Rust.

```
fn main() {
    let mut i = 32;
    let r = &i;
    println!("{}", *r);

    i = i+1; // Problematico!
    println!("{}", *r);
}
```

```
fn main() {
    let mut i = 32;
    let r = &mut i;
    println!("{}", i);    // Problema!

    *r = *r+1;
    println!("{}", *r);
}

fn main() {
    let a = 2;
    let p = &mut 12;
    *p = *p + 1;
    let p1 = & (a + 2);
    let p2 = &(*p1 * 2);

    println!("{}", {} {}, *p, *p1, *p2);
}
```

I riferimenti **non sanno** dove sta il dato puntato. Conoscono l'indirizzo, a non possono dire in quale zona della memoria il dato è memorizzato.

Una variabile locale è **sempre** allocata sullo stack. Questo si estende di una quantità pari alla dimensione del valore che deve essere memorizzato nella variabile. Quando la variabile esce dal proprio scope sintattico, lo stack si contrae ed il valore viene rilasciato. In alcune situazioni **non è nota la dimensione** del dato che deve essere memorizzato. In altri, occorre **prolungare il tempo di vita** del valore oltre quello del blocco sintattico in cui è definito.

Il tipo generico **Box<T>** è specifico per puntatori per dati dinamici che vengono allocati all'interno dello **heap**. Una variabile di questo tipo contiene il puntatore al valore e **possiede** il dato. Si garantisce che venga fatta l'allocazione di memoria dallo heap e venga fatto il rilascio al termine dell'uso della variabile. Il Box implementa il tratto **Drop** e quando il Box sparirà il dato verrà distrutto e rilasciata la memoria sullo heap. Si alloca un valore di tipo Box con il costrutto:

```
let b = Box::new(v)
```

dove **v** è un qualsiasi valore: `new` richiede di sapere che cosa ci devo mettere dentro il Box. Questa istruzione definisce la variabile **b** che conterrà un puntatore ad un blocco allocato sullo heap che a sua volta contiene il valore **v**. Si accede al valore contenuto nel blocco con l'espressione **\*b**. Se la variabile **b** è definita come mutabile, è possibile modificare il contenuto a cui si punta. Quando l'esecuzione del programma raggiungerà la fine del blocco di codice in cui la variabile **b** è stata definita, il blocco sarà rilasciato a meno che il contenuto di **b** sia stato mosso in un'altra variabile.

```
fn makeBox(a: i32) -> Box<i32> {
    let r = Box::new( (a, 1) );
    return r;
}
```

```
fn main() {
    let b = makeBox(5);
    let c = b.0 + b.1;
}
```

Il tratto **Display** stampa una variabile riferita con `{ }` e fornisce un formato di stampa orientato all'utente finale. Il tratto **Debug** stampa una variabile riferita con `{ :? }` e fornisce un formato orientato al programmatore; l'opzione `{ :#? }` fornisce un formato più leggibile (*pretty printing*). Il tratto **Pointer** `{ :p }` permette di stampare l'indirizzo di una variabile puntatore. Le tuple e il Box implementano il tratto Debug. I numeri implementano il tratto Display. Il tratto **Pointer** è necessario per leggere l'indirizzo.

Il tratto **Copy** permette di duplicare valori attraverso **memcpy**. Se il tipo implementa solo il tratto **Clone**, l'operatore `=` determina un **movimento** del dato. Se voglio copiare un dato, senza movimento (**deep copy**) si deve utilizzare esplicitamente il metodo `.clone()`. Il tratto **Copy** determina il tratto **Clone** e il metodo `.clone()` può essere definito in modo arbitrario dal programmatore. Il tratto **Copy** è eseguito implicitamente non può essere re-implementato. Il tratto **Clone** deve essere eseguito esplicitamente e può essere re-implementato.

Rust definisce anche i tipi dei puntatori nativi come **\*const T** e **\*mut T**, per qualsiasi tipo T. **\*const** è un puntatore ad un dato che non possiamo mutare. **\*mut** è un puntatore ad un dato che possiamo modificare. Questi sono, a tutti gli effetti, equivalenti ai puntatori in C e C++ e ne condividono tutti i problemi. Tuttavia, è possibile deferenziarli solo all'interno di blocchi **unsafe { ... }**. Se un programma non fa uso di blocchi unsafe, o se quelli che sono usati contengono solo codice corretto, allora si può essere certi che l'esecuzione del programma non darà origine a comportamenti non definiti. In Rust, l'accesso ai dati tramite puntatori raw è raramente necessario e viene usato principalmente quando si lavora con codice non sicuro (**unsafe cose**) e possono essere usati per accedere a memoria non gestita:

```
fn main() {
    let x = 10;
    let r: *const i32 = &x;

    unsafe {
        println!("Il valore di x tramite puntatore raw è: {}", *r);
    }
}
```

Un **Array** è una sequenza di oggetti **omogenei**, disposti **consecutivamente** nello stack. Un array ha una **dimensione definita** all'atto della sua creazione ed **immutabile**. Si crea un array racchiudendo la sequenza dei suoi valori tra parentesi quadre. Un Array ha tipo `[T; length]`, dove T è il tipo dei singoli elementi e length indica il numero dei valori contenuti. La



lunghezza di un array è immutabile. Si accede al contenuto dell'array con la notazione **nome[index]**:

```
let a: [i32; 5] = [1, 2, 3, 4, 5];      // a è un array di 5 interi let
mut b = [0; 5];                        // b è un array di 5 interi inizializzati a 0
                                        // NOTARE il ; per distinguere le notazioni
let l = b.len();                       // metodo len() calcola la lunghezza
                                        // l vale 5
let e = a[3];                          // e vale 4
b[1] = 8;                              // modifico il contenuto
```

Il metodo **get()** permette di fare un accesso sicuro. Restituisce un valore di ritorno di tipo **Option<T>**.

Rust offre la possibilità di fare riferimento ad una sequenza di valori consecutivi. Una **Slice** di elementi di tipo **T**, scritto **&[T]** è un tipo di dato formato da due valori consecutivi: il puntatore all'inizio della sequenza e il numero di elementi della sequenza. Per questa sua natura, viene detto **fat pointer**. Si crea una slice come riferimento (**prestito**) ad una **promozione** di un **Array** o di un **Vec**:

```
let a = [ 1, 2, 3, 4 ];
let s1: &[i32] = &a;                  //s1 contiene i valori 1, 2, 3, 4
let s2 = &a[0..2];                     // s2 contiene i valori 1, 2
let s3 = &a[2..];                      // s3 contiene i valori 3, 4
```

Di base, una slice è immutabile e acquisisce la possibilità di modificare il contenuto attraverso la notazione **let ms = &mut a[..]**; Come nel caso degli array, si accede ai valori contenuti in una slice con la notazione **s[index]**. Tentativi di accedere ad una posizione illecita comportano l'immediato arresto del programma (**panic!**).

Il tipo **Vec<T>** rappresenta una sequenza **ridimensionabile** di elementi di tipo **T**, allocati sullo **heap**. Offre una serie di metodi per accedere al suo contenuto e per inserire/togliere valori al suo interno. Una variabile di tipo **Vec<T>** è una tupla formata da tre valori privati:

- Un **puntatore** ad un buffer allocato sullo heap nel quale sono memorizzati gli elementi
- Un intero privo di segno che indica la **dimensione** complessiva del buffer
- Un intero privo di segno che indica **quanti elementi sono valorizzati** nel buffer

**Vec::new()** alloca un vettore inizialmente vuoto. Se si richiede ad un oggetto di tipo **Vec<T>** di inserire un nuovo elemento, questo verrà memorizzato nel buffer nella prima posizione libera e verrà incrementato l'intero che indica il numero di elementi effettivamente presenti. Nel caso in cui il buffer fosse già completo, verrà allocato un nuovo buffer di dimensioni maggiori e il contenuto del buffer precedente sarà riversato in quello nuovo, dove verrà poi inserito anche il nuovo elemento. Dopodiché il buffer precedente sarà de-allocato. E' possibile inizializzare un **Vec** attraverso la macro **vec!**

```
fn main() {
    let mut v1 = vec![1, 2, 3, 4, 5];
    let mut v2 = vec![0;8];
    v1[0] = 100;
    v2[1] = 5;

    println!("v1.size: {:?} v1.capacity: {:?}", v1.len(), v1.capacity());
    println!("v2.size: {:?} v2.capacity: {:?}", v2.len(), v2.capacity());
    println!
    ("{:?} \n{:?}", v1, v2);
}
```

Rust offre due modi principali di rappresentare le stringhe con rappresentazione **Unicode UTF-8**: come array di caratteri (**immutabili**), memorizzati in un'area statica, rappresentato dal tipo primitivo **str**. Come oggetti allocati dinamicamente (**mutabili**), utilizzando il tipo **String**

Le **costanti** di tipo stringa presenti nel codice sorgente sono racchiuse tra doppi apici `"`. Il compilatore provvede ad inserirle in un'apposita **area statica di memoria**, in modo compatto, senza aggiungere alcun terminatore. Hanno tempo di vita per tutta la durata del programma.

```
let s = "Ciao Mamma";
```

Poiché il tipo primitivo **str** non è direttamente manipolabile, si accede ad esso **solo tramite uno slice**, di tipo **&str**. Un **fat pointer** contiene l'indirizzo del primo carattere e la lunghezza della stringa. Non esiste **&mut str**

```
let s: &str = "Ciao Mamma";
```

Non si può accedere ai caratteri della stringa attraverso l'indicizzazione:

```
fn main() {
    let s = "Ciao Mamma";
    println!("{}", s);
    println!("{}", s.chars().nth(1).unwrap());
}
```

Gli oggetti di tipo **String** contengono:

- Un puntatore ad un buffer allocato dinamicamente
- L'effettiva lunghezza della stringa
- La capacità del buffer

Se la stringa è **mutabile** e vengono inseriti al suo interno più caratteri di quelli che il buffer può contenere, il buffer viene automaticamente allocato nuovamente con una capacità maggiore, così da ospitare quanto richiesto. Aspetto importante: se una funzione accetta un parametro di tipo **&str**, è possibile passare come argomento corrispondente il riferimento ad un oggetto **String**

```
fn main() {  
    let hello: &str = "hello,";  
    let mut s = String::new();  
    s.push_strS(hello);  
    s.push_str(" world!");  
}
```

Si crea un oggetto **String** con le istruzioni:

```
let s0 = String::new();           // Crea una stringa vuota  
let s1 = String::from("some text"); // Crea una stringa inizializzata  
let s2 = "some tet".to_string();  // Equivalente al precedente
```

Si ricava un oggetto di tipo **&str** da un oggetto **String** con il metodo: `s2.as_str()`;

Un oggetto **String** (se mutabile) può essere modificato:

```
s3.push_str("This goes to the end"); // aggiunge al fondo  
s3.insert_str(0, "This goes to the front"); // inserisce alla posizione data  
s3.remove(4); // elimina il carattere alla posizione indicata  
s3.clear(); // svuota la stringa
```

In altri casi si può costruire un altro oggetto **String**:

```
let s4 = s1.to_uppercase(); // forza il maiuscolo  
let s5 = s1.replace("some", " more "); // sostituisce un blocco  
let s6 = s1.trim(); // elimina spaziature iniziali e finali
```

In Rust, ci sono differenze fondamentali tra un **char** e un elemento di una **String** a livello di memorizzazione. Un **char** in Rust occupa sempre 4 byte e può rappresentare qualsiasi punto di codice Unicode. Una **String** è invece una sequenza di byte UTF-8 codificati, dove ogni carattere può contenere da 1 a 4 byte, a seconda del point code Unicode. `string_var.as_bytes()[i]` accede ad un singolo byte in una **String** con l'indicizzazione e genera un singolo byte (u8), non un carattere completo. Per ottenere caratteri completi occorre iterare sulla **String** usando il metodo `.chars()`, che decodifica correttamente la sequenza UTF-8:

```
fn main() {
```

```

let s1 = "Ciao mamma".to_string();
for c in s1.chars()
{
    print!("{}", c);
}

```

L'iterazione attraverso i caratteri di una `String` richiede la decodifica UTF-8 durante l'esecuzione e non si può accedere a caratteri arbitrari in una `String` in tempo costante  $O(1)$ , poiché occorre scorrere la stringa per decodificare i byte. Il tipo `&mut str` è un po' particolare: non si può cambiare un carattere con un altro se hanno dimensioni diverse in byte, dunque non si possono aggiungere o togliere caratteri da una stringa avendo un `&mut str`.

Posso memorizzare un array di caratteri in una slice di byte. `&[u8]` accetta qualunque combinazione dei valori dei byte, `&str` accetta solo valori UTF-8:

```

fn main() {
    let name: &[u8] = b"Torino";
    println!("{}", name);
}

```

Posso memorizzare un array di caratteri in un array statico di N byte. Non tutte le combinazioni di byte corrispondono a caratteri:

```

fn main() {
    let name: &[u8; 6] = b"Torino";
    println!("{}", name);
}

```

In Rust le stringhe sono rappresentate come le vede il Sistema Operativo. Quando è necessario interagire con il SO, i tipi `&str` e `String` possono essere convertiti nei tipi specifici della piattaforma che si chiamano **OsStr (vista)** e **OsString (oggetto mutabile)**, codificato secondo le specifiche dell'API del SO.

Il **path** identifica la collocazione di un file nel File System, con la rappresentazione del cammino dalla radice al file con un **separatore** che in Linux è `/` e in Windows è `\`. **Path (immutabile)** e **PathBuf (mutabile)** rappresentano una stringa adatta a rappresentare un path.

**CStr** e **Cstring** ci garantiscono l'interoperabilità con il linguaggio C. Il linguaggio C ha bisogno che ci sia lo 0 finale. Rust non mette lo 0 finale perché tenendosi la lunghezza non ha bisogno dello 0 finale. Se abbiamo bisogno di collegare una libreria C e se quella libreria ha bisogno di stringhe dovremo convertire prima le nostre Stringhe Rust nei corrispondenti `Cstr` e `Cstring`.

`&'static str` permette di allocare una stringa per tutta la durata del programma. Permette di definire uno slice ha un tipo di vita coincidente con il tempo di vita dell'intero programma:

```
fn main() {
    let static_string: &'static str = "Hello, world!";
    println!("{}", static_string);
}
```

Il corpo di una funzione è costituito da istruzioni e/o espressioni separate da `;`. Una istruzione ha come tipo di ritorno `()`, un'espressione può restituire un tipo arbitrario. I costruttori **let ...** e **let mut ...** sono istruzioni. Un blocco racchiuso tra `{ ... }` è un'espressione. Restituisce il valore corrispondente all'ultima espressione, a condizione che **non** sia terminata da `;`. Il costrutto **if ... else ...** è una espressione. Il ramo positivo ed il ramo negativo sono costituiti da blocchi che devono restituire lo stesso tipo di dato. Il costrutto **loop ...** è un'espressione. Crea un iterazione infinita che può essere interrotta eseguendo l'istruzione **break** seguita dal valore di ritorno (se presente). Una singola iterazione può essere parzialmente saltata eseguendo l'istruzione **continue**.

Le **funzioni** costituiscono il nucleo principale attorno al quale viene definito il comportamento di un programma. Una funzione è introdotta dalla parola chiave **fn** seguita dal nome e dalla lista di argomenti, ciascuno con il relativo tipo, racchiusa tra parentesi tonde. Se ritorna un valore diverso da `()`, la lista degli argomenti è seguita dal simbolo `->` e dal tipo ritornato. Il corpo della funzione è racchiuso tra `{ }` ed è composto da istruzioni. L'ultima espressione presente nel corpo, se priva di `;` finale, viene interpretata come valore di ritorno. In alternativa, è possibile utilizzare l'istruzione **return** seguita dal valore e da `;`

E' possibile **annidare** più costrutti di tipo loop ed interrompere o continuare un particolare livello di annidamento, facendo precedere l'istruzione loop da un'etichetta. L'etichetta è un identificatore preceduto da ```. Le istruzioni **break** e **continue** possono indicare l'etichetta a cui fanno riferimento:

```
fn main() {
    'outer: loop {
        println!("Entrato nel ciclo esterno");

        'inner: loop {
            println!("Entrato nel ciclo interno");
        }
    }
}
```

L'istruzione **while** permette di subordinare l'esecuzione del ciclo al verificarsi di una condizione. In modo analogo a quanto avviene in altri linguaggi. L'istruzione **for** ha una sintassi particolare:

```
for var in expression { code }
```

**expression** deve restituire un valore che sia (o possa essere convertito in) un iteratore: sono leciti, ad esempio, `array`, `slice` e `range`. Le notazioni `a..b` e `c..=d` indicano, rispettivamente, un intervallo semi-aperto e un intervallo chiuso. Possono essere usati in senso generale, riferendosi al dominio del tipo della variabile, oppure possono essere applicati ad una slice, riferendosi all'insieme dei valori leciti. Sono possibili diverse combinazioni:

- `..` indica tutti i valori possibili per un dato dominio
- `a..` indica tutti i valori a partire da **a** (**incluso**)
- `..b` indica tutti i valori fino a **b** (**escluso**)
- `..=c` indica tutti i valori fino a **c** (**incluso**)
- `d..e` indica tutti i valori tra **d** (**incluso**) ed **e** (**escluso**)
- `f..=g` indica tutti i valori tra **f** e **g** (**inclusi**)

L'espressione **match** permette di eseguire in modo condizionale blocchi di codice confrontando un valore con una serie di pattern alternativi. Essa confronta la **struttura** del valore con i singoli pattern indicati. Tali pattern possono contenere **variabili**, che, in caso di corrispondenza delle parti costanti, vengono legate al corrispondente frammento del valore confrontato. L'elenco dei pattern deve essere **esaustivo** del dominio dell'espressione. Ciascun pattern è separato dal blocco di codice da eseguire dal simbolo `=>`. Il pattern può essere annotato con una clausola **if** per limitarne l'applicabilità. I diversi rami sono separati da `,`. Le espressioni di confronto contenute nel pattern possono essere annotate con un identificatore seguito da `@`, per legare il valore confrontato al nome dato, così da poter fare riferimento ad esso nel blocco corrispondente.

L'espressione **match** offre una sintassi concisa e sofisticata per confrontare valori multipli così come per estrarre valori da tipi complessi. Per indicare un singolo valore, non occorre nessun operatore. la sintassi `val1.. = val2` indica un intervallo chiuso. Una barra verticale singola `|` può essere usata per indicare una disgiunzione (or). Il segno di sottolineatura `_` corrisponde a qualsiasi valore. I pattern sono valutati nell'ordine indicato, alla prima corrispondenza viene valutato il blocco associato, il cui valore diventa il valore dell'espressione complessiva. Il costrutto **match** è un'espressione e restituisce un valore (dunque tutti i bracci devono avere lo stesso tipo):

```
let s = match item {
    0 => "zero",                // valore singolo
    10 ..= 20 => "tra dieci e venti", // intervallo inclusivo
    40 | 80 => "quaranta o ottanta", // alternativa
    _ => "altro",              // qualunque cosa
}
```

Analogamente al C/C++, anche Rust consente di progettare eseguibili che adattano il comportamento in base ai parametri passati sulla **riga di comando**. Tali parametri si trovano dentro il contenitore **std::env::args**. Composto da valori di tipo `String`, nessun bisogno

di `argc`: **`args.len()`** ritorna il numero di parametri. Si può accedere come un classico **`argv`** del C/C++ con gli indici. Sfruttabile anche con altri costrutti più evoluti:

```
use std::env::args;
fn main() {
    let args: Vec = args().skip(1).collect();
    if args.len() > 0 {
        for i in 0..args.len() {
            println!("{}", args[i]);
        }
    }
}
```

Il **`crate std::io`** contiene la definizione delle strutture dati per accedere i flussi standard di ingresso/uscita. Questo tipo di operazioni, per definizione, possono fallire: di conseguenza tutti i metodi offerti restituiscono un oggetto di tipo **`Result<T, Error>`** che incapsula, alternativamente, il valore atteso, se l'operazione ha avuto successo, o un oggetto di tipo `Error`, in caso di fallimento. Per garantire la correttezza del programma, occorre gestire esplicitamente l'eventuale errore, verificando il contenuto del valore ritornato tramite il metodo **`is_ok()`**, oppure causare l'interruzione forzata del programma in caso di errore, utilizzando il metodo **`unwrap()`** che restituisce, se non c'è stato errore, il valore incapsulato. Per semplificare le operazioni di scrittura, sono disponibili due macro: **`print!(...)`** e **`println!(...)`** entrambe accettano una stringa di formato e una serie di parametri da stampare.

**`#[allow()]`** è un attributo inserito nel codice per disabilitare i warning del compilatore (chiamati lints). Può essere applicato su un elemento del programma oppure su un blocco o a livello di crate con **`#![allow()]`**.

La comunità degli sviluppatori di Rust ha elaborato una serie di regole sul formato dei nomi delle diverse entità del linguaggio. Si usano nomi nel formato **`UpperCamelCase`** per tutti i costrutti legati al sistema dei tipi. Si usano nomi nel formato **`snake_case`** per i costrutti di tipo valore. Alcune regole che generano warning possono essere disabiliate usando **`#[allow(non_snake_case)]`** o **`#![allow(non_snake_case)]`**.

**`#[deny()]`** e **`#[forbid()]`** sono attributi del compilatore che possono trasformare un warning in un errore di compilazione. **`deny`** può essere sovrascritto localmente, **`forbid`** è più rigoroso e non può essere sovrascritto

## Possesso

In Rust, ogni **valore** introdotto nel programma è **posseduto** da una ed una sola variabile. Un particolare blocco logico nel compilatore, detto **borrow checker**, verifica formalmente questo fatto, per ogni punto dell'esecuzione del programma. Ogni violazione porta ad un **fallimento della compilazione**. Possedere un valore significa essere **responsabili** del suo **rilascio**. Se il valore contiene una risorsa, questa deve essere liberata. Dopodiché occorre restituire al sistema operativo la memoria in cui il valore è memorizzato. Il rilascio avviene quando la variabile che lo possiede **esce** dal proprio **scope** sintattico o quando le viene assegnato un nuovo valore. Rust offre un meccanismo di **drop** mediante il quale è possibile associare azioni arbitrarie da eseguire prima che sia liberata la memoria. Il rilascio può essere rimandato a dopo, se il contenuto della variabile viene trasferito in un'altra variabile: in questo caso la nuova variabile diventa responsabile del suo rilascio.

Il sistema di **ownership** si basa su tre regole principali:

1. Ogni valore in Rust ha una variabile che lo possiede. Questa variabile è chiamata **possessore (owner)**
2. Ci può essere un solo owner per un dato valore alla volta. Quando si assegna il valore di una variabile ad un'altra, la proprietà del valore viene spostata (**mossa, moved**) dalla prima variabile alla seconda. La prima variabile non è più valida dopo lo spostamento (**viene consumata**)
3. Quando l'owner esce dallo scope, il valore viene **eliminato (drop)**

### Possesso e Rilascio

```
fn main() {  
    let mut v = Vec::with_capacity(4);           // v possiede il vec  
  
    for i in 1..=5 {  
        v.push(i);  
    }  
  
    println!("{:?}", v);  
}
```

### Movimento

```
let s1 = "hello".to_string();  
println!("s1: {}", s1);  
  
let mut s2 = s1;  
println!("s2: {}", s2)                               // s1 non è più accessibile
```

Se una variabile viene assegnata ad un'altra variabile oppure passata come argomento ad una funzione, il suo contenuto viene **mosso** nella destinazione. La variabile originale cessa di possedere il valore ed il possesso passa alla variabile destinazione. la variabile originale resta allocata fino a



quando non termina la sua visibilità. Eventuali accessi in **lettura** alla variabile originale porteranno ad **errori di compilazione**. Eventuali accessi in **scrittura** della variabile originale **avranno successo** e ne riabiliteranno la lettura. la variabile destinazione conterrà una **copia** bit a bit del valore originale.

Quando una variabile viene **inizializzata**, prende possesso del relativo valore. Se ad una variabile è **assegnato un nuovo valore**, quello precedentemente posseduto viene rilasciato e la variabile diventa possessore del nuovo valore

```
fn main() {
    let s1 = String::from("hello");
    println!("{s1}");
    {
        let mut s1 = String::from("ciao");
        s1.push_str("Mamma");
        println!("{s1}");
    }
    println!("{s1}");
}
```

Alcuni tipi, tra cui quelli numerici, sono definiti **copiabili**. Implementano il tratto **Copy**, non implementano il tratto **Drop**. Quando un valore viene assegnato ad un'altra variabile o usato come argomento in una chiamata a funzione, il valore originale rimane accessibile in lettura. Questo è possibile quando il valore contenuto non costituisce una “risorsa” che richiede ulteriori azioni di rilascio. I tipi semplici e le loro combinazioni sono copiabili, così come sono copiabili i riferimenti a valori non mutabili. I riferimenti a valori mutabili **non** sono copiabili. Da un punto di vista del codice generato, **non cambia nulla** tra copia e movimento. L'istruzione di assegnazione o il passaggio come argomento comporta la duplicazione del valore originale. Semplicemente, in caso di copia, il borrow checker non impedisce l'ulteriore accesso in lettura al dato originale

**Movimento (per tipi che non implementano Copy)**

```
let s = String::from("hello");
let b = s + " world";                                // non puoi usare più s
```

**Copia (per tipi che implementano Copy)**

```
let i = 42;
let j = i + 1;                                        // puoi ancora usare i e j
```

I tipi che implementano il tratto **Clone** possono essere duplicati invocando il metodo **clone()**. A differenza della copia e del movimento, la clonazione può comportare una **copia in profondità** dei valori. Di conseguenza, il costo della clonazione può essere elevato. Il metodo **clone()** prende un riferimento all'oggetto e ne restituisce uno nuovo, non è mai eseguito dal compilatore, ma può essere eseguito in modo esplicito dal programmatore. Il tratto **Copy** è esclusivamente

implementato dal compilatore. Affinché un tipo possa implementarlo occorre che implementi **anche** il tratto **Clone**, ma il tratto **Copy** è in mutua esclusione con il tratto **Drop**

#### Clonazione

```
let mut s1 = "hi".to_string();
let s2 = s1.clone();
s1.push('!');

println!("s1: {}, s1");           // hi!
println!("s2: {}", s2);          // hi
```

Per risolvere i problemi di accesso, Rust introduce diverse forme di puntatori, volte a rendere esplicite responsabilità e diritti di chi le maneggia. Il borrow checker si occupa di garantire che il codice complessivamente scritto sia conforme a tali regole e impedisce la compilazione in caso contrario. Un **riferimento** è un puntatore in sola lettura ad un blocco di memoria **posseduto da un'altra variabile**. Permette di accedere ad un valore senza trasferirne la proprietà. Il riferimento può esistere solo mentre esiste la variabile che possiede il dato a cui punta. Il compilatore de-riferenzia automaticamente il puntatore, quando si accede al valore tramite l'operatore `*`.

```
let point = (1.0, 0.0);           // point possiede il valore
let reference = &point;           // reference PUO' accedere al valore in
                                  // lettura finché esiste point

println!("{}", reference.0, reference.1);
```

Un riferimento **prende in prestito** l'indirizzo di memoria in cui esiste il valore. Fino a che il riferimento è accessibile, non è possibile modificare il valore, né tramite il riferimento, né tramite la variabile che possiede il valore. E' possibile creare ulteriori riferimenti a partire dal dato originale o da altri riferimenti ad esso. I riferimenti sono **copiabili**: viene duplicato il puntatore. Il compilatore garantisce che un riferimento punti **sempre** ad un dato valido. Finché esiste almeno un riferimento, il dato originale non potrà essere né modificato né distrutto.

A partire da una variabile che possiede un valore è possibile estrarre **un solo riferimento mutabile** per volta. Si crea un riferimento mutabile con la sintassi: **let r = &mut v;**

Mentre esiste un riferimento mutabile, non possono esistere riferimenti semplici (in sola lettura), non è possibile modificare né rimuovere la variabile che possiede il valore e non è possibile neanche leggere il dato originale. E' possibile creare un riferimento mutabile **solo se** la variabile che possiede il dato è dichiarata **mutabile** a sua volta

#### Prestito mutabile

```
let mut s = String::new();
let m = &mut s;                      // Puoi muovere m, puoi downgrade m come &_
```

```

// non puoi copiare m e non puoi usare s

Prestito
let s = String::from("Hello");
let r = &s;                // Puoi copiare r, puoi ancora usare &s
                           // non puoi &mut s, non puoi muovere s

```

I riferimenti sono implementati diversamente in base al tipo puntato:

- **Puntatori semplici**, se il compilatore conosce la dimensione del dato puntato
- **Puntatore + dimensione (fat pointer)**, se la dimensione del dato è solo nota in fase di esecuzione (per garantire che l'accesso spaziale sia safe)
- **Puntatore doppio**, se il tipo di dato puntato è noto solo per l'insieme di tratti che implementa

Il borrow checker garantisce che tutti gli accessi ad un riferimento avvengano solo in un **intervallo di tempo compreso** in quelle in cui il dato esiste. Serve ad impedire il fenomeno del **dangling pointer**. L'insieme delle righe in cui si fa accesso al riferimento costituisce il suo **lifetime**. Tale informazione è mantenuta, **dal compilatore**, insieme alle informazioni che descrivono il tipo del riferimento e non ha nessuna rappresentazione in fase di esecuzione. Sebbene in molte situazioni possa essere omesso, il tempo di vita di un riferimento può essere espresso nella firma del tipo, nel seguente modo:

- **&'a NomeTipo**, dove **a** è un identificativo qualsiasi e il simbolo **' (tick)** serve a qualificarlo come tempo di vita (serve per **assegnare un nome al tempo di vita**). Tale notazione è utile in quelle situazioni in cui occorre imporre vincoli sulla durata relativa dei tempi di vita di due o più riferimenti.

Se un riferimento è valido per l'intera durata di un programma, viene indicato con la notazione **&'static NomeTipo**. Una stringa espressa in formato letterale ("some string"), ha come tipo **&'static str**, in quanto la sequenza di caratteri viene allocata dal compilatore nella sezione delle costanti e non viene mai rilasciata

In molte situazioni il compilatore è in grado di conoscere il tempo di vita del riferimento e non richiede che venga indicato un nome simbolico. Solo in caso di ambiguità è necessario indicare un nome simbolico in modo esplicito. Questo succede quando creiamo delle funzioni che ricevono dei riferimenti (più di uno) e restituiscono come valore di ritorno un riferimento. Se 2 riferimenti:

- Usano **lo stesso nome simbolico di tempo di vita**, vuol dire che occorre che esista una intersezione dei tempi di vita dei 2 riferimenti
- Usano **2 nomi simbolici di tempo di vita diversi**, vuol dire che i 2 riferimenti non sono correlati e non ho bisogno che abbiano una intersezione dei tempi di vita e dunque una validità contemporanea

Per essere lecito, il tempo di vita **di un riferimento** deve essere contenuto nel tempo di vita **del valore a cui punta**. Il borrow checker si fa carico di garantire tale vincolo. Questo richiede di esplicitare il tempo di vita quando si memorizza un riferimento all'interno di una struttura dati o si usa un riferimento come valore di ritorno di una funzione.

**assert\_eq!()** è una macro che verifica che 2 espressioni siano uguali e se non lo sono genera un **panic** terminando il programma, altrimenti continua il programma. Le regole valgono anche quando si crea un riferimento ad una parte di una struttura dati più grande. Se si memorizzano dei riferimenti in una struttura, tutti questi riferimenti devono avere una durata di vita **maggiore** della struttura dati in cui sono memorizzati. Detto in altri termini, la struttura non può contenere elementi che hanno cessato di esistere

Una **slice** (fetta) è una **vista** di una sequenza contigua di elementi, la cui lunghezza **NON** è nota in fase di compilazione, ma disponibile durante l'esecuzione. Internamente viene rappresentata come una **tupla di due elementi**, il primo dei quali punta al primo valore della sequenza, mentre il secondo indica il numero di elementi consecutivi. Una slice di elementi **T** ha tipo **[T]**. Una slice **non possiede** i dati a cui fa riferimento, questi appartengono sempre ad un'altra variabile, da cui la slice viene derivata. Tutti i valori contenuti sono garantiti essere inizializzati, gli accessi sono verificati in fase di esecuzione. E' possibile ricavare una slice a partire da un array, ma anche da altri tipi di contenitori

```
let a: [i32; 5] = [1, 2, 3, 4, 5];           // a è un array di 5 interi
let s = &a[1..3]                             // s è una slice formata da 2 elementi [2, 3]
let two = s[0]                               // two contiene il valore 2
```

Riassumiamo le **regole del possesso**. Ciascun valore ha un **unico** possessore (variabile o campo di una struttura). Il valore viene rilasciato (drop) quando il possessore esce dal proprio scope o quando al possessore viene assegnato un nuovo valore. Può esistere, al più, un **singolo riferimento mutabile** ad un dato valore. **Oppure**, possono esistere **molte riferimenti immutabili** al medesimo valore, ma fintanto che ne esiste almeno uno, il valore non può essere mutato. Tutti i riferimenti devono avere una **durata di vita inferiore** a quella del valore a cui fanno riferimento.

Un programma Rust, pur non avendo un garbage collector, offre molteplici garanzie relative alla correttezza degli accessi in memoria e del rilascio delle risorse. Non esiste il concetto di **riferimento nullo** e, di conseguenza, non c'è il rischio di dereferenziarlo. Poiché il borrow checker vigila sulle assegnazioni dei riferimenti e sugli intervalli temporali in cui essi sono effettivamente usati impedendo accessi illeciti, non c'è il rischio di originare errori di **segmentazione** o **accesso illegale** ad aree ristrette di memoria, né la possibilità di avere riferimenti ad aree già rilasciate (**dangling pointer**). Poiché in ogni istante il borrow checker è in grado di determinare la dimensione di un

blocco di memoria cui un riferimento punta, non possono verificarsi **buffer overflow** né **buffer underflow**. Per lo stesso motivo, gli **iteratori** offerti da Rust non eccedono mai i **loro limiti**. **Riferimenti corretti nello spazio e nel tempo.**

Tutte le variabili sono **immutabili** per default e occorre una dichiarazione esplicita per renderle mutabili. Questo obbliga il programmatore a riflettere attentamente sul come e dove i dati debbano essere modificati e su quale sia il ciclo di vita di ciascun valore. Il modello di possesso non riguarda solo la gestione della memoria, ma anche la **gestione delle risorse** contenute in un valore.. L'assenza di un garbage collector impedisce **comportamenti non deterministici**, in particolare, la sospensione totale del funzionamento ogni qual volta occorra ricompattare la memoria

## Tipi composti

Spesso occorre mantenere unite informazioni tra loro eterogenee. Sebbene sia possibile utilizzare una tupla, questa tende a nascondere la semantica complessiva del dato. Quando si intende associare una semantica particolare o legare ad una struttura dati un insieme di comportamenti, è possibile introdurre una **struct**. Una **struct** è un costrutto che permette di rappresentare un blocco di memoria in cui sono disposti, consensu tipicamente, una serie di campi il cui nome e tipo sono indicati dal programmatore

```
struct Player {  
    name: String,  
    health: i32,  
    level: u8,  
}
```

Per convenzione, il nome della struct comincia con la lettera **maiuscola** e utilizza la concezione **CamelCase**. Per i campi, come per le variabili, le funzioni ed i metodi, si usa la convenzione **snake\_case**. Si istanzia una struct tramite un blocco preceduto dal nome della struttura contenente un valore per ciascun campo:

```
let mut s = Player { name: "Mario".to_string(), health: 25, level, 1};
```

Si accede ai singoli campi con la notazione puntata `var.field`

```
println!("Player {} has health {}", s.name, s.health);  
s.level += 1
```

Ogni struct introduce un nuovo tipo, il cui nome coincide con il nome della struct, basato sui tipi del cui compongono, questo permette di disambiguare l'uso che si intende fare delle singole informazioni contenute

In una tupla devo mettere i dati nella tupla nell'ordine giusto. In una struct posso mettere i dati **nell'ordine che si preferisce** purché siano presenti **tutti**.

```
#[derive(Debug)]  
struct Player {  
    name: String,  
    health: i32,  
    level: u8,  
}  
  
fn main() {  
    let p1 = Player {name: "Mario".to_string(), level: 1, health: 127};  
    let t1: (i32, f64, &str) = (42, 3.14, "ciao mamma");  
    println!("{:?} {:?}", p1, t1);  
}
```

Si può istanziare una nuova struct a partire da un'altra dello stesso tipo, i campi omessi ricavano i loro valori dalla struct ricevuta.

```
fn main() {
    let p1 = Player {name: "Mario".to_string(), health: 25, level: 1} ;
    let p2 = Player {name: "Giovanni".to_string(), .. p1};
    println!("{:?} \n{:?}", p1, p2);
}
```

Se i campi implementano il tratto **Default** è possibile utilizzare il valore di default.

```
fn main() {
    let p1 = Player::default();
    let p2 = Player {name: "Mario".to_string(), .. Player::default()};
    println!("{:?}\n{:?}", p1, p2);
}
```

Quando il nome del valore coincide con quello del campo è possibile abbreviare la notazione

```
fn main() {
    let name = "Mario".to_string();
    let health = 25; let level = 1;
    let p1 = Player {name, health, level} ;
    println!("{:?}", p1);
}
```

Esistono due tipologie di struct particolari:

- **Tuple struct:** si possono definire delle struct simili a delle tuple, indicando solo il tipo del campo, senza attribuire un nome. Le struct di questo tipo si istanziano come una tupla con l'aggiunta del nome della struct.
- **Unit struct:** si può definire una struct vuota, senza alcun campo, senza allocare memoria, analogamente al tipo (). Permettono di definire dei comportamenti (tratti) su un tipo che non ha dati

La destrutturazione di una struct è utile per ottenere un **parsing** di una struttura, così da gestirne più facilmente i campi, estraendoli in contenitori singoli da trattare separatamente. I campi della struttura generano delle variabili

```
struct Point {
    x: f32,
    y: f32
}

fn main() {
    let p = Point { x: 5., y: 10. };
    let Point { x: ascissa, y: ordinata} = p;
    println!("The original point was: ({},{})", ascissa, ordinata);
}
```

La disposizione in memoria dei singoli campi è scelta arbitrariamente da Rust ed è conseguenza di vincoli ed ottimizzazione e può essere controllata attraverso opportuni meccanismi. Ogni singolo campo, in base al proprio tipo, richiede che l'indirizzo a cui viene collocato sia multiplo di una potenza di 2 (**allineamento**). La funzione **std::mem::align\_of\_val(...)** permette di

conoscere l'allineamento richiesto da un particolare valore, mentre la funzione `std::mem::size_of_val(...)` ne indica la dimensione. I vincoli di allineamento dipendono dalla piattaforma di esecuzione (modello di CPU).

L'allineamento e la disposizione di una struct è controllata attraverso l'attributo `#[repr(...)]` anteposto alla dichiarazione della struct. In assenza di tale attributo, viene assunta la rappresentazione di default, che lascia libero il compilatore di riordinare la sequenza dei campi, per ottimizzare l'accesso. Indicando `#[repr(C)]` si ottiene una rappresentazione coerente con le regole di interfaccia binaria definite dal linguaggio C, fondamentali per l'interoperabilità con librerie scritte in altri linguaggi.

I **moduli** permettono di implementare il meccanismo di **incapsulamento (information hiding)**. Sia la struct nel suo complesso che i singoli campi che la formano possono essere preceduti dal **modificatore di visibilità**. Tale modificatore ha impatto sull'accesso al contenuto della struct da parte di codice presente in moduli diversi da quello in cui la struct è stata definita. Di base, **i campi sono** considerati **privati** (accessibili solo al codice del modulo corrente e ai suoi sottomoduli), però possono essere resi pubblici facendo precedere il nome della parola chiave **pub**.

In Rust ci sono 2 livelli di visibilità:

- Una struct è visibile solo nel modulo in cui è dichiarata o nei sottomoduli
- Aggiungendo il modificatore di visibilità **pub** si permette di far vedere la struct anche **al di fuori del modulo** in cui è dichiarata

```
mod module1 {  
  
    pub struct Test{  
        pub a: i32,  
        pub b: bool }  
    }  
  
    use module1::Test;  
    fn main() {  
        let t = Test { a: 12, b: false};  
        println!("Hello, world!");  
    }  
}
```



I benefici **dell'incapsulamento** sono numerosi:

- **Controllo dell'accesso:** permette di definire un'interfaccia pubblica stabile per un modulo o un tipo, nascondendo i dettagli implementativi interni che potrebbero cambiare
- **Prevenzione di modifiche involontarie:** impedisce al codice esterno di manipolare direttamente lo stato interno di una struttura in modi non previsti, mantenendo l'integrità dei dati
- **Modularità e organizzazione:** aiuta ad organizzare il codice in unità logiche e a ridurre l'accoppiamento tra diverse parti del programma
- **Manutenibilità:** rendendo esplicita l'interfaccia pubblica, è possibile modificare l'implementazione interna senza rompere il codice che utilizza l'interfaccia

Per essere efficace, occorre però poter associare un insieme di **comportamenti (metodi)** alla struct. a differenza di quanto avviene in altri linguaggi, in cui struttura e comportamento sono definite contestualmente in un blocco (classe), in Rust la definizione dei metodi associati ad una **struct** avviene separatamente, in un blocco di tipo **impl**

```
struct Something {  
    i: i32,  
    s: String  
}  
  
impl Something {  
    fn process(&self) {...}  
    fn increment(&mut self) {...}  
}
```

Rust non è un linguaggio ad oggetti nel senso tradizionale del termine e, conseguentemente, **non ha** il concetto di classe. Sebbene le struct possano apparire simili alle classi, il parallelismo è limitato. In particolare, le struct **non** sono organizzate in una gerarchia di **ereditarietà**. Una struct non può estendere un'altra struct, ma ogni struct è **indipendente**. Il concetto di metodo si applica invece a tutti i tipi, compresi quelli primitivi.

Si definiscono i metodi collegati ad un tipo in un blocco racchiuso tra parentesi graffe, preceduto dalla parola chiave **impl** seguita dal nome del tipo. le funzioni presenti in tale blocco il cui primo parametro si **self**, **&self** o **&mut self** diventano **metodi**

I metodi sono funzioni legate ad un'istanza di un dato tipo. Sintatticamente, **un metodo viene invocato a partire da un'istanza** del tipo a cui è legato. SI una la notazione **instance.method(...)**, dove *instance* è una variabile del tipo dato (detto anche **ricevitore**) e *method* il nome della funzione.

Semanticamente, il codice del metodo ha accesso al contenuto (pubblico e privato) del ricevitore attraverso la parola chiave **self**. Posso specificare in modi diversi come il metodo tratta la struttura

che prende in ingresso. Di fatto, i metodi legati ad una struct vengono implementati sotto forma di funzioni con un parametro ulteriore il cui tipo è vincolato alla struct per la quale sono definiti.

Il primo parametro di un metodo definisce il livello di accesso che il codice del metodo ha sul ricevitore:

- **self**: indica che il ricevitore viene passato **per movimento**, di fatto consumando il contenuto della variabile. E' una forma contratta della notazione **self: Self**
- **&self**: indica che il ricevitore viene passato **per riferimento condiviso**. E' una forma contratta di **self: &Self**
- **&mut self**: indica che il ricevitore viene passato **per riferimento esclusivo**. E' una forma contratta di **self: &mut Self**

**Self** rappresenta il tipo che sto implementando. Se presente, il parametro self compare come **primo elemento** nella dichiarazione del metodo. All'atto dell'invocazione del metodo, esso è ricavato implicitamente dal valore che compare a sinistra del punto che precede il nome del metodo. Le funzioni che non hanno come primo parametro **self** sono dette **funzioni associate** e svolgono il ruolo giocato dai costruttori e dati metodi statici nei linguaggi ad oggetti

```
struct Point {
    x: i32,
    y: i32,
}

impl Point {

    fn mirror(self) -> Self {                                // Consuma una struct Point e
        Self{ x: self.y, y: self.x }                          // produce una nuova struct
    }                                                         // stesso tipo

    fn length(&self) -> f64 {                                  // Opera su una struct Point
        senza                                                  // possederla né mutarla
            let a = self.x*self.x + self.y*self.y;
            let res = (a as f64).sqrt();
            res
        }

    fn scale(&mut self, s: i32) {                               // Opera su una struct
        Point                                                  // cambiandone il contenuto
            self.x *= s;
            self.y *= s;
        }
    }
}
```

In **Rust**, è possibile implementare **metodi statici** non indicando, come primo parametro **self**. Sono anche chiamate **associated functions** e sono definite su un tipo anziché su un'istanza di un tipo. Permettono la creazione di funzioni per la costruzione di un'istanza, metodi per la conversione di istanze di altri tipi nel tipo corrente o, semplicemente, l'accesso a funzionalità statiche. la chiamata in questo caso sarà usando **<Tipo>::metodo()**

```
let s = String::from("Hello world!");  
let mut arr = Vec::new();
```

In C++, tutte le classi contengono metodi particolari detti **costruttori**. In Rust, non esiste il concetto di costruttore. Qualunque frammento di codice, in un qualunque modulo che abbia visibilità di una data struct e dei suoi campi, può crearne un'istanza, indicando un valore per ciascun campo. Questo garantisce che il programmatore sia consapevole delle informazioni contenute al suo interno. Per evitare significative duplicazioni di codice e favorire l'incapsulamento, le implementazioni spesso includono metodi statici per l'inizializzazione delle istanze. Per **convenzione**, un metodo di questo tipo viene chiamato:

```
pub fn new(): -> Self {...}
```

Poiché Rust **non supporta l'overloading delle funzioni**, se servono più funzioni di inizializzazione, ciascuna di esse avrà un nome differente: in questo caso la convenzione è utilizzare un pattern come:

```
with_details(...) -> Self {...}
```

Un metodo costruttore è un metodo **senza ricevitore**. In C++, ogni classe prevede un particolare metodo detto **distruttore**. La presenza del distruttore abilita, in C++, un particolare approccio detto **Resource Acquisition Is Initialization (RAII)**. Poiché il distruttore è chiamato automaticamente quando una variabile locale esce dallo scope, si possono usare costruttore e distruttore in coppia per garantire che determinate azioni siano eseguite in un blocco di codice in cui sia presente una variabile locale appositamente dichiarata. Il paradigma RAII viene mutato dal C++ e costituisce un importantissimo modo per gestire automaticamente acquisizione e rilascio di risorse. Per poter implementare correttamente il concetto di RAII bisogna essere sicuri che, per ogni costruzione, ci sia una sola invocazione Drop e per questo motivo, il tratto **Drop** è **mutuamente esclusivo** con il tratto **Copy**. Se un tipo implementa il primo non può implementare l'altro, e viceversa.

Rust gestisce il rilascio di risorse contenute in un'istanza attraverso il tratto **Drop**. Tale tratto è costituito dalla sola funzione **drop(&mut self) -> ()**. Il compilatore riconosce la presenza di questo tratto nei tipi definiti dall'utente e provvederà a chiamare la funzione che lo costituisce quando le variabili di quel tipo escono dal proprio scope sintattico. Si può forzare il rilascio delle risorse contenuti in un oggetto usando la funzione **drop(some\_object)** che ne acquisisce il contenuto e determina l'uscita dallo scope.

In Rust, è possibile introdurre tipi enumerativi composti da un semplice valore scalare, ma anche incapsulare più **tuple** o più **struct** volte a fornire ulteriori informazioni relative allo specifico valore:

```
enum HttpResponse {
    Ok,
    NotFound(String),
    InternalError {
        desc: String,
        data: Vec },
}
```

**Enum** è definito come tipo **somma**. L'insieme dei valori che può contenere è l'unione dei valori delle singole alternative. Struct è un tipo **prodotto**: l'insieme dei valori che può contenere è il prodotto cartesiano degli insiemi legati ai singoli campi. La possibilità di legare, agli specifici valori, uno o più dati è alla base di molti pattern di programmazione tipici di Rust.

La **rappresentazione in memoria** di un enum in Rust dipende dalla natura delle varianti e dalla presenza o meno di dati associati ad esse:

```
enum Color {
    Red,
    Green,
    Blue,
}
```

In un **enum senza dati**, Rust utilizza un'implementazione basata su **tag**, dove ogni variante è rappresentata da un numero intero univoco. In questo caso, la memoria necessaria per rappresentare un'istanza di Color è equivalente a quella necessaria per archiviare un intero.

Rust deve allocare memoria sia per il selettore (**tag**) che identifica la variante, sia per i dati associati alla variante. In memoria gli enum occupano lo spazio di un intero da 1 byte più lo spazio necessario a contenere la variante più grande.

La rappresentazione in memoria è progettata per essere efficiente e compatta e il tag può essere implicito e non rappresentato. La **capacity** di una stringa non può essere negativa. Il primo byte delle altre varianti è usato per rappresentare il tag.

Il costrutto **match** si presta particolarmente per gestire in modo differenziato valori enumerativi. Il comportamento offerto dal pattern matching e la possibilità di legare variabili temporanee in base alla struttura del valore che viene analizzato offre una sintassi compatta ed efficiente per esprimere comportamenti alternativi (**destructuring assignment**). Il fatto che i pattern confrontati debbano essere esaustivi, garantisce che il codice resti coerente anche se il numero di possibili alternative presenti nell'enumerazione cambia nel tempo:

```
fn compute_area(shape: Shape) -> f64 {
    match shape {
        Shape::Square { s } => s*s,
        Shape::Circle { r } => r*r*3.1415926,
        Shape::Rectangle {w, h} => w*h,
```

E' possibile legare **metodi** ad un'enumerazione. Aggiungendo un blocco **impl...** come nel caso delle struct. Il metodo deve essere applicabile a tutte le varianti.

L'utilizzo del pattern matching e la possibilità di destrutturare un valore complesso non sono limitate al costrutto **match**. Quando non è necessario esplorare tutti i casi possibili si possono usare i costrutti:

```
if let <pattern> = value
while let <pattern> = <value>
```

Tali costrutti verificano se il valore fornito corrisponda o meno al pattern indicato e, nel caso, eseguono le necessarie assegnazioni alle variabili contenute nel pattern.

Come verrà meglio presentato in seguito, è possibile definire **tipi generici**, ovvero costrutti che contengono dati il cui tipo è specificato attraverso una meta-variabile, indicata accanto al nome del tipo, racchiusa tra i simboli < e >. I frammenti di codice che utilizzano un tipo generico hanno il compito di indicare quale sia il tipo concreto da sostituire alla meta-variabile. `Vec<T>`, ad esempio, rappresenta un generico vettore di valori omogenei di tipo T. Rust offre due importanti **enumerazioni generiche**, che sono alla base della sua libreria standard:

- **Option<T>**: rappresenta un valore di tipo T opzionale (ovvero che potrebbe non esserci)
- **Result<T, E>**: rappresenta alternativamente un valore di tipo T o un errore di tipo E

**Option<T>** è un tipo di Enum definita nella *standard library* che permette di rappresentare il caso in cui un valore può essere presente o assente. **Option<T>** contiene due possibili valori: **Some (T)**, che indica la presenza e contiene il valore, **None**, che indica che il valore è assente. L'istruzione **match** risulta particolarmente utile con questo tipo di valori.

**Result<T, E>** si usa per indicare l'esito di una computazione. Può valere **Ok (T)** se la computazione ha avuto successo e il valore restituito ha tipo T, oppure **Err (E)** se la computazione è fallita e viene restituito E per descrivere la ragione del fallimento

## Polimorfismo

**Polimorfismo** significa associare comportamenti comuni ad oggetti i cui tipi sono molto diversi tra di loro. Sebbene il processo di analisi di un dominio applicativo tenda a favorire la sua suddivisione in un molteplicità di tipi distinti, la necessità di minimizzare il codice scritto spinge verso l'identificazione di pattern comuni, che possano essere condivisi tra i tipi per unificare la struttura del codice. Questo segue il principio **DRY (Don't Repeat Yourself)**. La soluzione individuata è il **polimorfismo**, ovvero la capacità offerta dai linguaggi di **associare comportamenti comuni ad un insieme di tipi differenti**. Può basarsi sul paradigma della **programmazione generica**, dove è possibile formulare funzioni o strutture dati in cui uno o più tipi non sono specificati per nome, ma tramite simboli astratti, abilitando così la struttura di codice in grado di operare con una molteplicità di tipi. Oppure può sfruttare la definizione di **interfacce comuni**, che possono essere implementate dai singoli tipi: questo consente di fare riferimento all'interfaccia invece che al tipo concreto.

Un **tratto** è una collezione di **metodi** che definiscono un comportamento che un **tipo** può **implementare**. Il tratto esprime la capacità che un tipo offre di avere un certo comportamento. I tratti in Rust costituiscono l'equivalente delle interfacce di Java e C# o delle classi astratte pure in C++. **un tratto definisce un insieme di metodi**, eventualmente associando loro un'implementazione di default. A differenza di quanto accade in C++ o Java, se si invoca su un valore una funzione relativa ad un tratto, non si ha normalmente un costo aggiuntivo (**zero-cost abstraction**), nè gli oggetto che implementano tratti hanno una penalità in termini di memoria per ospitare il puntatore alla VTABLE. Tale costo si presenta solo quando si crea esplicitamente un riferimento dinamico (**&dyn TraitName**) quando chiedo al compilatore di dimenticarsi del tipo, ma di considerare solo in virtù del tratto che implementa. Si definisce un tratto con la sintassi:

```
trait SomeTrait { fn someOperation(&mut self) -> SomeResult; ... }
```

Una struttura dati concreta, come struct od enum, può esplicitamente dichiarare di implementare un dato tratto attraverso il blocco seguente:

```
impl SomeTrait for SomeType { ... }
```

**Dato un valore il cui tipo implementa un tratto**, è possibile **invocare su tale valore i metodi del tratto**, con la **normale** sintassi basati sul `.`, a condizione che il tratto sia stato dichiarato nello stesso crate o che sia stato importato attraverso il costrutto **use**

**SomeNamespace::SomeTrait;** . Alcuni tratti (come `Clone` e `Iter`) non necessitano di essere importati esplicitamente in quanto fanno parte di una porzione di codice della libreria standard (**`std::prelude`**) che viene importato automaticamente in ogni crate. L'implementazione del tratto può essere fatta o dove viene definito il tratto o dove viene definita la struttura dati.

Un metodo è una funzione che utilizza come primo parametro la parola chiave **`self`** o una sua variazione (**`&self`**, **`&mut self`**, ...). Il tipo del parametro `self` può anche essere **`Box<Self>`**, **`Rc<Self>`**, **`Arc<Self>`**, **`Pin<Self>`**. I metodi sono invocati con l'operatore `.` sul ricevitore su cui è applicato il metodo.

Se una delle funzioni definite da un tratto non usa, come primo parametro, nè **`self`** nè un suo derivato, questa non è legata all'istanza del tipo che la implementa e può essere invocata usando come prefisso il nome del tratto o il nome del tipo che la implementa.

Un tratto può avere uno o più **tipi associati** (oltre al tipo `Self` specifico del tratto), **nell'implementazione** specifica a quale tipo si riferisce (detto **tipo concreto**). Questo permette alle funzioni del tratto di fare riferimento, in modo astratto, a tali tipi che dovranno essere poi specificati nel contesto del tipo che implementa il tratto stesso. Firma generica comune, le singole implementazioni forniscono i dettagli sul tipo.

Nella definizione di un tratto è lecito indicare, per una data funzione, **un'implementazione di default**. Le funzioni che implementano il tratto saranno libere di adottarla o potranno sovrascriverla con altro codice (**override**), purché venga rispettata la firma delle funzioni (tipo dei parametri e dei valori di ritorno). Questo è particolarmente comodo in quelle situazioni in cui un dato metodo può essere implementato in funzione di altri metodi del tratto.

La notazione **`trait Subtrait: Supertrait {...}`** indica che i tipi che implementano **`Subtrait`** devono implementare anche **`Supertrait`**. Le due implementazioni sono tra loro indipendenti. Il sottotragto include tutte le funzionalità del super tratto e ne può aggiungere di proprie.

Invece di dover scrivere esplicitamente il tipo concreto è possibile indicare solo il tratto che il tipo deve soddisfare. Funzioni che accettano o restituiscono un tipo che implementa uno (o più con la notazione `+`) tratti specifici. Quando il compilatore incontra una funzione con parametro **`impl Trait`**, sta generando una funzione generica **monomorfizzata**, ossia per ogni tipo concreto con cui la funzione viene chiamata, il compilatore crea una versione specializzata della funzione con quel tipo concreto specifico al posto di `impl Trait`

L'invocazione (**dispatching**) dei metodi di un tratto può avvenire in due modalità distinte:

- **Invocazione statica:** se il tipo del valore è noto, il compilatore può identificare l'indirizzo della funzione da chiamare e generare il codice corrispondente senza alcuna penalità
- **Invocazione dinamica:** se si dispone di un **puntatore** ad un valore di cui il compilatore sa solo che implementa un dato tratto, occorre eseguire una chiamata indiretta, passando per una `VTABLE`.

Variabili o parametri destinati a contenere puntatori ad un valore che implementa un tratto sono annotati con la parola chiave **dyn**.

I riferimenti/puntatori ai tipi tratto vengono detti **oggetti-tratto**. Possono essere condivisi o mutabili e devono rispettare le regole dell'esistenza in vita del valore a cui fanno riferimento. Gli oggetti-tratto vengono implementati tramite **double pointer**. Per ogni tratto che definisce oggetti-tratto sono allocate un numero di `VTABLE` pari al numero di tipi concreti che implementano il tratto.

I requisiti per un tratto per poter essere utilizzato come oggetto tratto deve avere tutte le firme dei metodi che soddisfano le seguenti condizioni:

- Assenza di parametri di tipo generici
- Self non deve essere obbligatoriamente un tipo di dimensione statica e nota a livello di compilazione (**Sized**)
- Self non deve essere usato per valore in nessun parametro, né per il valore di ritorno

Un tratto può diventare oggetto-tratto se i suoi metodi possono essere chiamati senza la necessità di conoscere la dimensione statica specifica del tipo che implementa il tratto, questo è necessario perché gli oggetti tratto sono rappresentati esclusivamente da un puntatore al dato e da un puntatore alla `VTABLE`.

Un tipo è **Sized** se la sua dimensione è nota a livello di compilatore. La maggior parte dei tipi in Rust sono `Sized`, tra cui: tipi primitivi, array, puntatori. Alcuni tipi non hanno una dimensione nota al momento della compilazione. I più comuni sono: slice, oggetti tratto e string slice. Il vincolo **Sized** è spesso implicito. Se non si specifica diversamente, Rust assume che i parametri di tipo siano `Sized`. Se si vuole operare con un tipo che può essere **UnSized** occorre utilizzare la notazione **?Sized**. Se si vogliono accettare anche tipi non `Sized`, si deve rimuovere il vincolo e usare un puntatore.



Rust definisce un insieme di tratti base la cui eventuale implementazione da parte di un dato tipo abilita una serie di scorciatoie sintattiche legate agli operatori presenti nel linguaggio, sostituendo a tutti gli effetti il meccanismo di **operator overloading** presente in C++. E' possibile confrontare due istanze di un dato tipo con gli operatori `==` e `!=` se il tipo implementa i tratti **Eq** o **PartialEq**. E' possibile un confronto basato su `<`, `>`, `<=` e `>=` se il tipo implementa i tratti **Ord** o **PartialOrd**. Gli operatori binari `+`, `-`, `*`, `/`, `%`, `&`, `^`, `<<`, `>>` sono rispettivamente associati ai tratti **Add**, **Sub**, **Mul**, **Div**, **Rem**, **BitAnd**, **BitXor**, **Shl**, **Shr**. Le operazioni unarie `-` e `!` corrispondono a **Neg** e **Not**.

Il tipo di Output definisce il tipo del risultato dell'operazione. Il parametro generico **Rhs** (**Right Hand Side**) permette di definire operazioni tra tipi diversi (di default è `Self`). Il tipo di **Rhs** non è per default `Self`, ma dipende dal contesto.

Implementare un confronto di uguaglianza richiede il puntuale rispetto di alcune proprietà logiche essenziali: **riflessività**, **simmetria**, **transitività**. Rust introduce due tratti per esprimere questa capacità. **PartialEq** ed **Eq**, entrambi definiti dai metodi `eq(&self, other: &RHS) -> bool` e `ne(&self, other: &RHS) -> bool`.

**PartialEq** richiede che siano garantite, dall'implementazione, sia la proprietà simmetrica sia quella transitiva. **Eq** (che è un sotto-tratto di **PartialEq**) impone anche il rispetto della proprietà riflessiva: i tipi floating-point (`f32` e `f64`) non implementano questo tratto (`NaN != NaN`). Il tratto **Eq** è un tratto senza metodi (tratto marker) e la sua dichiarazione solo per specificare che l'uguaglianza gode anche della proprietà **riflessiva**. Il tipo associato **RHS** (**Right Hand Side**) definito, per default, come `Self`, in rari casi può essere necessario permettere confronti tra tipi differenti. Il metodo `ne(...)` è normalmente preso dalla sua implementazione di default, come opposto del risultato di `eq(...)`

Il parametro generico **Rhs** permette di effettuare un confronto tra tipi diversi e può essere un tipo di dimensione ignota al compilatore. Il tipo di **Rhs** deve essere `Self`, per rispettare il vincolo di riflessività.

Sebbene sia possibile implementare a mano tutti i metodi richiesti da un certo tratto, a volte è meglio affidarsi al compilatore. Se la definizione di una struct o di una enum è preceduta dall'attributo `#[derive(...)]`, il compilatore provvede ad aggiungere automaticamente l'implementazione dei tratti indicati all'interno del costrutto `derive(...)`. Solo un certo sottoinsieme di tratti possono essere generati automaticamente, spesso a condizione che i tipi dei dati contenuti nel tipo per cui si esegue la derivazione soddisfino opportuni vincoli.

I tratti **PartialOrd** e **Ord** permettono rispettivamente di definire relazioni d'ordine parziali e totali su un dato insieme di valori. E' possibile confrontare due valori per determinare se uno è minore di, maggiori di, o uguale all'altro, ma non è garantito che tutti i valori possano essere confrontati tra loro. Gli altri metodi (`lt`, `le`, `gt`, `ge`) forniscono implementazioni predefinite basate sul risultato di `partial_cmp`.

E' possibile utilizzare **sintatticamente** una struttura dati come fosse un array, abilitando la notazione `t[i]` se si implementano i tratti **Index** e **IndexMut**. L'espressione `t[i]` viene riscritta dal compilatore come `*t.index(i)`, se si accede in lettura al risultato dell'espressione, o come `*t.index_mut(i)`, se si accede in scrittura. Questi due tratti sono definiti nel seguente modo:

```
trait Index {
    type Output: ?Sized;
    fn index(&self, index: Idx) -> &Self::Output;
}
trait IndexMut: Index {
    fn index_mut(&mut self, index: Idx) -> &mut Self::Output;
}
```

Molti tipi della libreria standard implementano i tratti **Index** e **IndexMut**. Poichè sia l'indice usato che il tipo associato restituito dall'operazione di indicizzazione sono parametrizzati, è lecito avere implementazioni multiple del tratto per un dato tipo.

I tratti **From** e **Into** permettono di effettuare conversioni di tipo, prendono possesso del valore, lo convertono e ritornano il possesso al chiamante. I tratti sono perfettamente duali, scrivere **T: From<i32>** equivale a scrivere **i32: Into<T>**. Se si implementa il tratto **From**, il tratto **Into** viene generato automaticamente. L'implementazione del tratto **From** **non è simmetrica**: se è possibile passare dal tipo T al tipo U, non è affatto detto che sia possibile tornare indietro dal tipo U al tipo T:

```
trait From: Sized {
    fn from(other: T) -> Self;
}
trait Into: Sized {
    fn into(self) -> T;
}
```

Per gestire le conversioni tra tipi che possono fallire vengono forniti i tratti **TryFrom** e **TryInto** che posseggono le stesse proprietà di **From** e **Into**. I metodi `try_from` e `try_into` ritornano il tipo **Result<T, E>** per garantire la gestione dell'errore:

```
pub trait TryFrom: Sized {
    type Error;
    fn try_from(value: T) -> Result;
```

```

}
pub trait TryInto: Sized {
    type Error;
    fn try_into(self) -> Result;
}

```

Il tratto **FromStr** permette di gestire la conversione da stringa e l'eventuale fallimento. Il metodo **from\_str()** viene implicitamente richiamato tutte le volte che si utilizza il metodo **parse()**:

```

pub trait FromStr {
    type Err;
    fn from_str(s: &str) -> Result;
}

```

E' possibile trattare una struttura dati come se fosse sintatticamente un puntatore implementando i tratti **Deref** e **DerefMut**. La sintassi **\*t** per un tipo che implementa **Deref** e che non sia un riferimento né un puntatore nativo, equivale a **\*(t.deref())** e restituisce un valore immutabile di tipo **Self::Target**. Analogamente, se il tratto implementa **DerefMut**, **\*t** equivale a **\*(t.deref\_mut())** e restituisce un valore mutabile di tipo **Self::Target**:

```

trait Deref {
    type Target: ?Sized;
    fn deref(&self) -> &Self::Target;
}
trait DerefMut: Deref {
    fn deref_mut(&mut self) -> &mut Self::Target;
}

```

Il tipo **Self::Target** deve essere qualcosa contenuto, posseduto o referenziato in **Self**. La notazione **\*t** prende a prestito, in modo condiviso o esclusivo, il valore **t** per poter eseguire il metodo del tratto corrispondente. I riferimenti restituiti da **deref(&self)** o da **deref\_mut(&mut self)** prolungano, come conseguenza delle regole sul tempo di vita, il prestito di **self** fino al termine del loro utilizzo.

I due tratti svolgono anche un secondo ruolo: permettono la **conversione automatica** dal tipo **&Self** al tipo **&Self::Target** e da **&mut Self** a **&mut Self::Target**. Questa proprietà viene detta **deref coercion**. Si applica a:

- **Smart Pointer**: puntatori che puntano ad un dato nello heap, per default
- **Tipi Wrapper**: tipi che incapsulano un dato, se implementano il tratto **Deref**

Quando il compilatore Rust si aspetta un riferimento di tipo **&U** (o **&mut U**) ma riceve un **&T** (o **&mut T**), esso verifica se **T** implementa **Deref<Target = U>** (o **DerefMut<Target = U>**). Se lo fa, il compilatore inserisce implicitamente una o più chiamate al metodo **deref** (o **deref\_mut**) per ottenere il riferimento desiderato.

Nel caso di tipo puntatore ad un dato nello heap il compilatore esegue la **deref coercion**, chiamando il metodo `deref` e restituendo il tipo `Target`. La `deref coercion` permette di utilizzare i tipi in modo più flessibile. Tramite la `deref coercion` un `&String` viene automaticamente convertito in `&str`. Si possono così applicare alle `String` tutti i metodi che lavorano su un `&str`.

Macro come **`println!`** e **`format!`** consentono di stampare un valore associato al segnaposto **`{}`** a condizione che tale valore implementi il tratto **`Display`**. Tale tratto rappresenta la capacità del tipo di creare una visualizzazione di un proprio valore comprensibile ad un **utente finale**. Da un punto di vista pratico, non può essere derivato. La sua implementazione richiede la definizione di un solo metodo:

```
trait Display {  
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result;  
}
```

Il tratto **`Debug`** ha la stessa firma di **`Display`**, il suo scopo tuttavia è differente: creare una rappresentazione comprensibile al programmatore del valore. Viene attivato usando il segnaposto **`{:?}`**. Può essere sintetizzato automaticamente attraverso la notazione **`#[derive(Debug)]`**.

Il tratto **`Clone`** indica la capacità di un tipo di creare un duplicato di un valore dato. Questo può essere usato per trasformare un riferimento `&T` in un valore posseduto `T`. Tale trasformazione, ovviamente, può essere molto costosa in termini di tempo e memoria, tuttavia è sempre sotto il controllo del programmatore perché tale funzionalità non è mai attivata automaticamente.

Il tratto **`Copy`** è un sotto-tratto di **`Clone`**. Esso implica il fatto che la copia ottenuta sia, a tutti gli effetti, il solo duplicato dei bit presenti nel valore originale, senza nessuna trasformazione. La sua implementazione può essere veloce ed efficiente. Questo tratto può essere implementato solo da strutture dati i cui campi implementino il tratto stesso. La presenza del tratto **`Copy`** trasforma la semantica delle operazioni di assegnazione. Quello che normalmente determina un **movimento**, che rende inaccessibile il valore originale, diventa una **copia**. **`Copy`** è in mutua esclusione con **`Drop`**.

Il tratto **`Drop`** permette di definire il metodo **`drop(&mut self)`** che fa le veci del distruttore nel linguaggio C++. Il compilatore Rust garantisce che tale metodo sarà chiamato nel momento in cui la variabile che contiene un valore di questo tipo uscirà dal proprio scope sintattico o subito prima che le venga assegnato un nuovo valore. Questo permette di rilasciare eventuali risorse possedute dalla struttura dati o abilitare comportamenti duali a quelli della creazione dell'oggetto, tipici del pattern **`RAII (Resource Acquisition Is Initialization)`**. Questo tratto è mutuamente esclusivo con il tratto **`Copy`**. Questo vincolo elimina, in assenza di blocchi **`unsafe { ... }`**, la

possibilità di avere un doppio rilascio delle risorse presenti sullo heap. Questo tratto si accompagna alla funzione globale **fn drop<T>(\_x:T) {}**, essa forza il passaggio di possesso di un valore ad una nuova variabile (**\_x**) che uscirà di scena subito, provocandone la distruzione.

Attraverso l'utilizzo dell'operatore **..** è possibile definire intervalli di valori per tutti i tipi che implementano il tratto **RangeBounds<T>**. Quest'ultimo è implementato da diversi tipi base in Rust e consente l'utilizzo di sintassi come **..a**, **..b**, **..=c**, **d..e**, **f..=g**. E' necessario implementare i metodi **end\_bound(&self)** e **start\_bound(&self)** che ritornano entrambi il tipo **Bound<&T>**.

In Rust gli errori vengono ritornati attraverso l'utilizzo del tipo **Result<T, E>**. Il tipo generico **E** può assumere qualsiasi valore tuttavia è preferibile utilizzare solo tipi che implementano il tratto **Error**. Per implementare **Error**, un tipo deve semplicemente implementare il tratto **Display**. Questo perché la descrizione dell'errore è ora considerata parte della sua rappresentazione formattata:

```
pub trait Error: Debug + Display {
    fn source(&self) -> Option<&(dyn Error + 'static)> { ... }
}
```

Il metodo **source** è pensato per permettere di tracciare la sequenza di fallimenti.

Un sistema di tipi stringente facilita la creazione di codice robusto, identificando in fase di compilazione quei frammenti di codice che violano il sistema dei tipi. In certe situazioni, per non violare il sistema dei tipi, occorre replicare una grande quantità di codice generando problemi in fase di manutenzione. Occorre, infatti, garantire che eventuali modifiche ad una versione del codice vengano propagate a tutte le altre. Sia il C++ che Rust permettono di estendere il sistema dei tipi utilizzando una forma di meta-programmazione (detta **template programming** in C++ e **generics** in Rust) grazie alla quale è possibile descrivere strutture dati e funzioni che contengono dati il cui tipo è rappresentato da una meta-variabile, permettendo così di esprimere dei concetti più generali.

In C++, come in Rust, si può definire una funzione in modo che operi su un tipo di dato non ancora precisato:

```
fn max(
    t1: T,
    t2: T) -> T where T: Ord {
    return
        if t1 < t2 { t2 }
        else { t1 }
}
```

In Rust si possono usare costrutti generici per generalizzare i parametri/tipo di ritorno di una funzione o per definire un tipo composto basato su parti variabili. Si definisce un **tipo generico** basato sulle meta-variabili `T`, `U`, `V...`, indicano tali all'interno di parentesi angolari dopo il nome del tipo. Ciascuna meta-variabile può essere soggetta ad eventuali restrizioni, indicate come l'insieme dei tratti che deve implementare e/o come il tempo di vita e deve garantire. Se una struttura generica implementa dei metodi, i nomi delle meta-variabile ed i vincoli a cui sono soggette vengono ripetuti nel blocco `impl`:

```
struct MyStruct<T> where T: SomeTrait {
    foo: T,
    //...
}
impl<T> struct MyStruct<T>
    where T: SomeTrait {
    fn process(&self) { ... }
}
```

Quando il compilatore incontra la definizione di tipi/funzioni generici, **si limita a verificare formalmente la coerenza** del costrutto, senza generare alcun codice. Se in qualche parte del programma si utilizza un tipo/funzione generico legando le meta-variabili in esso contenute a tipi concreti, il costrutto generico viene **istanziato**, espandendo la definizione iniziale con i necessari dettagli necessari a generare il codice relativo. Se il costrutto generico, in parti diverse del programma, è legato a tipi concreti differenti, il compilatore genera ulteriori espansioni della definizione che, pur avendo una matrice comune, risulteranno indipendenti tra loro. Questo processo prende il nome di **monomorfizzazione**. L'uso di tipi generici facilita il processo di astrazione da parte del programmatore, affidando al compilatore il compito di rifinire i dettagli necessari a gestire specifici tipi di dato.

Rust offre due modi per specificare i vincoli sui tipi generici. Una versione compatta **<T: SomeTrait>** e una versione estesa **<T> ... where T: SomeTrait**, posta dopo la lista dei parametri della funzione. In entrambi i casi, se è necessario indicare che il tipo deve implementare più tratti, questi possono essere combinati con il segno **+**.

Sebbene tratti e tipi generici siano due modi per implementare il polimorfismo, tra i due esiste un legame molto più profondo. Uno o più tratti possono essere usati come vincoli per limitare l'utilizzo di un tipo generico ai soli casi in cui ha senso farlo. Si può definire un **tratto generico**, i cui metodi, cioè, ricevono o restituiscono valori generici, eventualmente vincolati da ulteriori tratti.

Occorre comprendere le somiglianze e le differenze tra una funzione non generica che opera su oggetto.tratto e un'altra generica il cui parametro è vincolato da un tratto e scegliere quando usare una forma o l'altra:

```
fn dynamic_process(w: &mut dyn Write) { ... }  
fn generic_process(w: &mut T) where T:Write { ... }
```

Gli oggetti-tratto richiedono l'uso di un **double pointer** per permetterne l'accesso, ma non richiedono la duplicazione del codice dovuta al processo di monomorfizzazione. L'uso di strutture generiche, in generale, porta a codice più efficiente, non solo perché le chiamate alle funzioni non necessitano di transitare per la VTABLE, ma perché il compilatore, conoscendo il tipo concreto in fase di monomorfizzazione, può generare codice più compatto, valutando il risultato dell'elaborazione delle parti costanti in fase di compilazione e sfruttare tecniche di **code inlining** per ridurre l'impatto dell'invocazione di funzioni. Non tutti i tratti permettono di definire oggetti-tratto. Non è possibile definire un oggetto-tratto legato a più tratti disgiunti, mentre è possibile, in una funzione generica, vincolare una meta-variabile ad implementare più tratti.

## Lifetime

Se una funzione riceve un parametro di tipo riferimento (mutabile o meno), il tempo di vita del riferimento diventa parte integrante della firma della funzione. E' necessario infatti che tutte le operazioni eseguite dalla funzione sul riferimento **siano compatibili** con la validità del dato memorizzato al suo interno:

```
fn f(p: &i32) { ... } ⇒ fn f<'a>(p: &'a i32) { ... }
```

Il compilatore provvede in molti casi ad effettuare autonomamente la riscrittura indicata senza il bisogno dell'intervento del programmatore (**lifetime elision**).

Uno dei casi più frequenti in cui il compilatore non riesce a dedurre il corretto tempo di vita è legato a funzioni che **restituiscono un riferimento**, estratto da una delle strutture dati ricevute in ingresso. Se una funzione ha due o più riferimenti in ingresso e un riferimento in uscita, il compilatore non è in grado di decidere (senza eseguire il codice) da quale parametro provenga il riferimento in uscita. L'inserimento degli identificatori nella firma della funzione risolve questa ambiguità. Occorre **annotare** il tipo restituito con la corretta **etichetta**.

Se la funzione riceve **più riferimenti**, può essere necessario indicare se il loro tempo di vita sia vincolato al più breve o se siano disgiunti. Abbiamo due casi:

- **Si usa un solo identificatore:**

- `fn f<'a>(p1: &'a i32, p2:&'a i32) -> &'a i32 { ... }`

- **Si usano etichette diverse:**

- `fn f<'a, 'b>(p1: &'a i32, p2:&'b i32) -> &'a i32 { ... }`

Le **annotazioni** del tempo di vita equivalgono ad **etichette** che sono applicate ai riferimento per dire al compilatore che un riferimento vivrà almeno quanto quest'altro riferimento. Non cambiano la durata effettiva dei dati a cui i riferimenti puntano, ma aiutano il compilatore a verificare che tutti i riferimenti siano sempre validi e non diventino **pendenti (dangling)** puntando a memoria deallocata.

Rust, per default, provvede ad assegnare, ad ogni riferimento presente tra i parametri di una funzione (o in una struttura dati) un tempo di vita distinto:

- Una funzione con **un parametro** ha un parametro di lifetime (`'a`)
- Una funzione con **due parametri** ha due parametri di lifetime (`'a` e `'b`)



Tali tempi di vita si propagano al codice che fa uso di tale struttura dati e, in assenza di ambiguità, non richiede di esplicitare gli identificatori dei tempi di vita.

Elisione dei tempi di vita:

1. Se c'è un solo parametro di lifetime in input, quel parametro è assegnato a tutti i parametri di lifetime di output.
2. Non ci sono riferimenti in output
3. Se ci sono più parametri di lifetime di ingresso, ma uno di loro è `&self` o `&mut self` (poiché si tratta di un metodo), il parametro di lifetime di `self` è assegnato a tutti i parametri di lifetime di output. Questo parte dal presupposto che se un metodo di un oggetto restituisce un dato preso in prestito, questo sia stato preso dai dati posseduti dall'oggetto stesso

Le **annotazioni** devono essere riportate solo nella signature della funzione e non nel corpo della funzione. Se il riferimento restituito dipende solo da un parametro, occorre specificare l'annotazione solo per il parametro per cui c'è la dipendenza. Nel caso in cui la funzione sia generica, le meta-variabili di tipo vengono indicate dopo gli identificatori del tempo di vita.

Se la funzione restituisce un riferimento ad una variabile locale che contiene una stringa costante, bisogna riportare l'annotazione di lifetime **`'static`** che indica che il riferimento restituito ha una durata statica, cioè per l'intera durata del programma:

```
fn crea_stringa() -> &'static str {
    let s = "hello";
    s
}
fn main() {
    let s = crea_stringa();
    println!("{}", s);
}
```

Lo scopo degli identificatori relativi al ciclo di vita è **duplice**:

- Per il codice che **invoca** la funzione (chiamante), essi indicano su **quale**, tra gli indirizzi in ingresso, è basato il risultato in uscita.
- Per il codice **all'interno** della funzione (chiamata), essi garantiscono che vengano restituito solo indirizzi cui è **lecito accedere** per (almeno) il tempo di vita indicato.

All'atto dell'invocazione, gli identificatori forniti dal programmatore (o inseriti automaticamente dal compilatore, quando possibile) sono legati all'effettivo intervallo minimo (espresso come insieme di linee di codice) nel quale il valore da cui il prestito è stato preso debba restare bloccato per non violare le assunzioni su cui la funzione è basata. Tentativi di modificare il valore originale da

cui il prestito è preso prima che il tempo di vita sia trascorso portano ad errori di compilazione che costituiscono la base della robustezza del sistema di possesso e prestito offerto da Rust.

Se la funzione riceve come parametri un riferimento ad una struttura dati contenente un riferimento ed un altro riferimento, il compilatore non può inferire la relazione tra **il tempo di vita della struttura** in cui il riferimento è memorizzato e **il riferimento stesso**. Occorre specificare al compilatore se esiste una relazione tra il riferimento passato come parametro e quello contenuto nella struttura dati (caso più frequente) oppure non esiste alcuna relazione.

Lo stesso tipo di ragionamento vale se il dato viene salvato all'interno di una qualsiasi struttura dati. Rust verifica che il valore a cui si punta abbia un tempo di vita maggiore o uguale del tempo di vita della struttura dati. Questo richiede di esplicitare il tempo di vita della struttura rispetto al tempo di vita dei riferimenti in essi contenuti. Un riferimento che cessa di vivere si propaga a tutti i contenitori.

Se una struttura che contiene riferimenti è contenuta, a sua volta, in un'altra struttura, anche quest'ultima deve avere il tempo di vita specificato in modo esplicito. Nel caso di riferimenti inseriti in strutture dati, se la definizione del metodo permette di ricavare in modo implicito la dipendenza dei tempi di vita, si può utilizzare l'annotazione **anonymous lifetime** (<' \_>)

## Chiusure

La **programmazione funzionale** è un paradigma di programmazione che descrive che cosa deve essere fatto invece di specificare come farlo. Si contrappone alla programmazione imperativa. Tratta il calcolo come la valutazione di funzioni matematiche. Le caratteristiche principali della programmazione funzionale includono:

- **Funzioni pure:** le funzioni non hanno effetti collaterali e producono sempre lo stesso output con gli stessi input
- **Immutabilità:** i dati non possono essere modificati dopo la creazione, ma si creano nuove strutture dati
- **Funzioni di ordine superiore:** le funzioni possono ricevere altre funzioni come argomenti e restituire funzioni come risultato
- **Ricorsione:** utilizzata al posto dei cicli iterativi
- **Valutazione pigra:** le espressioni vengono valutate solo quando necessario

Rust non è un linguaggio di programmazione puramente funzionale, ma offre molti concetti e tecniche della programmazione funzionale. Molti linguaggi offrono la possibilità di trattare una funzione come un tipo dati qualsiasi, consentendo di memorizzarla in una variabile. In Rust è possibile assegnare ad una variabile il puntatore ad una funzione (che avrà come tipo **fn (T<sub>1</sub>, ..., T<sub>n</sub>) -> U**) o assegnare un valore che implementa un tratto funzionale: **FnOnce**, **FnMut**, **Fn**.

Qualunque sia la natura del dato assegnato, si utilizza la variabile che contiene il puntatore come se fosse essa stessa una funzione. Sia il tipo di ritorno che tutti i tipi degli argomenti devono corrispondere a quanto dichiarato nella definizione della variabile (e della funzione):

```
fn f1(i: i32, d: f64) -> f64 {
    return i as f64 * d
}
fn main() {
    let ptr = f1;                                // variabile puntatore a funzione
    let num = ptr(2, 3.14);
    println!("{}", num);
}
```

Una **funzione lambda** è una funzione anonima costituita da un blocco di codice espresso in forma letterale. Tale forma dipende dal linguaggio di programmazione. In Rust si racchiudono i parametri formali tra **| |**:

```
let f = | v | { v + 1 }
```

Una funzione lambda, una volta **definita** ed **assegnata** ad una **variabile**, può essere invocata trattando **la variabile come se fosse una funzione**. Ovvero facendo seguire, al nome della variabile la lista degli argomenti racchiusi in parentesi tonde:

```
fn main() {  
    let answer = | v | { v + 1 };  
    println!("The answer is: {}", answer(5));  
}
```

Il corpo di una funzione lambda può fare riferimento alle variabili che sono visibili nel contesto in cui è definita, acquisendone un riferimento, una copia o il possesso completo, in base alla sintassi usata. Tali variabili, che compaiono nel corpo della funzione lambda, sono dette **variabili libere**. La funzione lambda così ottenuta viene detta **chiusura**, in quanto racchiude, al proprio interno, una copia dei valori catturati, rendendoli disponibili quando sarà successivamente invocata.

Facciamo un po' di chiarezza sulla terminologia:

- **Funzioni di ordine superiore:** funzioni i cui parametri e/o il cui tipo di ritorno sono a loro volta una funzione
- **Funzione anonima:** funzione definita senza un nome e il loro uso è diretto (ossia la dichiarazione viene scritta dentro il codice). Il compilatore è in grado di inferire i tipi degli argomenti e del valore di ritorno
- **Funzione lambda:** sinonimo di funzione anonima
- **Chiusura:** funzione lambda che può catturare variabili (dette **variabili libere**) dell'ambiente circostante

Per default in Rust, tutte le variabili libere che compaiono nel corpo di una funzione lambda sono **catturate per riferimento**. Il compilatore, automaticamente, crea un prestito in lettura (**&**). Se occorre modificare il contenuto delle variabili catturate, acquisendo con **&mut**, occorre dichiarare la funzione lambda come mutabile con **let mut f = | ... | { ... };**. Il borrow checker verifica che tali riferimenti siano coerenti tra loro e con il tempo di vita dei valori cui si riferiscono. Se occorre, è possibile indicare che la funzione lambda deve acquisire il **possesso dei valori** contenuti nelle variabili libere. Lo si fa antepoendo alla definizione della funzione lambda la parola chiave **move**, ad esempio: **let f = move | ... | { ... };**. Il compilatore può inferire che la variabile libera viene mossa e consumata.

In Rust, il compilatore trasforma una chiusura in una **tuple struct**, avente campi quante sono le variabili libere. Le variabili libere possedute (con **move**) vengono mosse all'interno della **struct**. Le variabili libere non possedute copiano il riferimento all'interno della **struct**.

Rust definisce **tre tratti funzionali** che possono essere implementati **solo** tramite chiusure. Quale tratto venga implementato, dipende da **cosa** e **come** viene catturato:

```
trait FnOnce {
    type Output;
    fn call_once(self, args: Args) -> Self::Output;
}
trait FnMut: FnOnce {
    fn call_mut(&mut self, args: Args) -> Self::Output;
}
trait Fn: FnMut {
    fn call(&self, args: Args) -> Self::Output;
}
```

Il compilatore di Rust inferisce automaticamente quale tratto o quali tratti funzionali una closure implementa analizzando cosa fa la closure con le variabili catturate:

- Se la closure muove una variabile catturata e la consuma, implementa solo **FnOnce**
- Se la closure muta una variabile catturata per riferimento mutabile senza consumarla, implementa **FnMut** e **FnOnce**
- Se la closure prende in prestito le variabili o non cattura nulla, implementa **Fn**, **FnMut** e **FnOnce**

La gerarchia segue il principio che ogni closure implementa **FnOnce** (e può essere chiamata 1 volta), mentre **FnMut** e **Fn** dipendono dal tipo di accesso alle variabili catturate.

Una chiusura che implementa il tratto **Fn<Args>** si limita ad accedere in sola lettura alle variabili libere (**&**). Finché la funzione esiste, le variabili libere sono in prestito condiviso e non possono essere cambiate. Pertanto, la chiusura può essere invocata un numero qualsiasi di volte e produce, a parità di argomenti, sempre lo stesso risultato.

Una chiusura che implementa il tratto **FnMut<Args>** può essere invocata più volte, ma ha catturato una o più variabili in modo esclusivo (**&mut**). Questo tipo di chiusura produce effetti collaterali ed ha pertanto uno **stato**. Esecuzioni successive con gli stessi parametri in ingresso possono dare risultati differenti.

Una chiusura implementa il tratto **FnOnce<Args>** se consuma uno o più valori come parte della propria esecuzione. Pertanto, potrà essere invocata una sola volta.

E' possibile implementare una funzione che accetta come parametro una funzione lambda ricorrendo alla programmazione generica. Ogni funzione lambda forma infatti un tipo a sé, la cui unica istanza è la funzione lambda stessa. Tuttavia, esse implementano tutte almeno uno dei tratti funzionali: è quindi possibile definire una funzione generica che accetta come parametro di

ingresso un'istanza del tipo `F`, soggetta al vincolo che tale tipo deve implementare uno dei tratti funzionali. Quale specifico tratto richiede, dipende dalla natura del codice che si intende:

- **`FnOnce(Args)`** accetterà qualsiasi chiusura, ma questa potrà essere invocata una sola volta
- **`FnMut(Args)`** e **`Fn(Args)`** sono via via più restrittivi sulle operazioni che è lecito eseguire all'interno della funzione lambda

```
fn higher_order_function(f: F) where F: Fn(T) -> U {  
    // ... codice che usa f(...)
}
```

Analogamente, è possibile scrivere **una funzione che restituisce una chiusura**. Tale tipo di funzione di ordine superiore è detta **factory di funzione**. Se la chiusura ritornata cattura qualche variabile, occorre fare attenzione al fatto che ciò implica memorizzare un riferimento all'interno della chiusura stessa. Questo fatto potrebbe imporre restrizioni sul tempo di vita del dato catturato non facilmente compatibili con il fatto che la chiusura ritornata deve sopravvivere alla funzione stessa. In questa situazione, si usa comunemente il modificatore **`move`** per trasferire il possesso di tali dati alla chiusura stessa. Può, inoltre, essere necessario richiedere che il dato catturato sia clonabile, se l'esecuzione della chiusura ritornata tendesse a consumare il dato e non si volesse ridurla ad **`FnOnce<Args>`**.

Se si vogliono restituire tipi diversi di closure in base ad una condizione, si usa un oggetto tratto:

```
fn crea_operazione(tipo: &str) -> Box i32 {  
    match tipo {  
        "raddoppia" => Box::new(|x| x * 2),  
        "quadrato"  => Box::new(|x| x * x),  
        _           => Box::new(|x| x),  
    }  
}  
  
fn main() {  
    let op = crea_operazione("quadrato");  
    println!("{}", op(4));  
}
```

// 16

## Gestione degli errori

Tutte le computazioni possono fallire prematuramente, impedendo che possa essere restituito il valore atteso o che vengano eseguiti tutti gli effetti collaterali richiesti. Questi fallimenti possono essere dovuti a cause differenti:

- Errori di programmazione
- Errori causati dall'input fornito dall'utente finale
- Errori causati dal sistema per mancanza di risorse

Alcuni sono facilmente prevedibili, altri sono dovuti a limiti del sistema di elaborazione che raramente sono presi in considerazione perchè il programmatore tende a semplificare la realtà. Indipendentemente dalle cause che li hanno prodotti, i fallimenti possono essere catalogati in due gruppi principali:

- Malfunzionamenti **recuperabili**: quelli che non hanno compromesso lo stato del programma, per cui è possibile attivare una strategia di ripristino
- Malfunzionamenti **non recuperabili**: quelli che causano un'alterazione imprevedibile dello stato o che indicano l'impossibilità di procedere ulteriormente nella computazione

Nei casi irrecuperabili, occorre terminare il processo, eventualmente eseguendo qualche operazione di pulizia sull'ambiente esterno. Negli altri, occorre mettere in atto una strategia di ripristino dello stato. Non è detto che il punto in cui si verifica il fallimento possieda abbastanza contesto per discriminare come comportarsi. Occorre fare in modo che, in caso di fallimento della computazione all'interno di una funzione, il controllo torni al suo chiamante, corredato di una opportuna descrizione di quanto successo. Questo requisito tende ad aumentare la complessità del codice, introducendo rami diversi lungo i quali la computazione può procedere, favorendo l'introduzione di errori logici alla quantità di dettagli cui occorre badare. Per questo motivo, i linguaggi **moderni** includono particolari costrutti a supporto della gestione delle eccezioni.

Rust **non utilizza il concetto di eccezione**, ma offre i tipi algebrici **Result<T, E>** e **Option<T>** per esprimere gli esiti delle computazioni. Offre inoltre la macro **panic! (...)** per forzare l'interruzione del thread corrente producendo una descrizione testuale di quanto successo, se corrisponde al thread principale, l'intero processo viene chiuso, stampando una stringa su **stderr**.

Rust offre una risposta funzionale al problema della modellazione degli errori basata sul tipo algebrico generico **Result<T, E>**. Modella l'unione di tutti i possibili risultati con successo **T** e tutti gli errori **E** che possono verificarsi nell'esecuzione di una funzione:

```
enum Result {
    Ok(T),
    Err(E),
}
```

**Result<T, E>** mette a disposizione svariati metodi che consentono di accedere ai dati contenuti al suo interno:

- **is\_ok(&self) -> bool** e **is\_err(&self) -> bool**: permettono, rispettivamente, di determinare se l'esito di un'operazione ha avuto successo o meno
- **ok(self) -> Option<T>** e **err(self) -> Option<E>**: consumano il risultato trasformandolo in un oggetto di tipo **Option<T>** piuttosto che **Option<E>**. **ok(self)** restituisce **Option<T>**, valorizzato con il dato in caso di successo. **err(self)** restituisce **Option<E>**, valorizzato con l'errore in caso di fallimento
- **map(self, op: F) -> Result<U, E>**: applica la funzione al valore contenuto nel risultato, se questo è ok, altrimenti lascia l'errore invariato
- **unwrap(self) -> T**: restituisce il valore contenuto, se è valido, ma invoca la macro **panic!(...)** se il risultato contiene un errore; in modo duale, il metodo **unwrap\_err() -> E** viene utilizzato per estrarre il valore di errore da un **Result** quando ci si aspetta che l'operazione sia fallita. Se l'operazione invece è riuscita, il programma andrà in **panic**

In alcune situazioni, può capitare che l'errore che potenzialmente viene restituito da una funzione non possa di fatto succedere, a conseguenza di quanto si è appena verificato nel corso dell'esecuzione. Oppure che il programmatore assuma che non sia necessario mettere in atto una strategia di contenimento e gestione dell'errore, lasciando semplicemente terminare il processo. Il tipo **Result<T, E>** mette a disposizione i metodi **unwrap()** e **expect(...)** che restituiscono il valore di tipo **T**, se presente, ed invocano la macro **panic!(...)** in caso di errore. **expect(...)** permette di indicare una stringa che sarà usata al posto del messaggio standard generato da **unwrap()**, nel caso si sia verificato un errore.

Per l'enum **Result<T, E>** il metodo **unwrap()** si comporta nel seguente modo:

- Se il **Result** è **Ok(T)**, estrae e restituisce il valore **T** contenuto all'interno della variante **Ok**
- Se il **Result** è **Err(E)**, causa un panic con un messaggio che indica che è stato chiamato **unwrap()** su un valore **Err**



Il metodo **unwrap\_err()** si comporta nel seguente modo:

- Se il **Result** è **Err(e)**, il metodo restituisce il valore **e** contenuto all'interno della variante **Err**
- Se il **Result** è **Ok(T)** il metodo causa un panic con un messaggio che indica che è stato chiamato **unwrap\_err()** su un valore **Ok**

Nella funzione **fn ok(self) -> Option<T>** il metodo **ok()** converte un **Result<T, E>** in un **Option<T>**:

- Se il **Result** è **Ok(val)**, il metodo **ok()** restituisce **Some(val)**. Il valore contenuto nella variante **Ok** viene avvolto in un **Some** nell'**Option** risultante
- Se il **Result** è **Err(err)**, il metodo **ok()** restituisce **None**. Il valore di errore contenuto nella variante **Err** viene semplicemente scartato, e l'**Option** risultante indica l'assenza di un valore di successo

Nella funzione **fn err(self) -> Option<E>** il metodo **err()** converte un **Result<T, E>** in un **Option<E>**:

- Se il **Result** è **Err(err)**, il metodo **err()** restituisce **Some(err)**. Il valore contenuto nella variante **Err** viene avvolto in un **Some** nell'**Option** risultante
- Se il **Result** è **Ok(val)**, il metodo **err()** restituisce **None**, scartando il valore di successo

Il metodo **map()** si comporta nel modo seguente:

- Se il **Result** è **Ok**, prende una closure che viene chiamata con il valore contenuto in **Ok**. Il risultato della closure viene avvolto in un nuovo **Ok**.
- Se il **Result** è **Err(e)**, la closure non viene chiamata e **map()** restituisce lo stesso **Err(e)**

Il metodo **map\_err()** si comporta nel modo seguente:

- Se il **Result** è **Ok**, la closure non viene chiamata e **map\_err()** restituisce lo stesso **Ok**
- Se il **Result** è **Err(e)**, la closure viene chiamata con il valore contenuto in **Err(e)**. Il risultato della closure viene avvolto in un nuovo **Err** che contiene un nuovo valore di errore: **Err(op(e))**. Si noti che **O** è il tipo della closure, **F** è il tipo del nuovo valore di errore che la closure restituirà

Se l'invocazione di una funzione restituisce un valore di tipo **Result** contenente un errore, occorre mettere in atto una strategia di gestione: terminare in modo ordinato il programma. Ritornare l'esecuzione, evitando di entrare in un loop infinito. Registrare un messaggio nel log e propagare l'errore al chiamare, delegando ad esso la corrispondente responsabilità

**Panic** serve per manifestare il fatto che il programma ha raggiunto un punto di incoerenza oltre il quale non occorre proseguire per non fare danni. In alcune situazioni, lo stato del programma risulta così compromesso che non è pensabile eseguire azioni di ripristino: questo spesso è dovuto alla presenza di errori logici nel programma stesso. In queste situazioni, Rust offre la macro **panic!(...)** che accetta argomenti simili a quelli offerti dalla macro **println!(...)** per formulare un messaggio di errore. L'effetto dell'invocazione di questa macro è la contrazione dello stack, la distruzione delle variabili che implementano il tratto **Drop** fino alla terminazione del thread corrente. Se il thread in cui è stato invocato **panic!(...)** è il thread principale dell'applicazione, il processo termina con un codice di errore non nullo, altrimenti **continua**.

```
fn main() {
    let numbers = vec![1, 2, 3, 4, 5];
    let index = 5;
    if index < numbers.len() {
        let value = numbers[index];
        println!("Il valore all'indice {} è: {}", index, value);
    } else {
        panic!("Tentativo di accedere a un indice non valido.");
    }
}
```

Nella maggior parte delle funzioni in cui si verifica un errore, non si sa quale strategia mettere in atto per ripristinare lo stato del programma. Si può facilmente ovviare a questo problema restituendo un oggetto **Result<T, E>**. Questo porta, tuttavia, a costrutti alquanto complessi, con espressioni di controllo difficili da decodificare. Per semplificare la sintassi, Rust offre l'operatore **?** che può essere applicato a qualunque espressione che produce un valore di tipo **Result<T, E>**. Se il valore risulta essere **Ok(v)**, l'operatore restituisce il valore **v** racchiuso nell'enum. Se, invece, risulta essere **Err(e)**, la funzione corrente **termina immediatamente**, ritornando il valore che incapsula l'errore. Perchè questo comportamento possa funzionare, occorre che la funzione in cui è usato l'operatore **?** ritorni un oggetto di tipo **Result<U, E>**, e che il tipo dell'errore ritornato sia compatibile con l'errore **E** che proviene dalla funzione chiamata. Se ci sono più **?** i tipi degli errori devono essere congruenti.

L'utilizzo dell'operatore **?** permette di ottenere una sintassi molto più compatta (**syntax sugar**), evidenziando il comportamento della funzione lungo il cammino principale e demandando al compilatore la scrittura delle clausole **if/match** necessarie a valutare il comportamento da adottare.

In alcune situazioni, può essere sufficiente distinguere se la computazione ha prodotto il proprio risultato, oppure indicare che non è stato possibile completarla. per questi casi esiste il tipo **Option<T>** che racchiude due alternative:

- **Some<T>** usata per descrivere il risultato atteso
- **None** che indica l'assenza di risultato, senza descriverne le ragioni

L'operatore `?` può essere applicato anche al tipo **Option<T>**. La funzione in cui viene adottato deve avere tipo di ritorno **Option<U>**, con **U** qualsiasi. Per l'enum **Option<T>** il metodo **unwrap()** si comporta nel seguente modo:

- Se l'**Option** è **Some(T)**, il metodo restituisce il valore **T** contenuto all'interno della variante **Some**
- Se l'**Option** è **None** il metodo causa un panic con un messaggio che indica che è stato chiamato **unwrap\_err()** su un valore **None**

Il metodo **unwrap\_err()** non può essere applicato ad un **Option<T>**.

L'operatore `?` può essere usato solo in funzioni che ritornano un tipo di **Result<T, E>** oppure **Option<T>**. Può essere usato nel main solo se dichiarato che restituisce un valore di ritorno di tipo **Result<T, E>**.

Quando una funzione produce diverse tipologie di errore, è necessario propagare i diversi errori così da potersi specializzare le contromisure. Rust offre diversi modi per propagare errori eterogenei, la scelta spetta al programmatore sulla base delle sue esigenze. La libreria standard mette a disposizione l'implementazione generica del tratto **From**, che converte qualsiasi valore che implementi il tratto **Error** in **Box<dyn error::Error>**. Durante la conversione vengono perse le informazioni sul tipo dell'errore. Si può risalire allo specifico errore tramite l'utilizzo del **downcast\_ref()**. La conversione può avvenire in maniera implicita attraverso l'utilizzo dell'operatore `?`

Per propagare errori eterogenei senza forzare il sistema dei tipi è possibile implementare un tipo di errore custom. Tutti gli errori custom devono implementare il tratto **Error** e conseguentemente anche i tratti **Debug** e **Display**. L'utilizzo di un **enum** permette di racchiudere i diversi tipi di errore da gestire successivamente tramite l'utilizzo del costrutto **match**. È necessario implementare il tratto **From** per convertire i diversi errori nel tipo custom da propagare:

```
#[derive(Debug)]
enum SumFileError {
    Io(io::Error),
    Parse(ParseIntError),
}
```

Un grosso aiuto all'implementazione di tipi che implementano il tratto `Error` viene dal crate **thiserror**. Esso offre una implementazione della macro **derive** specializzata per il tratto **Error**. Definendo un tipo (`enum`, `struct`) preceduto dall'attributo `#[derive(Error, Debug)]` si ottiene l'implementazione automatica di questi due tratti:

- L'attributo `#[error("Messaggio con formato")]` posta di fronte alle singole voci enumerative o all'intera struct genera l'implementazione del tratto **Display**
- L'attributo `#[from]` posto di fronte ad un campo il cui tipo implementa il tratto **Error** genera l'implementazione del tratto **From** a partire dal tipo del campo

Questo approccio è particolarmente adatto nella creazione di librerie, dove si vogliono rendere i possibili errori parte della definizione del contratto (API).

Il crate **anyhow** definisce l'oggetto-tratto **anyhow::Error** che semplifica la gestione idiomática degli errori. Si può usare il tipo **anyhow::Result<T>** per incapsulare il valore di ritorno di una funzione che può fallire. Questo tipo offre un'implementazione automatica del tratto **From<T: Error>**, il che permette di utilizzare la notazione basata sull'operatore `?` per propagare l'errore ritornato. Quando viene generato un errore è possibile aggiungere una descrizione che contestualizza ciò che è successo tramite i metodi `context(...)` e `with_context(...)`. Il messaggio di errore associato verrà sostituito dalla stringa indicata seguita dalla causa originale. Questo crate interopera correttamente con **thiserror** e risulta adatto nella scrittura di codice applicativo, piuttosto che di librerie. In questo caso, la semplicità di scrittura del codice domina rispetto al controllo dell'informazione contenuta nell'errore generato, tenuto conto che sarà interpretato da altre persone.

## Iteratori

Un iteratore è una **struttura dati** dotata di **stato**, in grado di generare una **sequenza** di valori. L'esempio più semplice di un iteratore è:

```
for i in 0..10 {}           // i è un iteratore che si ricorda dove è arrivato
```

I valori possono essere estratti da un **contenitore**, di cui l'iteratore detiene un riferimento, o generato programmaticamente, come nel caso di un intervallo di valori. Un iteratore offre tipicamente un modo per verificare se sono presenti ulteriori valori da generare (`hasNext() -> bool`) ed un altro per accedere al valore successivo (`next() -> E`).

Un iteratore può offrire ulteriori metodi che consentono di derivare un nuovo iteratore, che trasforma la sequenza di valori in un'altra sequenza. Pressoché tutti i linguaggi moderni offrono il concetto di iteratore come parte della propria libreria standard. Essi permettono di accedere ai valori contenuti all'interno di collezioni o leggere il contenuto di un file di testo estraendo una riga alla volta.

L'uso degli iteratori si sovrappone concettualmente a quello dei cicli `for/while`, ma offre svariati vantaggi in termini di compattezza del codice, leggibilità, manutenibilità ed efficienza, nascondendo i dettagli necessari a generare/accedere ai singoli elementi. I vantaggi sono maggiormente evidenti se l'operazione svolta all'interno del ciclo è complessa e richiede, ad esempio, di scartare alcuni valori e di trasformare i restanti:

```
let v1 = vec![1,2,3,4,5,6];
let v2: Vec;

v2 = v1.iter()
    .filter(|val| { *val %2 == 0 })
    .map(|val| format!("{}",val))
    .collect();

println!("{:?}", v2); // ["a2","a4","a6"]
```

Un iteratore offre un modo uniforme di accedere agli elementi, indipendentemente da come essi siano generati o da dove siano prelevati, permettendo al codice che utilizza tali valori di ignorare la fonte ed essere più generico e nascondendo i dettagli dell'operazione. Gli iteratori operano in modalità **lazy**, solo a fronte della richiesta di un valore successivo, si occupano di generarlo/prelavarlo. Le operazioni fatte dagli iteratori non sono fatte in anticipo, ma solo 1 alla volta e su richiesta, risultando efficienti per lavorare con collezioni molto grandi. Gli iteratori possono abilitare l'elaborazione parallela dei dati ospitati in una collezione, permettendo di sfruttare la presenza di più core nell'elaboratore infine, la possibilità di derivare un iteratore da un altro iteratore aumenta la flessibilità del codice

Gli iteratori operano su una fonte di dati che può essere:

- **Un contenitore di dati:** un vettore, un array, una lista, una mappa, un file, qualsiasi cosa che contiene già i dati e che chiediamo di esplorare
- **Un generatore di numeri casuali o un range di valori** che genera dati ogni volta che lo chiediamo

In Rust un **iteratore** è una qualsiasi struttura dati che implementa il tratto **std::iter::Iterator**:

```
trait Iterator{
    type Item;
    fn next(&mut self) -> Option;
}
```

**Item** è il tipo del valore che l'iteratore produce, **next()** ritorna un **Option**, **&mut self** è dato dal momento che lavora su sé stesso modificando lo stato.

I **contenitori** presenti nella libreria standard mettono normalmente a disposizione tre metodi per ricavare un iteratore ai dati contenuti al loro interno:

- **iter():** crea un iteratore che restituisce oggetti di tipo **&Item** e non consuma il contenuto del contenitore
- **iter\_mut():** crea un iteratore che restituisce oggetti di tipo **&mut Item** e permette di modificare gli elementi all'interno del contenitore
- **into\_iter():** crea un iteratore che prende possesso del contenitore (e lo consuma) e restituisce oggetti di tipo **Item** estraendoli dal contenitore

Un tipo **Iterable** può segnalare la capacità di essere esplorato tramite un iteratore, implementando il tratto **std::iter::IntoIterator**. Il tratto **IntoIterator** è un tratto di conversione che consente ad un tipo di essere convertito in un iteratore. Per implementare **IntoIterator** per un tipo personalizzato, è necessario definire un metodo **into\_iter** che restituisca un iteratore per la collezione. Questo iteratore deve implementare il tratto **Iterator** e fornire un metodo **next** che restituisca gli elementi della collezione uno alla volta:

```
trait IntoIterator where Self::IntoIter: Iterator {
    type Item;
    type IntoIter: Iterator;
    fn into_iter(self) -> Self::IntoIter;
}
```

Ricapitolando:

- **Iterator:** qualunque tipo che implementa il tratto `Iterator`. I tipi `Iterator` devono definire il metodo `next` che restituisce i valori successivi nella sequenza. Un `iterator` è ricavato da un tipo `iterable` chiamando il metodo `into_iter()`. Un `iterator` produce **values**
- **Iterable:** qualunque tipo che implementa il tratto `IntoIterator`

Il singolo valore che l'iteratore produce si chiama **item**. Il codice che riceve gli items che un iteratore produce e genera un nuovo iteratore è detto **adapter**. Il codice che consuma un iteratore e produce un risultato è detto **consumer**.

Il tratto `iterator` definisce un nutrito gruppo di metodi che consumano un iteratore e ne derivano uno differente, in grado di offrire funzionalità ulteriori. Possono essere combinati in catene lunghe al termine delle quali occorre porre un consumatore finale. Tutti gli **adattatori** sono **lazy** di natura e non invocano il metodo `next()` dell'oggetto a monte se non a seguito di una richiesta proveniente da un loro consumatore.

Ci sono innumerevoli **adattatori** e sono i seguenti:

- **map:** esegue la chiusura ricevuta come argomento su ogni elemento dell'iteratore ritornato
- **filter:** ritorna un iteratore che restituisce solo gli elementi per i quali l'esecuzione della chiusura ricevuta come argomento ritorna `true`
- **filter\_map:** concatena in maniera concisa `filter` e `map`, l'iteratore risultante conterrà solo elementi per i quali la chiusura ritorna `Some(B)`
- **flatten:** ritorna un iteratore dal quale sono state rimosse le strutture annidate
- **flat\_map:** concatena in maniera concisa `map` e `flatten`, esegue la chiusura ricevuta e rimuove le strutture annidate
- **take:** ritorna un iteratore che contiene al più i primi `n` elementi dell'iteratore su cui viene eseguito
- **take\_while:** esegue la funzione ricevuta su tutti gli elementi dell'iteratore originale, conserva tutti gli elementi fino a quando la funzione ritorna `true`; dal momento in cui diventa `false`, scarta tutti i valori rimanenti
- **skip:** ritorna un iteratore che esclude i primi `n` elementi dell'iteratore su cui viene eseguito, se si raggiunge la fine ritorna un iteratore vuoto
- **skip\_while:** esegue la funzione ricevuta su tutti gli elementi dell'iteratore originale, esclude tutti gli elementi fino a quando la funzione ritorna `false`, dal momento in cui diventa `true` conserva tutti i valori rimanenti
- **peekable:** ritorna un iteratore sul quale è possibile chiamare i metodi `peek()` e `peek_mut()` per accedere al valore successivo senza consumarlo

- **fuse:** ritorna un iteratore che termina dopo il primo None
- **rev:** ritorna un iteratore con la direzione invertita
- **inspect:** ogni volta che riceve una richiesta, preleva un elemento dall'iteratore a monte e la passa sia alla funziona, che ha possibilità di ispezionarlo, che al consumatore a valle
- **chain:** prende come argomento un iteratore e lo concatena all'originale, ritorna un nuovo iteratore
- **enumerate:** ritorna un iteratore che restituisce una tupla formata dall'indice dell'iterazione e dal valore (i, val)
- **zip:** combina due iteratori per ritornare un nuovo iteratore che ha come elementi le coppie composte dai valori dei primi due iteratori
- **by\_ref:** prende in prestito un iteratore senza consumarlo, lasciando intatto il possesso dell'originale
- **copied:** ritorna un nuovo iteratore, tutti gli elementi dell'iteratore originale vengono copiati
- **cloned:** ritorna un nuovo iteratore, tutti gli elementi dell'iteratore originale vengono clonati
- **cycle:** raggiunta la fine di un iteratore riparte dall'inizio, ciclando all'infinito

Alcuni **consumatori** essenziali invece sono:

- **product:** moltiplica tutti gli elementi di un iteratore e ritorna il valore ottenuto
- **max:** ritorna il massimo tra gli elementi dell'iteratore, se trova due massimi equivalenti torna l'ultimo, se l'iteratore è vuoto viene ritornato None
- **max\_by\_key:** esegue la chiusura ricevuta come argomento su tutti gli elementi e ritorna quello che produce il risultato massimo
- **min:** ritorna il minimo tra gli elementi dell'iteratore, se trova due minimi equivalenti torna l'ultimo, se l'iteratore è vuoto viene ritornato None
- **min\_by\_key:** esegue la chiusura ricevuta come argomento su tutti gli elementi e ritorna quello che produce il risultato minimo
- **position:** cerca un elemento sulla base della chiusura ricevuta come argomento e ritorna la posizione
- **rposition:** cerca un elemento sulla base della chiusura ricevuta come argomento, partendo da destra e ritornando la posizione
- **last:** ritorna l'ultimo elemento dell'iteratore
- **fold:** esegue la chiusura ricevuta accumulando i risultati sul primo argomento ricevuto
- **try\_fold:** esegue la chiusura ricevuta fino a quando ritorna con successo, accumulando i risultati sul primo argomento ricevuto
- **find\_map:** esegue la chiusura ricevuta su tutti gli elementi e ritorna il primo risultato valido



- **partition:** consuma un iteratore e ritorna due collezioni sulla base del predicato ricevuto
- **reduce:** riduce l'iteratore ad un singolo elemento eseguendo la funzione ricevuta

I metodi sui **confronti tra iteratori** sono i seguenti:

- **cmp:** confronta gli elementi di due iteratori
- **eq:** verifica se gli elementi di due iteratori sono uguali
- **ne:** verifica se gli elementi di due iteratori sono diversi
- **lt:** verifica se gli elementi di un iteratore sono minori rispetto a quelli di un secondo iteratore
- **le:** verifica se gli elementi di un iteratore sono minori o uguali rispetto a quelli di un second iteratore
- **gt:** verifica se gli elementi di un iteratore sono maggiori rispetto a quelli di un secondo iteratore
- **ge:** verifica se gli elementi di un iteratore sono maggiori o uguali rispetto a quelli di un secondo iteratore

## Collezioni di dati

Tutti i linguaggi offrono, nella propria libreria standard, un insieme di strutture dati volte a semplificare la vita ai programmatori implementando quelli che sono i migliori algoritmi noti per gestire problemi comuni. Se esistono strategie di implementazione, spesso sono presenti versioni alternative con diverse caratteristiche in termini di prestazioni

| Descrizione                 | Rust                                  | C++                                                                                                      | Java                                          | Python                         |
|-----------------------------|---------------------------------------|----------------------------------------------------------------------------------------------------------|-----------------------------------------------|--------------------------------|
| Array dinamico              | <code>std::Vec&lt;T&gt;</code>        | <code>std::vector&lt;T&gt;</code>                                                                        | <code>java.util.ArrayList&lt;T&gt;</code>     | <code>list</code>              |
| Coda a doppia entrata       | <code>std::VecDeque&lt;T&gt;</code>   | <code>std::deque&lt;T&gt;</code>                                                                         | <code>java.util.ArrayDeque&lt;T&gt;</code>    | <code>collections.deque</code> |
| Lista doppiamente collegata | <code>std::LinkedList&lt;T&gt;</code> | <code>std::list&lt;T&gt;*</code><br><small>*esiste anche collegata solo in avanti (forward_list)</small> | <code>java.util.LinkedList&lt;T&gt;</code>    | —                              |
| Coda a priorità             | <code>std::BinaryHeap&lt;T&gt;</code> | <code>std::priority_queue&lt;T&gt;</code>                                                                | <code>java.util.PriorityQueue&lt;T&gt;</code> | <code>heapq</code>             |
| Tabella hash                | <code>std::HashMap&lt;K,V&gt;</code>  | <code>std::unordered_map&lt;K,V&gt;</code>                                                               | <code>java.util.HashMap&lt;K,V&gt;</code>     | <code>dict</code>              |
| Mappa ordinata              | <code>std::BTreeMap&lt;K,V&gt;</code> | <code>std::map&lt;K,V&gt;</code>                                                                         | <code>java.util.TreeMap&lt;K,V&gt;</code>     | —                              |
| Insieme Hash                | <code>std::HashSet&lt;T&gt;</code>    | <code>std::unordered_set&lt;T&gt;</code>                                                                 | <code>java.util.HashSet&lt;T&gt;</code>       | <code>set</code>               |
| Insieme ordinato            | <code>std::BTreeSet&lt;T&gt;</code>   | <code>std::set&lt;T&gt;</code>                                                                           | <code>java.util.TreeSet&lt;T&gt;</code>       | —                              |

| Descrizione                 | Accesso      | Ricerca      | Inserimento  | Cancellazione |
|-----------------------------|--------------|--------------|--------------|---------------|
| Array dinamico              | $O(1)$       | $O(n)$       | $O(n)$       | $O(n)$        |
| Coda a doppia entrata       | $O(n)$       | $O(n)$       | $O(1)$       | $O(1)$        |
| Lista doppiamente collegata | $O(n)$       | $O(n)$       | $O(1)$       | $O(1)$        |
| Coda a priorità             | $O(1)$       | -            | $O(\log(n))$ | $O(\log(n))$  |
| Tabella hash                | $O(1)$       | $O(1)$       | $O(1)$       | $O(1)$        |
| Mappa ordinata              | $O(\log(n))$ | $O(\log(n))$ | $O(\log(n))$ | $O(\log(n))$  |
| Insieme Hash                | -            | $O(1)$       | $O(1)$       | $O(1)$        |
| Insieme ordinato            | -            | $O(\log(n))$ | $O(\log(n))$ | $O(\log(n))$  |

Tutte le collezioni, messe a disposizione dalla libreria standard di Rust, offrono una serie di metodi comuni:

- **`new()`**: alloca una nuova collezione
- **`len()`**: permette di conoscere l'attuale dimensione della collezione
- **`clear()`**: rimuove tutti gli elementi della collezione
- **`is_empty()`**: ritorna true se la collezione è vuota
- **`iter()`**: per iterare sui valori della collezione
- **`extend()`**: per estendere i valori di una collezione con una seconda

Oltre a questi metodi di base, tutte le collezioni implementano i tratti **IntoIterator** e **FromIterator**:

- **into\_iter()**: permette di convertire qualsiasi collezione in un iteratore
- **collect()**: permette di ottenere una collezione partendo da un iteratore

Il tipo **Vec<T>** rappresenta una sequenza ridimensionabile di elementi di tipo **T**, allocati sullo heap. Si può creare un nuovo **Vec<T>** utilizzando il costruttore **Vec::new()** o la macro **vec![val1, val2, ...]**. Una variabile di tipo **Vec<T>** è una tupla formata da tre valori privati:

- **Un puntatore ad un buffer** allocato sullo heap nel quale sono memorizzati gli elementi
- **Un intero** privo di segno che indica la **dimensione** complessiva del buffer
- **Un intero** privo di segno che indica **quanti elementi sono valorizzati** nel buffer

Questo contenitore rappresenta il principale strumento per la gestione di collezioni di dati. E' stato progettato per garantire il minor overhead possibile e una forte interoperabilità con il codice unsafe.

Si può inserire un nuovo elemento al fondo del buffer con il metodo **push(...)**. Se è presente spazio non ancora usato, il valore verrà collocato nella prima posizione libera e verrà incrementato l'intero che indica il numero di elementi effettivamente presenti. Nel caso in cui il buffer fosse già completo, verrà allocato un nuovo buffer di dimensioni maggiori, e il contenuto del buffer precedente sarà riversato in quello nuovo, dove verrà poi anche inserito il nuovo elemento, dopodiché il buffer precedente sarà deallocato. Si ottiene un riferimento al contenuto del vettore usando la notazione **&v[indice]** oppure tramite i metodi **get(...)** e **get\_mut(...)**. Nel primo caso, verrà generato un panic se l'indice non ricade nell'intervallo lecito. Nel secondo caso, verrà restituito **Option::None** piuttosto che **Option::Some(ref)**.

**Vec<T>** offre una vasta serie di metodi per accedere al suo contenuto e per inserire/togliere valori al suo interno:

- **Vec::with\_capacity(n)**: alloca un vettore con capacità n
- **capacity()**: ritorna la lunghezza del vettore
- **push(value)**: aggiunge un elemento alla fine del vettore
- **pop()**: rimuove e ritorna un `std::Option` contenente l'ultimo elemento del vettore, se esistente
- **insert(index, value)**: aggiunge un elemento alla posizione ricevuta in argomento
- **remove(index)**: rimuove e ritorna l'elemento alla posizione ricevuta in argomento
- **first()** e **first\_mut()**: ritornano un riferimento (mutabile) al primo elemento dell'array

- **last()** e **last\_mut()**: ritornano un riferimento (mutabile) all'ultimo elemento dell'array
- **get(index)** e **get\_mut(index)**: ritornano un `std::Option` che contiene il riferimento (mutabile) all'elemento nella posizione ricevuta come argomento, se esistente
- **get(range)** e **get\_mut(range)**: ritornano un `std::Option` che contiene lo slice indicato dall'intervallo di indici, se esistente
- **retain(predicate)**: mantiene solo gli elementi che soddisfano il predicato specificato
- **extend(vec)**: permette di appendere un vettore (`vec`) ad un altro

Il tipo **VecDeque<T>** modella una coda a doppia entrata: esso alloca sullo heap una serie di elementi di tipo `T`. A differenza di **Vec<T>** permette l'inserimento e la rimozione, con costo unitario, sia all'inizio che alla fine del vettore, tramite i metodi **push\_back()**, **push\_front()**, **pop\_back()**, **pop\_front()**. **VecDeque<T>** risulta più veloce di **Vec<T>** se si eseguono molte **pop\_front()**; in tutti gli altri casi è preferibile utilizzare **Vect<T>**. Si può accedere con l'indicizzazione **deque[index]**. Viene implementato come un buffer circolare e non garantisce che gli elementi siano contigui in memoria. E' possibile rendere gli elementi contigui in memoria utilizzando il metodo **make\_contiguous()**.

I metodi che offre **VecDeque<T>**, oltre quelli precedentemente visti, sono i seguenti:

- **push\_front(value)**: aggiunge un elemento all'inizio della coda doppia
- **push\_back(value)**: aggiunge un elemento alla fine della coda doppia
- **pop\_front()**: rimuove e restituisce l'elemento in testa alla coda doppia
- **pop\_back()**: rimuove e restituisce l'elemento in coda alla coda doppia
- **front()**: restituisce un riferimento all'elemento in testa alla coda doppia senza rimuoverlo
- **back()**: restituisce un riferimento all'elemento in coda alla coda doppia senza rimuoverlo

Una **BinaryHeap<T>** è una collezione di elementi di tipo `T`, i valori sono salvati nello heap e l'elemento più grande si trova sempre nel fronte della struttura dati (**Max-Heap**). Il tipo `T` deve implementare il tratto **Ord**. Il metodo **peek()** permette di ritornare l'elemento più grande con complessità  $O(1)$ . Nel caso peggiore, se si modifica l'elemento attraverso il metodo **peek\_mut()**, la complessità diventa  $O(\log(n))$ .

Nella **BinaryHeap** in Rust, i dati sono memorizzati all'interno di un vettore `Vect<T>`. Questo fornisce un modo efficiente per memorizzare gli elementi in modo contiguo in memoria. Il **BinaryHeap** di Rust usa una `Vec<T>` internamente per contenere gli elementi. Sebbene i dati siano memorizzati in un vettore lineare, il **BinaryHeap** mantiene la proprietà di un albero binario

completo attraverso la relazione tra gli indici degli elementi nel vettore. Per un elemento all'indice  $i$ , a partire da 0:

- Il suo figlio sinistro si trova all'indice  $2 * i + 1$
- Il suo figlio destro si trova all'indice  $2 * i + 2$
- Il suo genitore (se  $i > 0$ ), si trova all'indice  $(i - 1) / 2$

La proprietà fondamentale di una **max-heap** è che il valore di ogni nodo è maggiore o uguale al valore dei suoi figli.

Quando un **nuovo elemento** viene aggiunto al BinaryHeap, viene inizialmente **appeso alla fine** del `Vec<T>`. Successivamente, viene eseguita un'operazione di **heapify up**. L'elemento appena inserito viene confrontato con il suo genitore. Se l'elemento è maggiore del suo genitore, vengono scambiati. Questo processo continua risalendo l'albero fino a quando l'elemento non è più maggiore del suo genitore o raggiunge la radice.

Quando si estrae l'elemento massimo (la radice) dal BinaryHeap, il primo elemento del `Vec<T>` viene rimosso e restituito. Per mantenere la proprietà dell'heap, **l'ultimo** elemento del `Vec<T>` viene spostato **nella posizione della radice**. Quindi, viene eseguita un'operazione di **heapify down**. L'elemento nella radice viene confrontato con i suoi figli. Se è minore di almeno uno dei suoi figli, viene scambiato con il figlio maggiore. Questo processo continua scendendo l'albero fino a quando l'elemento non è maggiore o uguale ad entrambi i suoi figli o raggiunge una foglia.

L'uso di un `Vec<T>` sottostante garantisce un accesso efficiente agli elementi. Le operazioni di push e pop hanno una complessità temporale di  $O(\log n)$  dove  $n$  è il numero di elementi nell'heap, poichè l'operazione di heapify coinvolge la risalita o la discesa lungo un ramo dell'albero binario, la cui altezza è logaritmica rispetto al numero di nodi.

Alcuni metodi offerti dal BinaryHeap sono:

- **from(data)**: crea una heap a partire da un iterabile di dati
- **push(value)**: inserisce un elemento nella heap
- **pop()**: rimuove e restituisce l'elemento massimo della heap
- **peek()**: restituisce un riferimento all'elemento massimo senza rimuoverlo
- **peek\_mut()**: fornisce un riferimento mutabile all'elemento massimo
- **into\_sorted\_vec()**: consuma la heap e restituisce un vettore ordinato degli elementi

BinaryHeap **non fornisce un metodo** `iter_mut()`. Permettere la modifica arbitraria di un elemento interno della coda tramite un riferimento mutabile invaliderebbe la struttura ad albero dell'heap, compromettendo le successive operazioni. Le modifiche sul posto sono possibili solo sulla radice tramite `peek_mut()`, che si occupa di riorganizzare l'heap in modo sicuro. I metodi di iterazione disponibili sono quindi:

- `iter()`: restituisce un iteratore su riferimenti immutabili
- `into_iter()`: consuma la coda prioritaria e trasferisce il possesso degli elementi

BinaryHeap è un max-heap per impostazione predefinita. BinaryHeap non offre un costruttore diretto per min-heap. Per ottenere un **min-heap** si può avvolgere il tipo degli elementi in `std::cmp::Reverse` senza aggiungere alcun costo computazionale. Il wrapper inverte l'ordinamento, facendo sì che il valore più piccolo finisca alla radice dell'heap.

**LinkedList<T>** permette di rappresentare in memoria una lista doppiamente collegata, il tempo di accesso è costante. Come **VecDeque<T>** permette di inserire e rimuovere elementi dalla lista da entrambe le estremità. I metodi attualmente offerti da **LinkedList<T>** sono un ristretto sottoinsieme dei metodi di **VecDeque<T>**, tuttavia è quasi sempre preferibile utilizzare **Vec<T>** o **VecDeque<T>** poiché superiori in termini di prestazioni ed uso della memoria.

Ogni elemento (nodo) contiene:

- Un valore di tipo T
- Un puntatore all'elemento precedente
- Un puntatore all'elemento successivo

Le operazioni sono efficienti nella testa o nella coda ma non supporta gli indici, abbiamo overhead di memoria per il puntatore prev e next e uno scarso principio di località di cache. Viene usato **raramente** per questi motivi.

Alcuni metodi nuovi offerti da **LinkedList<T>** sono:

- `split_off()`: divide una lista in due parti separate in base all'indice specificato e restituisce una nuova lista che contiene gli elementi dalla posizione specificata fino alla fine della lista originale
- `append()`: unisce due liste concatenando la seconda lista alla fine della prima

Una **HashMap<K, V>** è una collezione di coppie composte da una chiave di tipo **K** ed un valore di tipo **V**, i valori sono salvati nello heap come una singola hash table. E' preferibile utilizzare una **HashMap<K, V>** quando le chiavi **non** hanno un ordine. L'inserimento di nuova entry nella **HashMap<K, V>** può causare la riallocazione ed il movimento dei dati. **La chiave deve essere univoca** ed il tipo **K** deve implementare i tratti **Eq** e **Hash**.

Una **BTreeMap<K, V>** è una collezione di coppie composte da una chiave di tipo **K** ed un valore di tipo **V**, i valori sono salvati nello heap come un albero dove ogni entry rappresenta un nodo. E' preferibile utilizzare una **BTreeMap<K, V>** quando le chiavi hanno un ordine, per migliorare l'efficienza di accesso ai nodi. L'inserimento di una nuova entry nella **BTreeMap<K, V>** può causare la riallocazione ed il movimento dei dati. **La chiave deve essere univoca** ed il tipo **K** deve implementare il tratto **Ord**.

I nuovi metodi introdotti da **HashMap<K, V>** sono:

- **insert(key, value)**: inserisce una coppia chiave-valore nella mappa. Se la chiave non esiste, restituisce **None**, altrimenti sovrascrive il vecchio valore e restituisce **Some(vecchio\_valore)**
- **get(&key)**: restituisce una referenza all'elemento associato alla chiave specificata, se presente
- **get\_mut(&key)**: restituisce una referenza mutabile all'elemento associato alla chiave specificata, se presente
- **contains\_key(&key)**: verifica se la mappa contiene la chiave specificata
- **remove(&key)**: rimuove e restituisce l'elemento associato alla chiave specificata, se presente, altrimenti **None**
- **keys()**: restituisce un iteratore sugli elementi delle chiavi della mappa
- **values()**: restituisce un iteratore sui valori della mappa
- **entry(&key)**: restituisce un entry della mappa per la chiave specificata, che permette di manipolare l'elemento associato in modo sicuro

Rust offre la possibilità di ottimizzare l'utilizzo delle mappe, in particolare attraverso il metodo **entry** che permette di cercare una chiave all'interno di una mappa e ritorna un enum in base al risultato della ricerca. L'enum **Entry<'a, K, V>** a sua volta mette a disposizione diversi metodi per la gestione del risultato permettendo di ridurre il numero di spostamenti in memoria:

- **and\_modify()**: in caso di successo permette di eseguire delle azioni aggiuntive sul risultato ottenuto
- **or\_insert()**: in caso di fallimento è possibile inserire una nuova entry senza costi aggiuntivi poiché il puntatore sarà già indirizzato verso una zona di memoria libera
- **or\_insert\_with()**: come **or\_insert**, ma con funzione **f** per calcolare il valore predefinito

Un **B-tree** è caratterizzato da un parametro **B**, detto fattore di ramificazione minimo, che determina il numero di chiavi memorizzate in modo contiguo in ogni nodo. Ogni nodo (tranne la radice) contiene almeno **B-1** e al massimo **2B-1** chiavi. Ogni nodo ha almeno **B** e al massimo **2B** figli.

Alcuni metodi introdotti da **BTreeMap<K, V>** sono:

- **insert(key, value)**: inserisce una coppia chiave-valore nella mappa, se non esisteva
- **get(&key)**: restituisce una referenza all'elemento associato alla chiave specificata, se presente
- **contains\_key(&key)**: verifica se la mappa contiene la chiave specificata
- **remove(&key)**: rimuove e restituisce l'elemento associato alla chiave specificata, se presente
- **range(range)**: restituisce un iteratore sugli elementi della mappa in un intervallo specificato di chiavi
- **range\_mut(range)**: restituisce un iteratore mutabile sugli elementi della mappa in un intervallo specificato di chiavi
- **entry(&key)**: restituisce un entry della mappa per la chiave specificata, che permette di manipolare l'elemento associato in modo sicuro

Un **HashSet<T>** è un insieme di elementi **univoci** di tipo **T**. I valori sono salvati nello heap come una singola **hash table**. L'inserimento di nuova entry nell'**HashSet<T>** può causare la riallocazione ed il movimento dei dati. Un **HashSet<T>** è implementato come un wrapper attorno al tipo **HashMap<T, ()>**.

Una **BTreeSet<T>** è un insieme di elementi **univoci** di tipo **T**. i valori sono salvati nello heap come un **albero** dove ogni entry rappresenta un nodo. L'inserimento di una nuova entry nella **BTreeSet<T>** può causare la riallocazione ed il movimento dei dati.

Un insieme è equivalente ad una mappa con solo chiavi anziché la coppia chiave-valore:

- Un **HashSet<T>** è implementato come un wrapper attorno al tipo **HashMap<T, ()>**
- Un **BTreeSet<T>** è implementato come un wrapper attorno al tipo **BTreeMap<T, ()>**

I metodi sono tanti e reperibili sulle slide



## File I/O

Il SO ci offre la capacità di eseguire processi e di **memorizzare in modo permanente delle informazioni**. Un **file** consiste in un'astrazione offerta dal sistema operativo che lega **un blocco di byte** di dimensione arbitraria ad un nome. I file vivono in un'organizzazione chiamata File System con metadati che descrivono ciascun file. I nomi sono organizzati in una struttura gerarchica fatta di cartelle o directory che permette di identificare uno specifico file attraverso una concatenazione dei nomi di cartelle e nome del file che ne esprime il cammino a partire da una radice nota.

I file possono:

- Essere scritti (creati da zero)
- E' possibile aggiungere informazioni in fondo (append)
- Possono anche essere modificati al loro interno poiché l'accesso ai file è random

Ad ogni file sono associati vincoli di sicurezza. Il Sistema Operativo garantisce che solo chi dispone delle necessarie autorizzazioni possa leggere, scrivere o eseguire il file. Le librerie dei linguaggi di programmazione offrono meccanismi indipendenti dal Sistema Operativo per accedere ai contenuti di un file. Nel caso di Rust, l'astrazione principale è offerta dalla struct `std::fs::File`, che modella un **file** aperto in lettura e/o scrittura e ci offre anche il concetto di **cartella** dandoci la possibilità di elencare quello che c'è presente.

Ogni sistema operativo ha regole proprie per la definizione di cosa sia un percorso lecito per indicare un file, quali siano le radici note dei percorsi, come combinare segmenti parziali in un percorso complessivo ecc... . I tipi `std::path::Path` e `std::path::PathBuf` nascondono tali differenze offrendo un meccanismo portabile per comporre e scomporre un cammino e ricavare indicazioni sul file eventualmente referenziato. **Path**, analogamente a **str**, è **unsized** e accessibile in sola lettura. **PathBuf**, analogamente a **String**, possiede il proprio contenuto e può essere modificato. `Path` e `PathBuf` offrono proprietà diverse rispetto ai tipi `str` e `String` perchè permettono di navigare il file system di cartella in cartella, segmentando il path. Il problema è che il separatore è differente tra Windows (\) e Unix (/). Attraverso i **metodi** offerti da questi tipi, è possibile ricavare informazioni sulla esistenza del file, sulla sua natura e sui metadati associati.

Alcune operazioni che si possono eseguire sui file sono le seguenti:

- **std::fs::read\_dir()**: restituisce, se ha successo, un iteratore al contenuto della cartella **dir**. Le singole voci ritornate sono di tipo **std::fs::DirEntry** e descrivono gli elementi contenuti nella cartella in termini di nome, tipo, metadati e cammino.
- **std::fs::create\_dir()**: crea una nuova cartella. Fallisce se non si dispone delle necessarie autorizzazioni, se la cartella esiste già o se la cartella genitrice del cammino indicato non esiste.
- **std::fs::remove\_dir()**: rimuove una cartella. A condizione che esista, si disponga dei necessari permessi e che sia vuota
- **std::fs::copy()**: copia il contenuto di un file in un secondo file. Restituisce in caso di successo il numero di byte copiati.
- **std::fs::rename()**: rinomina (o sposta) un file in un secondo file. Sostituendo il contenuto del file destinazione con quello sorgente. Il comportamento di questa funzione dipende dal sistema operativo
- **std::fs::remove\_file()**: elimina un file. Se il file è in uso, la sua eliminazione può essere rimandata dal sistema operativo

L'accesso al blocco di byte legato ad un file è totalmente mediato dal sistema operativo. Per poter leggere o scrivere tale blocco occorre **aprire** il file. Il sistema operativo offre apposite funzioni che restituiscono un riferimento opaco al file sotto forma di **handle** o **file descriptor**. La struct **File** offre due metodi di base per aprire un file:

- **open(path)**: apre il file in lettura a condizione che esista
- **create(path)**: tronca il file a 0 byte, se esiste, o lo crea, se non esiste ancora, dopodiché lo apre in scrittura

**P** deve implementare il tratto **AsRef<Path>**. Questo tratto è implementato dai tipi che possono essere convertiti in un riferimento a **Path**.

La struct **std::fs::OpenOption** viene utilizzata per configurare le opzioni di apertura di un file prima di aprirlo effettivamente tramite la funzione **open**. **OpenOptions** fornisce diversi metodi per configurare queste opzioni, tra cui:

- **.read(bool)**: imposta la modalità di apertura per la lettura del file
- **.write(bool)**: imposta la modalità di apertura per la scrittura del file
- **.create(bool)**: specifica se creare il file se non esiste
- **.truncate(bool)**: specifica se troncare il file se esiste già
- **.append(bool)**: specifica se scrivere alla fine del file invece di sovrascrivere il contenuto esistente

Le funzioni `std::fs::read_to_string()` e `std::fs::write()` offrono un meccanismo compatto per leggere e scrivere il contenuto di un file di testo di moderate dimensioni. `read_to_string()` legge l'intero contenuto di un file e lo mappa in una stringa. Poiché un file può avere dimensioni molto maggiori del massimo blocco di memoria allocabile, occorre utilizzare tale funzione quando si è certi che il contenuto può essere ospitato nella memoria del processo. `write()` prende un byte array e lo scrive all'interno di un file. Da non confondersi con le funzioni con lo stesso nome che vedremo dopo.

Rust gestisce le **operazioni di I/O** attraverso l'utilizzo di alcuni tratti che implementano i metodi di base per le attività di lettura e scrittura. L'utilizzo di tratti favorisce la scrittura di codice generico, indipendente dal tipo specifico su cui viene eseguito. I tratti principali offerti da Rust sono: **Read**, **BufRead**, **Write** e **Seek**

In caso di errore durante le operazioni di I/O viene ritornata una delle varianti disponibili nell'enum **ErrorKind** che rappresenta le diverse categorie di errori che possono verificarsi durante le operazioni di input/output. Questi errori sono suddivisi in categorie per facilitare la loro gestione e consentire una maggiore precisione nell'individuazione dei problemi. Ecco un elenco di alcune varianti di **ErrorKind**:

- **NotFound**: indica che il file o la directory specificati non sono stati trovati
- **PermissionDenied**: indica che non si dispone delle autorizzazioni necessarie per eseguire l'operazione
- **AlreadyExists**: indica che il file o la directory che si sta cercando di creare esiste già
- **InvalidInput**: indica che l'input fornito è invalido o non valido per l'operazione
- **TimedOut**: indica che l'operazione ha raggiunto il timeout specificato
- **Interrupted**: indica che l'operazione è stata interrotta, indica un errore non fatale che generalmente può essere gestito riprovando l'operazione

Il tratto `std::io::Read` è un tratto che indica la capacità di leggere un flusso di byte. **File**, **Stdin** e **TcpStream** sono alcuni dei tipi che implementano questo tratto. Qualsiasi tipo che possa fornire dati in forma di byte può implementare **Read**. Per implementare il tratto **Read** è sufficiente fornire l'implementazione del metodo `read(buf: &mut [8]) -> Result<usize>`. Rust genererà tutti gli altri metodi sulla base dell'implementazione di `read(buf: &mut [u8])`. Il metodo `read()` legge un numero noto di byte alla volta. In caso di successo deve ritornare `Ok(n)`, dove `n` rappresenta il numero di byte letti con successo nel buffer, compreso tra 0 e `buf.len()`. Il valore `Ok(0)` può indicare che il flusso è terminato, oppure che il buffer passato ha lunghezza 0. Ogni chiamata al metodo `read()` può causare l'invocazione di una chiamata di sistema, con il conseguente costo legato al cambio di contesto.

L'implementazione del tratto **Read** mette a disposizione diversi metodi per gestire le operazioni più comuni di I/O:

- **read\_to\_end()**: continua a leggere fino all'EOF. Si limita a richiamare il metodo **read()** fino a quando quest'ultimo non ritorna un **Ok(0)** o un errore fatale
- **read\_to\_string()**: continua a leggere fino all'EOF e riceve come parametro una **&mut String**
- **read\_exact()**: prova a leggere l'esatto numero di byte necessario a riempire completamente **buf**, se non riesce, perchè di dimensione inferiore, ritorna **UnexpectedEof**
- **bytes()**: ritorna un iteratore sui bytes
- **chain()**: permette di concatenare due reader
- **take()**: limita il numero massimo di byte che sarà possibile leggere

La differenza tra **read\_to\_end()** e **read\_to\_string()** sta che nella prima, la funzione legge tutti i byte disponibili dalla sorgente fino alla fine e li accoda ad un buffer di tipo **Vec<u8>**. Mantiene i dati letti come una sequenza di byte non interpretata. E' la funzione più generica per leggere dati binari o testuali. Mentre la seconda, legge tutti i byte disponibili dalla sorgente fino alla fine e tenta di interpretarli come una stringa UTF-8 valida, accodandoli ad un buffer di tipo **String**. Assume che i dati in ingresso siano codificati in UTF-8. Se incontra byte non validi, l'operazione fallisce

**std::io::BufRead** è un sottotratto del tratto **Read**. Il tratto **BufRead** offre una serie di metodi che permettono di migliorare le prestazioni dell'I/O appoggiandosi ad un buffer in memoria di grande dimensione. Ogni chiamata a **read()** può dare origine ad una system call, l'utilizzo di un buffer permette di effettuare meno chiamate. Preferibile usare il tratto **BufRead** per letture corte e ripetute. Invece di leggere piccole quantità di dati ogni volta che il codice lo richiede, **BufRead** adotta una strategia di **prefetching**: alla prima richiesta di lettura, esso preleva un blocco di dati di dimensioni significative dalla sorgente sottostante e lo memorizza nel proprio buffer interno. L'implementazione del tratto richiede i metodi **fill\_buff()** e **consume()**.

I metodi offerti sono:

- **read\_line()**: appende al buffer il contenuto letto. Non sovrascrive il buffer, ma vi concatena i nuovi dati. Pertanto, se si desidera isolare ogni riga, è necessario invocare il metodo **clear()** prima di ogni lettura
- **read\_to\_string()**: consente di leggere l'intero contenuto della sorgente di input e di riversarlo in un unico buffer testuale
- **lines()**: genera un iteratore che permette di scandire le linee

Il comportamento di `fill_buf()` segue una logica deterministica. Se il buffer interno contiene ancora byte non ancora consumati, il metodo restituisce immediatamente uno slice che punta a tali dati e non viene effettuata alcuna operazione di I/O sulla sorgente. Solo quando il buffer risulta vuoto esegue una lettura dalla sorgente sottostante per popolare nuovamente il buffer. Il metodo `consume()` è strettamente legato al metodo `fill_buf()`. Il modello utilizzato tipico è:

1. Chiamare `fill_buf()` per ottenere un riferimento ai dati nel buffer interno
2. Elaborare i dati all'interno della slice restituita
3. Chiamare `consume()` per comunicare al `BufRead` quanti byte sono stati elaborati

`consume()` è il meccanismo attraverso il quale si comunica al `BufRead` di aver utilizzato una porzione dei dati che ha fornito al suo buffer. Senza `consume()` si finirebbe in un ciclo infinito, leggendo sempre gli stessi byte

**`std::io::Write`** è il tratto che indica la capacità di scrivere un flusso di dati. Questo tratto è implementato, tra gli altri, dalle struct **`file`**, **`Stdout`**, **`StdErr`** e **`TcpStream`**. Il tratto **`Write`** richiede l'implementazione dei metodi **`write`** e **`flush`**:

- **`write()`**: prova a scrivere l'intero contenuto del buffer ricevuto come argomento e ritorna il numero di byte scritti. Per questioni di ottimizzazione, il sistema operativo potrebbe scrivere solo parzialmente il contenuto del buffer ricevuto
- **`flush()`**: finalizza l'output garantendo che tutti gli eventuali buffer transitori siano correttamente svuotati

Il metodo **`write_all()`** si limita a chiamare ricorsivamente il metodo **`write`** fino a quando i dati sono stati tutti scritti o viene restituito un errore fatale.

La macro **`write!()`** viene utilizzata per scrivere in un buffer dati **formattati**. Come per altre macro che interagiscono con l'I/O è necessario gestire eventuali errori che possono verificarsi durante l'operazione di scrittura. **`write!()`** garantisce che viene effettuato il `flush`, ossia garantisce che, prima di tornare, chiede al Sistema Operativo di salvare i dati. Il SO salva nel Kernel il buffer, con l'intenzione di scriverlo realmente sul file system successivamente.

**`std::io::Seek`** ha come obiettivo quello di trattare il file come fosse un grosso array. La libreria standard implementa il tratto `Seek` per le strutture dove lo spostamento del puntatore è un'operazione efficiente e la dimensione del flusso di dati è nota o determinabile. Tratto che permette di riposizionare il cursore di lettura/scrittura in un flusso di byte. Quando il flusso attinge ad un dato di dimensione nota, è possibile posizionare il **cursore** in modo relativo rispetto all'inizio del flusso, alla sua fine o alla posizione corrente.

Il tratto offre i seguenti metodi:

- **seek()**: posiziona il cursore alla posizione in byte indicata dal parametro pos
- **stream\_position()**: restituisce la posizione corrente del cursore rispetto all'inizio del flusso
- **rewind()**: posiziona il cursore all'inizio del flusso

L'oggetto `File` implementa il pattern `RAII` che garantisce che la distruzione dell'oggetto comporta il rilascio complessivo delle risorse che possiede, legate alla **handle** del Sistema Operativo. Il SO ha allocato nel kernel una struttura dati che è referenziata attraverso la handle del file. La chiusura di un file determina il rilascio di queste risorse allocate nel Kernel. In Rust il metodo `close()` non c'è perchè la chiusura avviene all'atto della distruzione dell'oggetto `File` alla fine dello scope, in cui viene invocato il metodo `drop()` che invoca la chiamata di `close()` del Sistema Operativo.

Sebbene sia possibile operare a livello di singoli byte/caratteri e ricostruire un dato strutturato a partire dai suoi componenti elementari, spesso è poco conveniente farlo attraverso librerie predefinite. Rust mette a disposizione il framework **Serde** il cui scopo è generare implementazioni efficienti di funzioni di serializzazione e deserializzazione di strutture dati arbitrarie verso formati standard quali **JSON, CSV, Avro, BSON, YML** e altri. Tale framework definisce i tratti **`serde::Serialize`** e **`serde::Deserialize`** che possono essere implementati dai tipi definiti all'interno di un programma. `Serde` offre un'implementazione predefinita per la serializzazione dei tipi elementari e di alcuni tipi della libreria standard. Supporta inoltre la macro **`#[derive(Serialize, Deserialize)]`** per generare, in fase di compilazione, l'implementazione dei tratti corrispondenti, a condizione che la struttura dati cui tale macro è applicata non contenga elemento **patologici**.

Si inseriscono, nel file `cargo.toml` le dipendenze:

```
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
```

La seconda indica il tipo di formato da generare/leggere. Si decora la struttura con la macro **`#[derive()]`**:

```
#[derive(Serialize, Deserialize, Debug)]
struct Data {
    name: String,
    data: Vec,
    attributes: HashMap,
}
```

## Smart Pointer

Ogni valore manipolato da un programma è memorizzato nello spazio di indirizzamento del processo. L'operatore `&` e (`&mut` in Rust) permette di ottenere l'indirizzo del primo byte in cui è memorizzato. Nel caso di Rust, tale operatore attiva il **borrow checker** che vigila sull'uso che viene fatto dell'indirizzo ottenuto, imponendo tutti i vincoli di sanità necessari a fornire le garanzie date del modello del linguaggio. l'operazione duale, detta dereferenza o risoluzione del riferimento, trasforma un indirizzo nel corrispondente valore puntato. Si esprime con l'operatore `*` (`.` in Rust) e quando viene applicato ad un puntatore nativo o ad un riferimento Rust, il compilatore dà accesso al dato puntato.

Sia Rust che C++ permettono di ridefinire il comportamento degli operatori del linguaggio per tipi arbitrari. Questi meccanismi, applicati agli operatori dereferenza abilitano la definizione di tipi che **sembrano** puntatori ma che hanno ulteriori caratteristiche. Ciò ha portato all'introduzione del concetto di **smart pointer** e alla sua diffusione nelle librerie standard Rust, che ne hanno abbracciato l'idea per rappresentare puntatori che possiedono i dati a cui puntano.

Tramite l'uso di puntatori è possibile costruire strutture dati dinamiche come grafi, alberi e liste. Se questo, da un lato, offre grandi libertà al programmatore, dall'altro lo espone ad una serie di problemi legati alla difficoltà di dedurre la correttezza del codice che lo manipola. Le regole restrittive imposte dal borrow checker di Rust impediscono, con l'uso di soli riferimenti, la creazione di strutture cicliche. Ogni valore in Rust è parte di un solo albero la cui radice è contenuta in una qualche variabile. Questa minore capacità espressiva abilita, però, analisi più approfondite ed è alla base delle garanzie di sanità offerte dal linguaggio. Attraverso l'uso di smart pointer come `Rc<T>` e `Arc<T>`, è possibile avere più possessori di uno stesso valore. Smart pointer come `std::rc::Weak` e `std::sync::Weak` offrono invece la possibilità di avere strutture cicliche, nel rispetto di alcune restrizioni.

Rust offre una varietà maggiore di smart pointer rispetto al C++, allo scopo di coprire ulteriori casi e definire ottimizzazioni possibili nel caso specifico di programmi puramente sequenziali piuttosto che di programmi concorrenti. In generale, sono realizzati mediante struct che contengono le necessarie informazioni che implementano i tratti **Deref** e **DerefMut**. Quando il compilatore incontra l'espressione `*ptr` la trasforma in `* ptr.deref()` o `* ptr.deref_mut()` a seconda dei casi

**std::Box<T>** è una struttura che incapsula un puntatore ad un blocco allocato dinamicamente sullo heap all'atto della sua costruzione. Il dato puntato è **posseduto** unicamente da **Box**: quando la struttura esce dal proprio scope, sintattico, il blocco sullo heap viene rilasciato automaticamente, grazie all'implementazione del tratto **Drop**. E' possibile anticipare il rilascio del blocco, invocando la funzione **drop(b)**. Se la struttura viene mossa in un'altra variabile (o ritornata da una funzione), il possesso del puntatore passa alla destinazione che diventa responsabile del suo rilascio. Questo rende possibile ottenere cicli di vita che si estendono oltre la durata della funzione in cui il dato è stato creato.

Il tipo **T** può avere una dimensione non nota in fase di compilazione. In questo caso, l'oggetto di tipo **Box<T>** si trasforma in un fat pointer formato da un puntatore seguito da un intero di dimensione **usize** contenente la lunghezza del dato puntato. Analogamente, se al posto del tipo concreto **T** si indica un oggetto-tratto (**dyn Trait**), si ha un fat pointer composto da due puntatori: quello al dato sullo heap e quello a vtable del tratto:

```
fn produce(odd: bool) -> Box<i32> {
    let mut b = Box::new(0);
    if odd { *b = 5; }
    return b;
}

fn main() {
    let b1 = produce(false);
    println!("b1: {}", b1);
    let b2 = produce(true);
    drop(b1);
    println!("b2: {}", b2);
}
```

I box facilitano il trasferimento di proprietà di oggetti grandi senza copiarli. A titolo di esempio consideriamo la seguente struttura dati chiamati **Cons List (construct list)**, struttura dati che deriva dal mondo **LISP**:

```
enum List {
    Cons(i32, List),
    Nil,
}
```

Ogni elemento (cella) della lista è composta da due parti: il **valore** dell'elemento corrente e un **puntatore** alla cella successiva o ad un valore terminale. utilizzando il **Box**, il compilatore riesce a definire la dimensione della struttura dati:

```
enum List {
    Cons(i32, Box<List>),
    Nil,
}
```



Nelle situazioni in cui occorre disporre di **più possessori** di uno stesso dato **immutabile**, è possibile usare lo smart pointer **`std::rc::Rc<T>`**. Internamente mantiene:

- Una copia del dato
- Due contatori: il primo indica quante copie del puntatore esistono (**strong**), il secondo quanti riferimenti deboli (**weak**) sono presenti

E' possibile avere un'ulteriore copia del contatore attraverso la chiamata del metodo **`clone()`**. Ogni volta che questo **puntatore viene clonato**, il contatore **strong** viene incrementato. Quando il puntatore esce dal proprio scope, il contatore viene decrementato: se il risultato è 0, il blocco viene **rilasciato**. **`Rc<T>`** si presta a realizzare **alberi** e **grafi aciclici**. Può essere usato solo in contesti **single-thread**.

**`Rc`** dispone di metodi che operano sullo Smart Pointer. Per evitare problemi di omonimia con i metodi contenuti nel dato incapsulato, tutti i metodi di **`Rc`** sono funzioni associate e non prevedono il parametro **`self`**. Per motivi di efficienza, l'operazione di incremento e decremento sui campi privati **`strong_count`** e **`weak_count`** non è **thread-safe**. Per questo motivo, non è possibile utilizzare questo smart pointer in un contesto concorrente. Esiste un altro smart pointer, trattato in seguito, che supera questo limite **`std::sync::Arc<T>`**.

Il dato puntato da **`Rc`** è mutabile solo se non esistono altri riferimenti al dato (per la regola del borrow checker). Il metodo da utilizzare è **`get_mut()`**. Se non esistono altri puntatori (strong o weak) allo stesso dato, **`get_mut`** restituirà **`Some`** permettendo di modificare il valore contenuto, altrimenti restituirà **`None`**, poiché **non è possibile avere più riferimenti mutabili allo stesso valore**

Se si costruisse, usando **`Rc<T>`**, una sequenza circolare di puntatori, la memoria allocata non potrebbe più essere rilasciata. La catena dei puntatori terrebbe in vita tutti i blocchi, garantendo che il conteggio dei riferimenti valga almeno 1. E' possibile creare una struttura con dipendenze circolari utilizzando il tipo **`Weak<T>`**. Esso è una versione di **`Rc`** che contiene un riferimento senza possesso al blocco allocato:

- Se ciò a cui punto è ancora vivo, ci arrivo
- Se ciò a cui punto non è più vivo, non ci arrivo

Dunque **non partecipo al processo di tenuta in vita**, ma se l'oggetto è vivo ci posso arrivare. Si crea un valore di tipo **`Weak<T>`** a partire da un valore di tipo **`Rc<T>`** con il metodo **`Rc::downgrade(&rc)`**. Se il valore originale è ancora in vita (ovvero se **`strong_counter() > 0`**), è possibile costruire un nuovo valore di tipo **`Rc<T>`** invocando il metodo **`upgrade()`**. Esso ritorna un valore di tipo **`Option<Rc<T>>`**.

Un puntatore `Weak` può essere generato solo a partire da un `Rc`, attraverso il metodo `downgrade()`. Un `Weak` non è direttamente dereferenziabile (non gode del tratto `Deref`). Se faccio `*w` di un oggetto `Weak`, non ottengo l'accesso al campo. Per poter accedere al dato a cui quel `Weak` punta devo fare un upgrade. Se il puntatore `Rc` è ancora vivo, il `Weak` si tramuta in un `Rc`, in modo da poterlo possedere, almeno temporaneamente. Al termine dell'elaborazione, scompare il riferimento strong ottenuto, ma rimane la variabile di tipo `Weak` che può eventualmente essere riutilizzata con un successivo upgrade.

Il **borrow checker** garantisce, in fase di compilazione, che dato un valore di tipo `T` in ogni momento valgano i seguenti invarianti, mutuamente esclusivi:

- Non esista alcun riferimento al valore al di là del suo possessore
- Esistano uno o più riferimenti immutabili (**&T**), **aliasing**
- Esista un solo riferimento mutabile (**&mut T**), **mutabilità**

Esistono situazioni in cui l'analisi statica eseguita in fase compilazione è troppo restrittiva. Il modulo `std::cell` offre alcuni contenitori che consentono una mutuabilità condivisa e controllata. I tipi offerti possono funzionare **solo in contesti non concorrenti** (basati su singolo thread).

La struct `std::cell::Cell<T>` implementa la mutabilità del dato contenuto al suo interno (**memorizzato nello stack**) attraverso metodi che non richiedono la mutabilità del contenitore. `Cell<T>` non introduce **nessuna meta-informazione aggiuntiva**. Si dice che `Cell` implementa un meccanismo di **interior mutability** (nell'ambito di una **exterior immutability**): un dato conosciuto con una reference può essere modificato, facendo un'operazione di swap. Le `Cell` permettono di bypassare le regole di analisi statica fatta dal borrow checker del compilatore. In un programma single thread, si può creare una variabile di tipo `Cell`, conosciuta come reference semplice. `Cell<T>` non consente di creare riferimenti al dato contenuto al suo interno, ma solo alla stessa `Cell<T>`.

Ecco alcuni metodi introdotti da `Cell<T>`:

- **set()**: crea una nuova istanza di `Cell<T>`
- **get()**: restituisce il dato contenuto all'interno di `Cell` a condizione che `T` implementi il tratto `Copy`
- **take()**: restituisce il valore contenuto all'interno di `Cell`, sostituendolo con il risultato dell'invocazione di **default()**, a condizione che `T` implementi il tratto `Default`
- **replace()**: sostituisce il valore contenuto nella cella con quello passato come parametro e lo restituisce come risultato. Questo metodo non pone restrizioni sul tipo di dato
- **into\_inner()**: consuma la cella e restituisce il valore contenuto.

Una particolare combinazione può essere creata con `Cell` ed `Rc`. Un **`Rc<T>`** che tiene un **`Cell<T>`** corrisponde ad un dato che può avere molti proprietari e che può essere internamente mutato.

**`Cell<T>`** non consente di creare riferimenti al dato contenuto al suo interno, ma solo di inserire, estrarre o sostituire il valore. La struct **`std::cell::RefCell<T>`** permette di creare un dato con un **unico possessore**, a cui è possibile accedere attraverso smart pointer che simulano il comportamento di riferimenti condivisi e mutabili, ma la cui **compatibilità con le regole del borrow checker è stabilita in fase di esecuzione e non di compilazione**. Eventuali tentativi di violazione delle regole generano una condizione di `panic`, comportando la terminazione del thread corrente. Oltre ad incapsulare il dato, ha un campo che dice se il dato è stato imprestato:

- Se non ce l'ha nessuno, posso accedere al dato
- Se ce l'ha qualcuno in lettura e viene chiesto in lettura, si segna che sono in `n+1` ad avercelo in lettura. Se viene chiesto in scrittura, da un `panic`
- Se ce l'ha qualcuno in scrittura, e viene chiesto in lettura o scrittura, da un `panic`

Di seguito alcuni metodi introdotto da **`RefCell<T>`**:

- **`borrow()`**: restituisce uno smart pointer che implementa il tratto **`Deref<T>`**, oppure provoca un `panic` se è già presente un riferimento mutabile
- **`borrow_mut()`**: restituisce uno smart pointer che implementa il tratto **`DerefMut<T>`**, oppure provoca un `panic` se è già presente un riferimento semplice
- **`try_borrow()`**: restituisce un `Result` contenente un riferimento immutabile al valore, se possibile, senza `panic` se non è possibile ottenerlo
- **`try_borrow_mut()`**: restituisce un `Result` contenente un riferimento mutabile al valore, se possibile, senza `panic` se non è possibile ottenerlo
- **`into_inner()`**: consuma la `RefCell` e restituisce il valore contenuto al suo interno. Può essere eseguito solo se non esistono riferimenti attivi
- **`get_mut()`**: restituisce un riferimento mutabile al valore contenuto nella `RefCell` se non ci sono altri riferimenti attivi. Controllato dal borrow checker a tempo di compilazione

**std::borrow::Cow<'a, B>** è uno Smart Pointer che implementa il meccanismo **clone on write** progettato per rappresentare in modo efficiente dati che possono essere sia presi in prestito (borrowed) sia posseduti (owned). L'obiettivo primario di Cow è l'ottimizzazione delle prestazioni: permette di lavorare con riferimenti finché è possibile (evitando allocazioni costose sulla memoria dello heap) e di effettuare una copia solo nel momento esatto in cui si rende necessaria una modifica ai dati. Se si cerca di modificare il dato contenuto:

- Se questo è in prestito, il dato viene clonato: si prende possesso della copia e si effettua la modifica, lasciando l'originale invariato
- Se era già posseduto, non avviene nessuna clonazione e si opera la modifica direttamente

E' implementato sotto forma di enumerazione nel seguente modo:

```
pub enum Cow<'a, B>
where B: 'a + ToOwned + ?Sized,
{
    Borrowed(&'a B),           // Questo indica che il valore è preso in
                                // prestito e non può essere modificato
    Owned(::Owned),           // Questo indica che il valore è di proprietà
                                // e può essere modificato.
}
```

I metodi introdotti sono:

- **from()**: crea un Cow da un riferimento a dati presi in prestito. Il compilatore, in base al tipo di dato fornito, colloca il valore nella variante Owned o Borrowed
- **to\_mut()**: se il Cow è Borrowed, **clona** i dati, trasforma se stesso in Owned e restituisce il riferimento mutabile. Se è già Owned, restituisce semplicemente il riferimento mutabile
- **into\_owned()**: se il Cow è Borrowed, clona i dati e viene restituita la proprietà. Se è Owned, il valore viene restituito senza copie aggiuntive

L'argomento **self** di un metodo può anche avere come tipo **Box<Self>**, **Rc<Self>**, o **Arc<Self>**. In tal caso, il metodo può essere solo invocato a partire dal corrispondente tipo di puntatore. l'invocazione del metodo passa la proprietà del puntatore al metodo stesso. A differenza di quanto accade con i riferimenti, non è disponibile una forma abbreviata per la sintassi di **self**, il cui tipo deve essere dichiarato in modo esplicito, come nel caso dei parametri ordinari

## Modularità

Il processo di compilazione di un progetto Rust è basato sul concetto di **crate**. A livello sorgente, un crate è un'unità di compilazione. La compilazione di ciascun file appartenente al crate dà origine ad un file oggetto che potrà essere collegato con altri file oggetti per diventare un **eseguibile** completo.

Un crate binario costituisce un'unità di versionamento e di caricamento in fase di esecuzione. Quando un crate fa riferimento a codice esportato da un altro crate, quest'ultimo deve essere combinato con il primo per mettere a disposizione quanto richiesto. Questa operazione è detta **collegamento** e dipende da come la libreria è stata realizzata. Rust supporta la creazione di librerie con diverse forme di collegamento, che può essere:

- **Statico:** in cui l'eseguibile finale ha una copia completa di tutto il codice utilizzato
- **Dinamico:** in cui il codice esterno viene caricato all'atto dell'avvio dell'eseguibile, a partire da un file di tipo **.dll** (Windows) o **.so** (Unix)

In una **libreria statica** il codice viene copiato direttamente all'interno del file eseguibile del programma durante la fase di compilazione e linking. Durante l'esecuzione tutto il codice necessario è già presente all'interno dell'eseguibile. In una **libreria dinamica** durante la compilazione e il linking nel file eseguibile viene inserito solo un riferimento alla libreria dinamica e il codice della libreria non viene copiato al suo interno. La libreria dinamica viene caricata in memoria solo quando il programma viene eseguito. Il SO si preoccupa di trovare e caricare la libreria. Più processi possono condividere la stessa copia della libreria in memoria, portando ad un uso più efficiente delle risorse. Le librerie dinamiche sono mappate dal SO in uno spazio degli indirizzi di memoria riservati all'SO stesso.

La sezione **[lib]** del file `Cargo.toml` che descrive il progetto contiene dettagli relativi al tipo di libreria che si intende creare. occorre specificare l'opportuno valore della chiave **crate-type** per ottenere i diversi seguenti tipi di librerie:

- **rlib:** libreria statica Rust, il file risultante può solo essere usato da altri eseguibili Rust che ne includeranno le parti importate direttamente nel proprio codice binario
- **dylint:** libreria dinamica Rust, viene trasformata in un file **.dll** in windows, **.so** in linux, **.dylib** in macOS. Può essere utilizzata solo da codice Rust, ma sarà caricata in fase di esecuzione
- **staticlib:** libreria statica conforme alle specifiche C, viene trasformata in un file **.lib** in Windows e **.a** in Linux e macOS. Può essere usata da programmi scritti in altri linguaggi

- **cdylib**: libreria dinamica conforme alle specifiche C, viene trasformata in un file `.dll` in Windows, `.so` in Linux, `.dylib` in macOS. Può essere usata da programmi scritti in altri linguaggi

Il codice contenuto in un crate è costituito da un albero di **moduli**. Un modulo è un costrutto sintattico, introdotto dalla parola chiave **mod** e dal relativo nome che racchiude funzioni, tipi definiti dall'utente, tratti, implementazioni ed altri moduli. Tutto il codice che non è racchiuso in un blocco di tipo **mod nome\_modulo { ... }**, fa parte del modulo radice.

I moduli formano una gerarchia ad albero la cui radice è l'unità di compilazione corrente, rappresentata dalla parola chiave **crate**. Di base, tutti gli elementi definiti all'interno di un modulo sono considerati privati, e sono accessibili solo al codice presente nel modulo stesso o nei suoi sotto-moduli. Se un elemento definito all'interno di un modulo `m1` è preceduto dalla parola chiave **pub**, diventa pubblico e può essere utilizzato dal codice presente in un modulo `m2` che non sia un suo discendente, a condizione che tutti gli antenati del modulo `m1` siano a loro volta accessibili al modulo `m2`.

Per poter usare un simbolo definito in un modulo diverso da quello corrente occorre indicare al compilatore dove andare a reperirlo, facendo precedere il simbolo dal relativo cammino. Questo può essere assoluto o relativo. Un cammino assoluto comincia con il nome del crate in cui si trova il simbolo seguito dalla sequenza gerarchica di moduli che occorre attraversare per giungere al simbolo:

```
let size = std::mem::size_of_val(&f);
```

Si usa il separatore `::` per connettere i nomi dei sotto-moduli. Nel caso il simbolo appartenga al crate corrente, si può indicare come elemento iniziale la parola chiave **crate**. Un cammino relativo comincia con le parole chiave **self**, **super** o direttamente con il nome del sottomodulo in cui il simbolo è definito:

```
let result = self::my_mod::public_nested_mod::api(some, arguments);
```

Per evitare di dover ripetere l'intero cammino per ogni occorrenza del simbolo è possibile utilizzare il simbolo **\***. Questo rende disponibile, nel file corrente, la definizione di tutti i simboli pubblici del modulo indicato. Per poter accedere ad un simbolo, occorre che tutti gli elementi del cammino siano accessibili a chi ne fa richiesta, ovvero essere dichiarati come pubblici o contenere il modulo corrente.

Il codice relativo ad un sotto-modulo può essere inserito:

- **Nello stesso file** sorgente **del modulo genitore**, con la notazione **mod nome\_modulo { ... }**
- **In un file** sorgente **a sé**, presente nella stessa cartella, chiamato **nome\_modulo.rs**. In questo caso occorre che nel modulo genitore sia presente la dichiarazione del modulo, nel formato **mod nome\_modulo**.
- **In una sotto-cartella** chiamata **nome\_modulo**, nel file chiamato **mod.rs**. Questo può contenere direttamente la definizione degli elementi che lo compongono o fare riferimento ad ulteriori file sorgente, posti nella stessa sotto-cartella, contenenti ciascuno un sotto-modulo

Alcuni simboli particolarmente frequenti non necessitano di essere importati in modo esplicito, pur appartenendo a crate distinti da quello corrente (sono infatti parte della libreria standard). Questi sono elencati all'interno di un modulo chiamato **std::prelude** che viene importato automaticamente all'atto della compilazione di un file sorgente. In questo modo si evita di rendere verboso il codice sorgente. Ogni versione di Rust viene con un proprio preludio. Cargo provvede ad includere quello corretto in funzione del valore dell'attributo `package/edition` nel file **Cargo.toml**

## Test

Un test è un blocco di codice creato intenzionalmente per verificare se una certa porzione di codice funziona o meno. Verificare funzionalmente i singoli componenti di un sistema è un modo efficace e pratico di creare e mantenere codice di alta qualità. Un test non può dimostrare l'assenza di errori, ma aiuta a generare confidenza nel sistema nel momento in cui questo viene messo in campo e aiuta a conservare la correttezza della base di codice, quando il progetto deve essere mantenuto nel tempo. E' molto complesso riorganizzare su larga scala del codice in assenza di test di unità. I benefici legati ad un uso intelligente e bilanciato dei test di unità nel software sono profondi. Nelle fasi di **implementazione**, test di unità ben scritti diventano una specifica informale dei componenti di un sistema. Nelle fasi di **manutenzione**, i test esistenti servono da freno alle regressioni nel codice, stimolando una correzione immediata.

Un **test di unità** valuta il comportamento di un **singolo componente** software, indipendentemente dal resto del sistema, sollecitandone sia il comportamento tipico, ovvero conforme all'uso per cui è stato progettato, sia quello agli estremi o oltre il suo previsto campo di utilizzo. Tale sollecitazione viene esercitata tramite frammenti di codice che invocano il componente e verificano che esso risponda in modo atteso. Il test costituisce un punto di ingresso che può sempre essere utilizzato per validare il componente. I test di unità sono normalmente scritti ed eseguiti dallo sviluppatore. Il costo delle modifiche che vengono evidenziate da questo tipo di test è normalmente basso.

Un **test di integrazione** valuta il comportamento all'interfaccia di due moduli software. Si focalizza sul determinare la correttezza di tale interfaccia, evidenziando le incongruenze tra le parti che stanno interagendo. Come nel caso dei test di unità, è costituito da frammenti di codice che sollecitano l'interazione tra due parti. A differenza dei test di unità, tuttavia, ignora volutamente la struttura interna dei moduli che stanno integrando, assumendo il loro corretto comportamento alle specifiche definite. Per questo motivo viene eseguito solo su oggetti che hanno correttamente superato il test di unità. I costi delle modifiche che occorre introdurre se un test di integrazione fallisce sono generalmente più elevati, in quanto richiedono il ridisegno di una o entrambe le parti che interagiscono. I test di integrazione sono co-progettati da sviluppatori e tester e vengono eseguiti dai tester.

Un **test di sistema (end-to-end)** valuta il comportamento del prodotto finito, sia nei suoi aspetti funzionali che non funzionali. Basato su modelli di comportamento tratti da casi d'uso reali, progettato ed eseguito da tester, allo scopo di non farsi influenzare dalle assunzioni fatte durante lo sviluppo. Il test di accettazione verifica che il sistema sia conforme alle aspettative del cliente ed è eseguito dagli utenti finali.



I **test di unità in Rust** possono essere scritti nel modulo da testare o, meglio, in un suo sotto-modulo chiamato convenzionalmente **tests**. Questo facilita l'ispezione del codice quando il numero di test cresce significativamente. Il sotto-modulo contenente i test è preceduto dall'annotazione **`#[cfg(test)]`**. Questo informa il compilatore che il codice contenuto al suo interno deve essere incluso solo quando la compilazione avviene con il comando **`cargo test`**:

```
// funzione da collaudare
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
    }
}
```

La struttura generale di una funzione etichettata con **`#[test]`** è la seguente:

- **Arrange:** preparazione dei valori su cui operare
- **Act:** esecuzione del codice da testare
- **Assert:** verifica, mediante asserzioni, che i risultati ottenuti siano quelli attesi

Per supportare l'ultimo passo, la libreria standard di Rust mette a disposizione varie macro procedurali:

- **`assert!()`**: impone che la condizione sia vera
- **`assert_eq!()`**: impone l'eguaglianza tra i due valori
- **`assert_ne!()`**: impone la disuguaglianza tra i due valori

per verificare che un'espressione generi la condizione di panico, è possibile annotare il test anche con le annotazioni:

- **`#[should_panic]`**: viene verificato che la funzione generi un panic!
- **`#[should_panic(expected = "msg")]`**: viene verificato che il messaggio generato da panic! contenga **`"msg"`**

Se una funzione di test restituisce un valore di tipo **`Result<T, Error>`** è possibile utilizzare la condizione di errore per indicare che il test è fallito.

I **test di integrazione in Rust** servono per testare il comportamento del crate dall'esterno come se fosse usato da qualcun altro. Vivono in un posto separato, sono contenuti in una cartella separata denominata **tests**, a lato della cartella **src** del progetto. I file sorgente all'interno della cartella

di test importano i simboli pubblici oggetto di valutazione tramite istruzioni di tipo **use**. Solo le **librerie** possono avere test di integrazione.

Il comando **cargo test** ricompila il programma con la direttiva **test** ed esegue tutti i test contenuti nel package, compresi quelli eventualmente presenti all'interno della documentazione. Viene prodotto un report dettagliato che indica ogni singolo fallimento, dove viene evidenziato il risultato atteso e quello realmente ottenuto seguito da un riassunto con il nome di tutte le funzioni di test che sono fallite. E' possibile lanciare solo alcuni test, specificando il nome della funzione come ulteriore parametro del comando **cargo test add\_**. Se uno o più test è decorato con l'attributo **#[ignore]**, viene normalmente tralasciato dall'esecuzione, ma è possibile includere tali test con il comando **cargo test --ignored**.

Normalmente, le funzioni di test sono prive di parametri, devono poter essere invocate direttamente dal motore di test. La libreria **rtest** permette di creare test parametrici, costituiti cioè da funzioni dotate di parametri in ingresso. Tali funzioni devono essere precedute dall'attributo **#[rtest]**. I parametri possono essere forniti in due modi alternativi:

- Tramite un elenco di tuple alternative, descritte tramite l'attributo **#[case(...)]**
- Tramite funzioni di set-up, annotate con l'attributo **#[fixture]**, il cui nome deve coincidere con il nome del parametro di ingresso presente nei test, il cui risultato viene usato come parametro

## Concorrenza

Un programma concorrente dispone di due o più flussi di esecuzione contemporanei, per perseguire un obiettivo comune tali flussi possono essere eseguiti in parallelo e/o alternarsi nel tempo, sotto il controllo di uno scheduler. Ogni processo è **isolato** rispetto agli altri e vive nel suo spazio di indirizzamento. L'isolamento complica la cooperazione tra processi e un isolamento imperfetto può creare possibili interferenze con il File System o con il sistema reale.

All'atto della creazione, un processo dispone di un **unico flusso di esecuzione**. Il thread principale può richiedere allo scheduler la **creazione di altri thread**. Un thread rappresenta una **computazione indipendente**, basata su un proprio stack collocato nello stesso spazio di indirizzamento in cui operano gli altri thread del processo. Tale computazione si svolge fino al proprio termine, potendo dare origine ad un risultato o ad un errore.

Il Sistema Operativo e/o le librerie di supporto allocano le risorse fisiche necessarie. Lo scheduler ripartisce, nel tempo, l'utilizzo dei core disponibili tra i diversi thread in modo non deterministico. Tutti i thread creati sono identificati in modo univoco e viene mantenuto, per quelli in uso, l'indicazione del loro stato di esecuzione. La gestione dei thread può essere demandata direttamente al S.O., in questo caso si parla di **thread nativi**, oppure gestita da librerie a livello utente, con il supporto parziale del Sistema Operativo. Questi vengono definiti **green thread** o **fiber** e richiedono una certa forma di cooperazione da parte del codice che deve, in modo esplicito, invocare lo scheduler per cedere l'uso della CPU. Rust offre nella propria libreria standard supporto per i thread nativi, e librerie di terze parti per il supporto di green thread

I diversi sistemi operativi offrono funzionalità simili per governare l'interazione di un programma con l'insieme dei thread che lo costituiscono. Le funzioni supportate sono:

- **Creazione** di un thread, indicano la funzione che rappresenta la computazione che deve essere svolta e la dimensione dello stack richiesto. Questa operazione restituisce un **handle** opaco mediante il quale fare riferimento al thread.
- **Identificazione** del thread corrente, sotto forma di valore univoco a livello di sistema
- **Attesa** della terminazione di un thread, a partire dalla sua handle, e accesso al suo stato finale. Operazione di **join**

Tra le funzioni **non supportate**, spicca la richiesta di **cancellazione** di un thread, che può essere implementata in modo cooperativo dal thread stesso.

La concorrenza implica una **riduzione del sovraccarico** dovuto alla comunicazione tra processi. Sebbene la parallelizzazione possa essere fatta anche creando processi separati, il costo di

comunicazione e sincronizzazione tra processi è più alto di quello tra thread. I thread infatti condividono lo spazio di indirizzamento ed è possibile trasferire la proprietà di strutture dati da un thread ad un altro semplicemente comunicando il **puntatore**.

La concorrenza implica anche la possibilità di **sovrapporre temporalmente** attività di computazione e operazioni di I/O. I sistemi operativi tendono ad offrire API bloccanti che arrestano, di fatto, la prosecuzione di un thread fino a che il dato richiesto non è pronto. Suddividendo l'algoritmo in più thread, si può cercare di sfruttare i tempi di attesa che un dato thread subisce a seguito delle operazioni di I/O per eseguire, in altri thread, operazioni utili al risultato. Ciò può permettere un miglioramento delle prestazioni anche in ambiente single core. Occorre che la complessità aggiunta dalla suddivisione sia compensata da un effettivo guadagno in termini di tempi di esecuzione.

La concorrenza consente di sfruttare appieno le capacità di **elaborazione** delle CPU **multicore**, garantendo vero parallelismo e più flussi di esecuzione che possono svolgersi contemporaneamente, riducendo il tempo di elaborazione. Abbiamo così un **aumento** significativo **della complessità** del programma, con nuove fonti e tipologie di errori e il **non determinismo dell'esecuzione**. La memoria **non** può più essere pensata come un **deposito statico**, i dati scritti possono cambiare in conseguenza dell'attività di altri thread. I thread devono **coordinare l'accesso** alla memoria tramite opportuni costrutti di **sincronizzazione**. La presenza di cache legate ai singoli core introduce non determinismo nell'ordinamento e nella visibilità delle azioni sulla memoria

La **concorrenza** indica più flussi di elaborazione attivi nello stesso intervallo di tempo, il cui avanzamento può avvenire in alternanza su uno stesso core sotto il controllo dello scheduler, oppure in parallelo se sono disponibili più core. Il **parallelismo** è un caso particolare in cui i flussi di elaborazione sono realmente in esecuzione nello stesso istante, perché assegnati a core distinti. **Ogni parallelismo è una forma di concorrenza, ma non ogni concorrenza implica parallelismo.**

Se all'interno di un processo sono presenti due o più thread, questi possono procedere indipendentemente nella propria computazione, sebbene sia possibile creare thread che si ignorano reciprocamente e non necessitano alcuno scambio di informazione, sul piano pratico questo avviene molto raramente. L'utilità di suddividere la computazione globale in più sotto-computazione nasce dal fatto che ciascuna di esse contribuisce in qualche modo al risultato finale. I meccanismi soggiacenti alla comunicazione/sincronizzazione interferiscono con le ottimizzazioni usate dai processori per migliorare l'esecuzione, introducendo una serie di **complessità inattese** e lontane dal pensiero comune legato al modello di esecuzione sequenziale.

In un sistema single-core il concetto di thread è puramente una astrazione offerta dal sistema operativo. Il processore si limita ad alterare il proprio ciclo di esecuzione basato sulla successione delle micro operazioni **fetch/decode/execute**, procedendo di istruzione in istruzione la logica del codice macchina. Il SO può intervenire in questa sequenza grazie ad un'interruzione che attiva lo scheduler. Questo salva lo stato dei registri in una qualche area di memoria dedicata alle meta informazioni del thread corrente e li ripristina con il contenuto relativo ad un thread differente (**task switching**). Se la CPU è dotata di due o più core, non cambia molto. Ciascun core procede indipendentemente e lo scheduler provvede a gestire le attività di tutti, allocando di volta in volta i core disponibili ad eseguire l'uno o l'altro thread secondo le necessità.

Piche l'esecuzione di ciascun thread procede indipendentemente, un'eventuale necessità di comunicazione deve essere soddisfatta passando per **l'utilizzo di un'area di memoria condivisa**, in cui il thread T1 possa depositare le informazioni che intende comunicare al thread T2 . Sebbene i due thread utilizzino lo stesso spazio di indirizzamento e possano accedere al dato memorizzato, questa operazione risulta **più complessa** di quanto si possa pensare. Per rendere la comunicazione utile sul piano pratico, può essere inoltre necessario avvalersi di pattern di interazione che definiscono con precisione i ruoli che le due o più parti coinvolte possono giocare.

Quando un thread legge il contenuto di una locazione di memoria, può trovare:

1. Il valore iniziale contenuto nel file eseguibile che è stato mappato in memoria
2. Il valore che **questo stesso thread** ha precedentemente depositato all'interno della locazione
3. Il valore che è stato depositato **da un altro thread**
4. Un valore **impredicibile** che non è stato scritto da nessuno

La presenza di cache hardware e il possibile riordinamento delle istruzioni da parte della CPU rendono il terzo caso **problematico**. In generale **non è predicibile** quale valore venga letto senza controllare letture e scritture da parte dei thread. Occorre usare un costrutto di sincronizzazione esplicito che permetta di definire l'ordine di esecuzione.

Ipotizziamo che un thread abbia modificato una variabile, ma questa variazione non è ancora giunta all'aggiornamento della memoria. Per evitare che un secondo thread legga da memoria un dato non coerente, occorre che le operazioni di scrittura siano fatte attraverso istruzioni Assembly che invalidano la cache. Istruzioni che rallentano molto l'operazione, ma garantiscono che la lettura sia sicura.

I problemi principali aperti sono:

- **Atomicità:** si vede che cosa c'era prima o si vede che cosa ci sarà dopo, ma non si vede il durante. Se due thread fanno **accesso alla stessa struttura dati**, rispettivamente in lettura e scrittura non c'è nessuna garanzia su quale delle due operazioni sia **eseguita per prima**
- **Visibilità:** se un thread legge un dato che un altro thread sta modificando il valore letto può essere **diverso** sia dal valore **iniziale** che da quello **finale**. Le scritture che un thread fa devono arrivare fino alla memoria principale per diventare visibili a tutti
- **Ordinamento:** problema legato alle ottimizzazioni che il compilatore e il processore può fare che cambiano l'ordine di esecuzione delle istruzioni. Può succedere che aggiornamenti diversi abbiano dei tempi di propagazione differenti e dunque la loro visibilità non mantenga lo stesso ordine

Ciascuna famiglia di processori offre una risposta ai problemi menzionati. La piattaforma **x86** adotta un modello di memoria forte **quasi sequenzialmente consistente** ed offre le istruzioni di tipo **fence**, per controllare l'ordine delle operazioni di memoria e forzano il completamento delle operazioni di scrittura, bloccando temporaneamente gli altri core che dovessero cercare di accedere nel frattempo allo stesso segmento di indirizzi:

- **LFENCE (load fence):** garantisce che tutte le letture dalla memoria effettuata prima della LFENCE siano completate prima di eseguire letture successive
- **SFENCE (store fence):** garantisce che tutte le scritture dalla memoria effettuate prima della SFENCE siano visibili prima di eseguire scritture successive
- **MFENCE (memory fence):** garantisce che tutti gli accessi alla memoria iniziati prima della MFENCE siano completati prima di quelle successivi

Sulla piattaforma **ARM** viene usato un modello molto più lasco, basato su liste di propagazione dei cambiamenti, ed offre le istruzioni di tipo **memory barrier** che consentono l'ordinamento casuale delle istruzioni:

- **DMB (Data Memory Barrier):** assicura l'ordine delle operazioni di memoria prima e dopo la barriera, ossia tutte le operazioni di memoria avviate prima delle DMB sono completate prima che gli accessi dopo la barriera possano iniziare
- **DSB (Data Synchronization Barrier):** blocca l'esecuzione finché tutte le operazioni di memoria prima della barriera non siano completate
- **ISB (Instruction Synchronization Barrier):** svuota la pipeline delle istruzioni garantendo che tutte le istruzioni dopo la barriera vengano ricaricate dalla memoria

Se tali istruzioni non vengono incluse all'interno del codice generato, **non è garantito un ordinamento predicibile** alle operazioni di **lettura e scrittura di dati condivisi**, in presenza di attività concorrenti. C++ e Rust condividono il modello di memoria su cui sono basati e annegano,

nelle funzioni di libreria dei tipi dedicati alla concorrenza, tali istruzioni allo scopo di garantire le necessarie proprietà di funzionamento.

**L'uso superficiale** dei costrutti di sincronizzazione porta a:

- Blocchi passivi che si aspettano reciprocamente (**deadlock**)
- Blocchi attivi dove i processi cambiano continuamente il loro stato in funzione dello stato di altri processi, senza fare avanzamenti utili nell'elaborazione, in un ciclo infinito di azioni inutili (**livelock**)
- Assenza di terminazione non riesco a smettere perchè non esiste la possibilità di informarmi che devo smettere (**loop infinito**)

**L'assenza** di costrutti di sincronizzazione, porta a risultati imprevedibili, anche molto lontani da quanto sarebbe logico aspettarsi. Si possono verificare **malfunzionamenti casuali** dovuti al comportamento non deterministico e asincrono dell'esecuzione concorrente, estremamente difficili da riprodurre e da eliminare. Gli errori possono manifestarsi cambiando la **piattaforma di esecuzione** oppure soltanto dopo numerose esecuzioni, tipicamente emergono nel momento meno adatto.

Il sistema operativo mantiene, al proprio interno, una rappresentazione dei thread presenti all'interno di un dato processo. Ad ogni thread viene associato un identificativo unico (**TID - Thread ID**), il suo stato di esecuzione e le informazioni necessarie a salvare/ripristinare lo stato dei registri interni al processore. Provvede inoltre ad allocare, nello spazio di indirizzamento del processo, un blocco di indirizzi contigui destinato ad ospitare lo stack che governerà la sua esecuzione. Esiste un numero massimo di thread allocabili creando un nuovo thread sottraendo risorse di memoria per lo stack.

Tutti i thread presenti in un processo **condividono**:

- Le variabili globali
- Le costanti
- L'area eseguibile in cui è contenuto il codice
- Lo heap

L'esecuzione di ogni singolo thread procede secondo le normali regole sequenziali, per cui è possibile prevedere cosa avvenga **prima** e cosa **dopo**. Se più thread sono in esecuzione, non è possibile fare assunzioni sulle velocità relative di avanzamento, se non ricorrendo a forme esplicite di **sincronizzazione** e **comunicazione**. La sincronizzazione può riguardare il raggiungimento di un particolare stato da parte di un thread, abilitando di conseguenza altri a **procedere** oppure l'esigenza di un thread di eseguire azioni su aree condivise, allo scopo di **impedire** ad altri di

accedere alle stesse aree. In alcuni casi, all'informazione logica che abilita/impedisce la prosecuzione di altri thread, si accompagna il trasferimento di informazioni più strutturate, che rappresentano l'esito totale o parziale di una computazione avvenuta o la richiesta di elaborazione di ulteriori dati.

L'output dei programmi precedenti è **solo uno** dei possibili risultati. Se lo stesso programma viene seguito più volte, si ottengono risultati differenti. E' assolutamente possibile che tutte le righe di uno dei thread precedano quelle dell'altro in base al numero di core disponibili e alla durata del **quanto** di schedulazione adottato dai Sistemi Operativi. L'unica certezza è che le righe che cominciano con "aaaa" sono tra loro ordinate in modo crescente, così come le righe cominciano con "bbbb".

I thread, nel programma precedente, non hanno punti di contatto, ma qualora vi siano, il **non determinismo** può dare origine a **comportamenti del tutto inattesi**. Questo può essere visto facilmente in C/C++, dove l'assenza di restrizioni da parte del borrow checker, non obbliga il programmatore ad avere cura degli aspetti di sincronizzazione. Per contro, **Rust si fa garante che un'intera gamma di possibili errori non possano verificarsi**.

**L'interferenza** si verifica quando più thread fanno accesso ad uno stesso dato, **modificandolo**. La sua presenza dà origine a **malfunzionamenti casuali**, molto difficili da identificare.

Se due thread cercano di accedere in lettura/scrittura ad una stessa variabile si verifica una **corsa critica (data race)**. In base a condizioni non controllabili dal programmatore, il dato memorizzato potrebbe essere quello scritto dal primo thread, quello scritto dal secondo oppure un terzo valore **completamente arbitrario**. L'accesso in lettura/scrittura a variabili il cui contenuto è scritto da altri thread è soggetto a diversi vincoli. Deve essere preceduto/seguito da istruzioni che proteggano da dati obsoleti presenti nella cache. Deve avvenire solo quando c'è l'evidenza che il dato non sta venendo modificato da altri. Se si sta operando una lettura in attesa di un risultato, si vuole evitare di eseguire cicli continui di polling, che consumano inutilmente cicli di CPU e batteria.

La certezza che l'operazione di accesso corrisponde alla variabile in memoria è garantita da **apposite istruzioni macchina** (fence/barrier), che dipendono dal processore responsabile dell'esecuzione del codice. La certezza che l'accesso ai dati sia effettuato solo da un thread alla volta è garantito da **invarianti a livello sistema** che può essere fornita solo da meccanismi offerti dal sistema operativo (**mutex** e **condition variabile**) che, controllando la schedulazione dei thread, può farsi carico che avvenga/non avvenga una determinata condizione. Ne consegue che i meccanismi di sincronizzazione dipendono dalla coppia processore/sistema operativo, che collettivamente definiscono l'interfaccia binaria dell'applicazione. Le librerie standard dei diversi linguaggi di



programmazione si fanno carico di standardizzare tale comportamento, offrendo API comuni a livello di codice sorgente

**Mutex** genera la barriera e garantisce che entrando nel blocco di codice venga invalidata la linea di cache corrispondente alla variabile condivisa e quando esco da quel blocco di codice venga fatta la flush della cache

**Condition Variable** fa attendere un thread fino a quando una condizione non si verifica senza consumare CPU e, appoggiandosi ad un mutex garantiscono una barriera di protezione e coerenza della memoria

Tutti gli oggetti **condivisi mutabili** devono godere di questa proprietà. Occorre fare in modo che **non capitino mai** che un **thread operi** su un dato, alterandone il contenuto mentre **un altro** sta già operando sullo stesso oggetto. Non devono essere visibili **stati transitori** dell'oggetto, dovuti al meccanismo di aggiornamento in cui solo una parte dell'informazione contenuta è cambiata.

In quasi tutti i linguaggi di programmazione, è compito del programmatore riconoscere **quando, dove e come** utilizzare la sincronizzazione. Un uso sbagliato porta a **risultati disastrosi**. Occorre dimostrare la **correttezza formale** dell'algoritmo complessivo garantendo che, qualunque sia l'ordine di esecuzione scelto dal sistema operativo o la latenza introdotta dai sottosistemi di memoria, il programma resti corretto. Questo è possibile grazie alla presenza di primitive offerte dal Sistema Operativo/Compilatore/Librerie di esecuzione che garantiscono alcune **invarianti di basso livello**. Se un programma non è corretto, è inutile pensare di testarlo o ottimizzarlo. L'esito dei test **non è deterministico**: potrebbero passare per pura combinazione fortuita delle condizioni al contorno, ma non danno alcuna garanzia. Le ottimizzazioni non farebbero che esacerbare il problema, rendendone più difficile la comprensione.

In Rust, le limitazioni imposte dal borrow checker sulla esclusività dell'accesso in scrittura unite all'utilizzo di tratti che modellano il comportamento che un tipo esibisce quando viene passato da un thread ad un altro, diventano **garanti della correttezza degli accessi alle variabili condivise** a livello di compilazione, trasformando errori in esecuzione difficili da identificare e replicare in errori di compilazione. Questo ha portato a definire questo aspetto di Rust come **fearless concurrency**. Resta comunque responsabilità del programmatore la comprensione dell'algoritmo e la garanzia di terminazione dello stesso, così come la verifica dell'assenza di blocchi attivi e passivi che possono impedire il procedere dell'algoritmo.

Le possibili soluzioni per l'accesso condiviso sono:

- **Tipi Atomic:** alla base di tutti i meccanismi di accesso condiviso ci sono istruzioni apposite a garantire operazioni di tipo Read-Modify-Write di tipo atomico, ovvero non interrompibili e non osservabili nei loro stati intermedi. Tali operazioni sono limitate a tipi semplici e sono esposte dalle librerie standard di Rust attraverso opportune astrazioni, che incapsulano l'utilizzo di barriere di memoria
- **Mutex:** per estendere le garanzie di atomicità e dipendenza casuale a strutture dati più complesse, occorre introdurre il concetto di Mutex. Essi estendono il principio della mutua esclusione a thread differenti. Un Mutex può essere libero o posseduto da un singolo thread. Se un secondo thread cerca di ottenere il possesso del Mutex mentre è in uso da parte di un altro thread, rimane in attesa fino a che esso non viene rilasciato
- **Condition variable:** in alcuni casi, occorre attendere che si verifichi una condizione più complessa del semplice rilascio di un mutex da parte di un thread. Una condition variable permette di realizzare tale attesa, a condizione che il thread che causa l'avverarsi della condizione si occupi di segnalarlo, generando una notifica tramite appositi metodi. Una condition variable può essere usata solo in coppia con un Mutex.

In Rust si crea un thread nativo attraverso la funzione `std::thread::spawn(...)`. Essa accetta una closure che rappresenta la computazione che il thread deve svolgere. Ritorna una struct di tipo `std::thread::JoinHandle<T>` dove **T** rappresenta il tipo restituito dalla computazione del thread. Per sapere quando la computazione è terminata e quale valore abbia prodotto, occorre utilizzare il metodo `join()` offerto dalla handle. Tale metodo restituisce un'enumerazione di tipo `std::thread::Result` che contiene, nell'opzione **Ok**, il valore finale e nell'opzione **Err**, il valore eventualmente passato alla macro **panic!**, nel caso in cui sia stata invocata nel corso della computazione dal thread stesso. Non si crea nessun rapporto di parentela tra thread creatore e thread creato, né occorre che l'uno sopravviva all'altro. Il thread creato non sa neanche chi è il thread creatore. Quando l'handle di un thread esce dallo scope e viene rilasciato, non c'è più modo di avere notizie sull'esito del thread creato, che acquisisce lo stato di detached. La standard library di Rust assegna ad ogni thread un identificatore accessibile attraverso la chiamata `thread::current().id()`:

```
use std::thread;

fn main() {
    let t1 = thread::spawn(f);
    let t2 = thread::spawn(f);

    println!("Hello from the main thread.");
    t1.join().unwrap();
    t2.join().unwrap();
    println!("End.");
}
```

```
fn f() {
    println!("Hello from another thread!");

    let id = thread::current().id();
    println!("This is my thread ID: {id:?}");
}
```

E' possibile configurare un thread prima di lanciare la sua esecuzione tramite la struct **std::thread::Builder**. Permette di assegnare al thread un nome a scelta e di definire la dimensione dello stack da associare al thread. Il metodo **spawn(...)** consuma l'oggetto Builder, crea il thread corrispondente e restituisce un enum di tipo **io::Result<JoinHandle>**

Nell'ambito del proprio sforzo di garantire la correttezza degli accessi alla memoria e l'assenza di comportamenti non definiti, Rust introduce due tratti marker il cui scopo è fornire indicazioni sul comportamento di un tipo in un contesto multi-thread. Il tratto **std::marker::Send** è applicato automaticamente a tutti i tipi che possono essere trasferiti in sicurezza da un thread ad un altro, ovvero in grado di garantire che non è possibile avere accessi al loro contenuto **contemporaneamente**. Il tratto **std::marker::Sync** è applicato automaticamente a tutti i tipi **T** tali che **&T** risulta avere il tratto **Send**, ovvero che **possono essere condivisi** in sicurezza tra thread differenti, senza creare problemi di comportamenti non definiti. Riferimenti, `RefCell` e puntatori nativi **non hanno** il tratto **Send**. L'esecuzione indipendente dei thread non consente infatti al borrow checker di fornire le proprie garanzie di correttezza. `Rc` **non ha** il tratto **Send**, implica operazioni di aggiornamento che causano race condition.

I tipi elementari godono del tratto **Send**. I tipi composti godono del tratto **Send** se tutti i loro campi lo posseggono. Se un tipo dispone del tratto **Send**, è lecito passarlo **per valore** ad altri thread. L'uso del movimento garantisce la non contemporaneità degli accessi. `Rf` e `RefCell` non possono essere usati in contesto multi-thread. Se un tipo dispone del tratto **Sync**, è lecito passarlo **come riferimento non mutabile** ad altri thread o, in altre parole, un tipo **Sync** se è lecito accedervi in modo concorrente a partire da un riferimento mutabile. I tipi che implementano una mutabilità interna (come `Cell`) non dispongono del tratto **Sync**.

E' possibile creare thread solo se i dati catturati dalla funzione lambda che ne descrive la computazione e **il suo tipo di ritorno** hanno il tratto **Send**. In mancanza di questa garanzia, il borrow checker genera un errore di compilazione, rilevando la propria impossibilità a garantire la correttezza di quanto si sta chiedendo di eseguire. Rust non lascia spostare un riferimento con durata limitata (non **static**) in un thread perché non può garantirne la validità nel thread.

Quando il thread principale termina, fa terminare tutti gli eventuali thread ancora attivi. Se un thread viene creato mediante la primitiva **std::thread::spawn()** il compilatore non può fare assunzioni sulla sua durata. Di conseguenza, impedisce l'utilizzo di riferimenti condivisi tra la

funzione lambda del thread e la funzione all'interno della quale il thread viene creato. La libreria standard offre un ulteriore modo di creare un thread, tramite la funzione **`std::thread::scope(|s: std::thread::Scope| { ... })`**, essa accetta come parametro una funzione lambda il cui compito è racchiudere l'intero ciclo di vita dei thread creati al suo interno. Il parametro passato a tale funzione offre il metodo **`.spawn(...)`** mediante il quale è possibile creare nuovi thread. Terminata l'esecuzione della funzione lambda, la funzione **`scope(...)`** non ritorna fino a che tutti i thread creati al suo interno non sono terminati. Questo permette al borrow checker di considerare corretto l'uso di riferimenti a variabili locali, dato che la loro durata sarà almeno pari a quella della funzione **`scope(...)`** e, di conseguenza, a quella dei thread creati al suo interno.

La libreria standard di Rust supporta due modelli base per la realizzazione di programmi concorrenti:

1. La condivisione di dati basata su sincronizzazione degli accessi ad una **struttura dati condivisa**, a cui tutti i thread interessati possono accedere in lettura e scrittura.
2. La condivisione di dati basata sullo **scambio di messaggi** che prevede uno o più mittenti ed un solo destinatario

Sono inoltre disponibili librerie esterne che supportano ulteriori modelli:

- La libreria **actix** supporta il modello degli attori
- La libreria **rayon** supporta il modello work stealing
- La libreria **crossbeam** permette la condivisione di dati memorizzati nello stack del thread genitori con i thread figli

Per poter condividere dati modificabili tra thread differenti, occorre disporre di un qualche **meccanismo che permetta, ad un solo thread alla volta, di acquisire il permesso di modifica**, bloccando lo svolgimento degli altri thread che dovessero richiedere accesso alla stessa risorsa. Il modo più semplice di ottenere questo comportamento è attraverso l'uso di un **mutex**. Costrutti di questo tipo sono offerti nativamente dai sistemi operativi e sono riesportati in modo indipendente dalla piattaforma dalle librerie standard C++ e Rust. Gli oggetti nativi offerti dai sistemi operativi offrono due metodi: **`lock()`** e **`unlock()`**. Invocando **`lock()`**, un thread richiede il possesso del mutex: se questo non può essere garantito al momento, perchè il mutex è in uso ad un altro thread, **l'invocazione si blocca** fino a che il mutex non è stato rilasciato dall'attuale possessore. E' lecito invocare **`unlock()`** solo se il thread che lo esegue è l'attuale possessore del mutex.

Entrambi i metodi **lock()** e **unlock()** includono una barriera di memoria. Essa garantisce la visibilità delle operazioni eseguite fino a quel punto dagli altri thread e permette di imporre la dipendenza causale. Sul piano pratico, occorre associare ad ogni risorsa condivisa un mutex che deve essere **sempre** acquisito prima di fare accesso alla risorsa. Nelle astrazioni base offerte dai sistemi operativi e nell'implementazione offerta in C++, **non c'è una corrispondenza sintattica tra un mutex e la struttura dati che questo protegge**. La relazione, in questi ambienti, è nella mente del programmatore. Un mutex può, in linea di principio, proteggere molte strutture diverse, ma riduce il grado di parallelismo complessivo del programma.

L'accesso ad uno stato condiviso in Rust richiede l'utilizzo di **due blocchi in cascata**:

- Il **primo** volto a permettere il possesso multiplo di una struttura dati in sola lettura da parte di più thread, realizzato mediante il costrutto **std::sync::Arc<T>**
- Il **secondo** che consente l'acquisizione in lettura/scrittura della struttura dati, realizzato alternativamente mediante il costrutto **std::sync::Mutex<T>**, con **std::sync::RwLock<T>** oppure ricorrendo ai **tipi atomici**.

Questa combinazione che prende spunto dal pattern RAI, permette di **rendere esplicito** nella struttura del codice e nei pattern di accesso ai dati **cosa** sia condiviso e **impedisce** di fatto **l'accesso senza** il corretto **possesso** del lock relativo, permettendo al compilatore di bloccare ogni tentativo di accesso non conforme.

Un oggetto di tipo **Mutex** incapsula un dato di tipo **T** oltre al riferimento ad un mutex nativo del sistema operativo. L'unico modo per accedere il dato è invocare il metodo **lock()**. Questo metodo restituisce un oggetto di tipo **LockResult<MutexGuard<T>>** e resta bloccato fino a che non è stato possibile acquisire il mutex nativo. Se l'ultimo thread che ha acquisito il mutex fosse terminato prima di averlo rilasciato, il mutex si troverebbe nello stato **avvelenato**, e la risposta conterrebbe un errore. Se il metodo **lock()** ha successo, la risposta contiene un **MutexGuard<T>**. Tale oggetto implementa il tratto **Deref<T>** e si comporta come uno smart pointer, dereferenziandolo si ottiene un riferimento mutabile al dato **T**. Quando il **MutexGuard<T>** esce dallo scope, il mutex nativo viene rilasciato, permettendo ad altri thread di chiederne il possesso. A tutti gli effetti **MutexGuard<T>** implementa il pattern RAI, ma, per come viene costruito, è necessariamente disponibile solo se si possiede il mutex.

Il **lock()** permette di accedere al dato condiviso e garantisce che la cache venga pulita e dunque si va a leggere il dato direttamente dalla memoria. Quando si esce dal **lock()** viene effettuata la flush della cache garantendo che il dato sia trasferito in memoria prima che altri possano leggerlo.

Un oggetto di tipo **Mutex** può avere un solo possessore. Per superare questo vincolo, lo si incapsula all'interno di un oggetto di tipo **std::sync::Arc<T>**. **Arc<T>** permette di

condividere il possesso di un dato, allocandolo nello heap e mantenendo un conteggio dei riferimenti esistenti di tipo **thread-safe**. E' possibile duplicare un oggetto di questo tipo attraverso il metodo **clone()**, tale metodo si limita a duplicare il puntatore al blocco sullo heap, avendo cura di incrementare il contatore dei riferimenti associati al dato. Il dato clonato viene ceduto ad un thread specificando la parola-chiave `move` di fronte alla funzione lambda che ne descrive la computazione.

Il campo **poison** in un Mutex è un meccanismo per gestire situazioni in cui un thread proprietario di un blocco del Mutex termina in modo anomalo senza rilasciare il blocco. Quando ciò accade, il Mutex entra in uno stato di **veleno**, il che significa che il blocco non è stato rilasciato correttamente. Questo stato è segnalato al prossimo thread che cerca di ottenere il blocco. Quando un thread tenta di acquisire il blocco dopo che è stato rilasciato in modo anomalo, il Mutex solleva un errore per segnalare la situazione di **poisoning**. Questo aiuta ad identificare e gestire situazioni in cui un thread potrebbe aver lasciato il blocco del Mutex in uno stato inconsistente, aiutando a garantire l'integrità dei dati condivisi. E' però possibile accedere al dato protetto dal Mutex anche nel caso di Mutex avvelenato attraverso la lettura dell'errore, mediante il metodo **into\_inner()**.

Se gli accessi in lettura e in scrittura sono sbilanciati, può essere conveniente sostituire alla **struct Mutex<T>**, la **struct RwLock<T>**. Questa offre il metodo **read()** per accedere, in modo condiviso, in lettura ed il metodo **write()** per accedere in modo esclusivo in scrittura. I metodi **read()** e **write()** restituiscono rispettivamente oggetti di tipo **LockResult<RwLockReadGuard>** e **LockResult<RwLockWriteGuard>**. Il risultato contiene un errore se il lock è avvelenato. Questo capita se un thread che lo possedeva è terminato senza averlo rilasciato o cercando di acquisirlo ulteriormente. Entrambi gli oggetti di guardia implementano il pattern RAII, rilasciando il lock nel momento in cui sono distrutti.

Il modulo **std::sync::atomic** mette a disposizione alcune strutture dati che costituiscono primitive di comunicazione tra thread basate sul principio della memoria condivisa. Esso offre versioni atomiche di valori booleani, numeri interi con e senza segno e puntatori nativi (**AtomicBool**, **AtomicIsize**, **AtomicUsize**, **AtomicI8**, **AtomicU8**, **AtomicI16**, **AtomicU16**, **AtomicI32**, **AtomicU32**, **AtomicI64**, **AtomicU64**). Ciascun tipo è associato ad operazione che, se usate correttamente, permettono di sincronizzare gli aggiornamenti di tali valori (read and modify) tra thread differenti. L'operazione di aggiornamento diventa indivisibile, evitando **undefined behavior**

Permettono operazioni di lettura **load** e scrittura **store** con associata barriera di memoria:

```
impl AtomicI32 {  
    pub fn load(&self, ordering: Ordering) -> i32;  
    pub fn store(&self, value: i32, ordering: Ordering);  
}
```

Ogni operazione atomica definisce un argomento di tipo **std::sync::atomic::Ordering** che specifica come le operazioni atomiche sincronizzano la memoria tra i thread. Le possibili varianti sono:

- **Relaxed**: questa è la variante più debole. Non ha vincoli di ordinamento
- **Release**: applicabile solo per operazioni che eseguono una **scrittura** e determina che tutte le operazioni non atomiche eseguite prima dell'operazione atomica **non** vengano riordinate **dopo** di essa
- **Acquire**: applicabile solo per operazioni che eseguono una **lettura** e determina che tutte le operazioni non atomiche eseguite dopo l'operazione atomica **non** vengano riordinate **prima** di essa
- **AcqRel**: applicabile solo per operazioni che combinano sia **letture** che **scritture** e garantisce sia l'acquire che il release
- **SeqCst**: impone che tutti i thread vedono tutte le operazioni sequenzialmente consistenti nello stesso ordine

Sebbene siano tutti **thread-safe** (implementano il tratto **Sync**), non offrono meccanismi di condivisione esplicita. Come tutti i valori in Rust, sono soggetti alla regola del possessore unico. Per permettere a più thread di accedere al loro valore è comune incapsularli all'interno di un elemento di tipo **Arc<T>** oppure dichiararli come **variabili globali**, attraverso la parola chiave **static**. Implementano il meccanismo di mutabilità interna. Poiché le operazioni di modifica sono garantite essere **thread-safe**, i metodi che ne modificano il contenuto richiedono solo un accesso condiviso (**&self**) e non un accesso esclusivo (**&mut self**).

I tipi atomici **fetch\_modify** offrono funzionalità di tipo **Read-Modify-Write** modificano la variabile atomica, ma leggono anche il valore originale in una singola istruzione atomica. Questa operazione controlla, atomicamente, se il valore atomico è uguale ad un determinato valore e solo in tal caso lo sostituisce con un nuovo valore. Restituirà il valore precedente e ci dirà se lo ha sostituito o meno.

Il thread in attesa consuma tempo di CPU **attivamente**. Gli **spinlock** sono efficienti solo quando la sezione critica è molto breve e la probabilità di contesa è bassa. In caso contrario, i thread in attesa consumano cicli di CPU inutilmente. E' possibile che si verifichi un **livelock** se più thread tentano ripetutamente di acquisire il blocco ma falliscono continuamente, continuando a **spinnare** senza

fare progressi. Gli spinlock non garantiscono la fairness. Un thread potrebbe attendere più a lungo di altri per acquisire il blocco.

**Le variabili globali possono essere solo immutabili.** Variabili globali mutabili possono essere solo inserite in blocchi **unsafe**.

Analogamente a quanto succede con gli smart pointer di tipo **Rc<T>**, anche nel caso di **Arc<T>** la creazione di catene circolari impedisce il rilascio delle strutture. Per questo motivo è disponibile la **struct std::sync::Weak<T>** che permette di realizzare dipendenze circolari con riferimenti che non partecipano al conteggio, garantendo così la possibilità di rilascio. Per fare accesso al dato puntato, occorre invocare il metodo **upgrade()**, che restituisce un valore di tipo **Option<Arc<T>>**. Si crea un oggetto di tipo **Weak<T>** a partire da un riferimento di tipo **Arc<T>** invocando su quest'ultimo il metodo **downgrade()**.

Spesso un thread deve aspettare un risultato intermedio prodotto da altri thread e, per motivi di efficienza, l'attesa non deve consumare risorse e deve terminare non appena un dato è disponibile. Una soluzione potrebbe basarsi sul polling del valore del risultato. La presenza di dati condivisi richiede come minimo l'utilizzo di un mutex per garantire l'assenza di interferenze tra i due thread che devono fare accesso ai dati. Il polling ha due limiti:

- Consuma capacità di calcolo e batteria in cicli inutili
- Introduce una latenza tra il momento in cui il dato è disponibile e il momento in cui il secondo thread si blocca

Per gestire queste situazioni, i sistemi operativi offrono il concetto di **condition variable**, strutture dati di sincronizzazione che permettono di bloccare l'esecuzione di un thread, così da evitare il consumo di CPU, nell'attesa che qualcosa succeda.

L'uso di una **condition variable** è basata sulla cooperazione all'interno del sistema. Se un thread si sospende in attesa di una condizione, è necessario che tutti i thread che eseguono azioni che potrebbero provocare il verificarsi della condizione si facciano carico di inviare una notifica alla condition variable. Il pattern di utilizzo prevede che esista un'espressione booleana il cui valore possa essere usato per determinare se occorre attendere o meno. La valutazione di tale espressione deve avvenire mentre si possiede un mutex, per garantire l'assenza di corse critiche. In Rust, questo vuol dire che le variabili che consentono la valutazione della condizione sono incapsulate nel mutex. Ogni **condition variable** deve essere usata in coppia con un singolo mutex. Eventuali tentativi di usare mutex diversi per una stessa condition variable può determinare un fallimento in fase di esecuzione.



C++ offre la classe **std::condition\_variable**. Essa viene usata in coppia con un oggetto di tipo **std::mutex** racchiuso all'interno di un oggetto di tipo **std::unique\_lock<std::mutex>**. Rust offre la struct **std::sync::Condvar**. La sua semantica è totalmente allineata con la corrispondente classe C++. La struttura dei suoi metodi facilita il collegamento con il dato protetto dal mutex rendendo più naturale il suo utilizzo. I metodi principali di **Condvar** sono:

- **new()**: crea una nuova istanza
- **wait()**: sospende il thread corrente fino alla ricezione di una notifica. Durante la sospensione, rilascia il lock; al ricevere della notifica, riacquisisce il lock e restituisce una nuova guardia
- **notify\_one()**: sveglia un thread a caso tra quelli in attesa sulla condition variable
- **notify\_all()**: sveglia tutti i thread in attesa sulla condition variable, che usciranno, uno alla volta, dal metodo **wait** possedendo il lock

Concettualmente, una condition variable mantiene una collezione di thread in attesa che si verifichi la condizione attesa. Inizialmente la lista è vuota. Quando un thread esegue il metodo **wait(...)**, viene sospeso e aggiunto alla lista. Quando sono eseguiti i metodi **notify\_one()** o **notify\_all()**, uno o tutti i thread presenti nella collezione sono risvegliati. Si basa sul Sistema Operativo per sospendere/risvegliare i thread. La presenza di un unico lock fa sì che, se più thread ricevono la notifica, il risveglio sia progressivo. Non appena un thread rilascia il lock, un altro può acquisirlo e proseguire. La relazione tra l'evento e la notifica è solo nella testa del programmatore. Per questo, si rende esplicito l'evento che si è verificato appoggiandosi ad una o più variabili condivise.

E' possibile che un thread in attesa su una condition variable sia risvegliato in assenza di un'esplicita notifica. Problema delle cosiddette **notifiche spurie**. Occorre, al ritorno dal metodo **wait()**, controllare se la condizione attesa è verificata. Per semplificare tale verifica, esiste una versione del metodo di attesa che riceve come argomento una funzione volta a valutare il predicato richiesto, ovvero **wait\_while()**. Al risveglio, riacquisisce il lock e valuta la funzione **condition:** se questa restituisce **true**, si riaddormenta, altrimenti esce dall'attesa.

Analogamente, se un thread ha eseguito una qualche azione che può abilitare la prosecuzione di un altro thread, ed invoca il metodo **notify\_one()** o **notify\_all()** per segnalare tale fatto, è possibile che la notifica vada persa. Succede se l'altro thread **non ha ancora eseguito** la corrispondente istruzione di attesa. Per questo motivo, occorre sempre racchiudere l'istruzione di attesa in un ciclo che verifica se occorre o meno addormentarsi e, al risveglio, se ci siano le condizioni o meno per continuare a dormire. In entrambi i casi, il metodo **wait\_while()** protegge.

Altri metodi permettono di limitare il tempo massimo di attesa, permettendo al thread di risvegliarsi anche in assenza del verificarsi della condizione, ma a seguito dello scadere di un time-out. Come **wait**, il lock specificato verrà riacquisito all'uscita della funzione **wait\_timeout\***. Restituisce una tupla, in cui il primo campo è un `MutexGuard` e permette di verificare se la condizione si è verificata e il secondo campo fornisce un'informazione sulla condizione di time-out. I metodi sono:

- **wait\_timeout():** attende per un tempo massimo pari a **Duration**
- **wait\_timeout\_while():** attende per un tempo massimo pari a **Duration**; eventuali notifiche ricevute portano a ri-addormentarsi se la funzione **condition** restituisce `false`

Le barriere (**Barrier**) sono strumenti di sincronizzazione che consentono a un certo numero di thread di attendersi l'un l'altro ad un determinato punto di sincronizzazione. In Rust, la libreria standard **std::sync::Barrier** offre una `Barrier` che può essere utilizzata per questo scopo. Un thread che arriva al punto di sincronizzazione chiama la funzione **wait()**

**OnceLock** è una struttura fornita dal modulo **std::sync::OnceLock** che offre un modo sicuro e conveniente per gestire l'inizializzazione lazy di valori statici o globali in un contesto multithread. `OnceLock` utilizza meccanismi di sincronizzazione a basso livello per garantire che la chiusura di inizializzazione venga eseguita atomicamente e una sola volta. I metodi utili di `OnceLock` sono:

- **new():** crea una nuova istanza di `OnceLock` nello stato non inizializzato
- **get():** restituisce un `Option<&'static T>` al valore memorizzato. Restituisce `None` se il valore non è ancora stato inizializzato.
- **get\_or\_init(f: F):** se il valore non è ancora stato inizializzato, chiama la chiusura `f`, memorizza il risultato e restituisce un riferimento a esso. Se il valore è già stato inizializzato, restituisce direttamente un riferimento al valore esistente senza chiamare la chiusura
- **set(value: T):** tenta di impostare il valore della `OnceLock` con `value`. Restituisce `Ok(())` se l'impostazione ha avuto successo e `Err(value)` se la `OnceLock` era già stata inizializzata. Questo metodo è utile se l'inizializzazione avviene in un punto diverso dalla prima richiesta del valore
- **is\_initialized():** restituisce `true` se il valore è già stato inizializzato, `false` altrimenti
- **take():** consuma la `OnceLock`, restituendo un `Option<T>` contenente il valore e lasciando la `OnceLock` in uno stato non inizializzato. Questo metodo richiede che la `OnceLock` non sia una `static`

## Canali

Abbiamo due tipologie di modelli di concorrenza:

- **Struttura dati condivisa:** ci si sincronizza per comunicare
- **Scambio di messaggi:** si comunica per sincronizzarsi

In alternativa alla condivisione dello stato, Rust offre un meccanismo di comunicazione e sincronizzazione tra i thread basato sulla condivisione di messaggi. La funzione `std::sync::mpsc::channel<T>()` restituisce una coppia ordinata (tupla) formata da una `struct Sender<T>` ed una `struct Receiver<T>`. Tutti i dati inviati tramite il metodo `send()` della prima possono essere consumati attraverso il metodo `recv()` della seconda, nello stesso ordine in cui sono stati inviati. Il metodo `send()` offre la garanzia che chi lo invoca **non** sarà **bloccato** (ovvero il canale di comunicazione ha una capacità infinita di memorizzazione temporanea dei messaggi). Il metodo `recv()` si blocca senza consumare cicli di macchina in attesa di un messaggio o della terminazione dell'oggetto `Sender` e di tutti i suoi eventuali cloni. Il dato trasmesso deve essere di tipo `T` e deve godere del tratto **Send**. L'implementazione fornita gode della proprietà **multipl producer - single consumer** ovvero permette di creare più cloni dell'oggetto `sender`, mentre obbliga ad avere una singola copia dell'oggetto `receiver`.

In questo modello di comunicazione, **il singolo dato prodotto da un thread viene ceduto al canale e da questo al thread ricevente che diventa il possessore finale del valore**. Questa operazione agisce al tempo stesso da **sincronizzazione** (la ricezione è necessariamente successiva all'invio, ossia non posso leggere fino a che qualcuno non abbia scritto) e da **comunicazione** (il dato passato rappresenta l'unità di messaggio). Un numero arbitrario di messaggi può essere scambiato sul canale, a condizione che il ricevitore sia attivo. Se il ricevitore viene deallocato, eventuali tentativi di invio falliscono con la generazione di un valore di tipo `SendError<T>`. Se tutti i trasmettitori vengono deallocati, tentativi di lettura sul ricevitore falliscono con la generazione di un valore di tipo `RecvError`.

Un **channel** è un tubo a 2 estremità: quella di ingresso e quella di uscita. All'interno del channel possono essere presenti dei messaggi. Chi conosce l'estremità di ingresso può produrre un dato e cederlo al canale. Il dato viene prodotto e affidato al canale. Sta nel canale fino a quando qualcun altro, nell'estremità di uscita non cerca di leggerlo. Chi cerca di leggerlo acquisisce il possesso di quel dato e ci fa quello che vuole. Il ricevitore legge il valore solo se un trasmettitore lo ha prodotto. Contemporaneamente ottengo 2 informazioni: so che l'altro ha fatto un certo lavoro e ottengo un dato corrispondente al lavoro svolto.

Operativamente il flusso è il seguente: preparo il channel prima di creare i 2 thread che hanno bisogno di parlarsi. Do l'estremità di ingresso al thread sender e quella di uscita al thread receiver. Il canale conserva l'ordine con cui sono stati scritti i messaggi. Chi legge, legge alla velocità che può. Finché non c'è nulla nel tubo, rimane bloccato, senza consumare CPU. Non ho bisogno di usare la condition variable, perché se il canale è vuoto, l'operazione di lettura mi sospende automaticamente. Appena qualcuno inietta qualcosa nel canale, se io sono in attesa, mi sveglio e lo prendo. Se non sono in attesa, sta nel canale. Vado nel canale, prendo il dato e poi torno nel canale a vedere se c'è un altro dato.

Il Channel ci offre un modo per far sapere al receiver che non potrà più ricevere nulla perché non ci sono più sender: non appena tutti i sender vanno via, il receiver esce con un errore che dice che non ci sono più sender e quindi può smettere, senza aver bisogno di sapere a priori di quanti pezzi dovrà aspettare. Parallelamente, anche il sender riceve una indicazione. Se l'unico receiver andasse via (ossia viene distrutto e se ne fa il drop), un tentativo di invio fallisce poiché il channel sa che non c'è più la possibilità di ricevere il dato

Il producer trasmette un dato che sta possedendo e lo cede al canale. Questo permette al producer di non avere nessuna idea dell'identità del consumer. L'unica cosa che conosce il producer è il canale. Il canale lo renderà disponibile non appena possibile. Il canale non è limitato. Io posso mettere nel canale quanti messaggi voglio (**unbounded**). Se quell'altro non sta leggendo perché è occupato, i dati stanno nel canale. E' come se fosse un tubo che si allunga per contenere i dati. Appena il ricevitore chiede di leggere, legge il primo (quello più vecchio). Il canale mantiene l'ordine di ricezione. I messaggi vengono letti nell'ordine con cui sono stati inseriti.

Quando l'ultimo sender esce di scena e il canale diventa vuoto. Il receiver riceve un **error (RecvError)**. Questo permette di capire, da parte del receiver, che lui deve finire. Non basta che siano morti tutti i sender, se ho ancora dei messaggi nel canale, questi vengono svuotati. Il canale mi garantisce che tutto quello che lui ha preso in carico verrà smaltito e lo consegnerà (ammesso che vi sia un receiver). Parallelamente il sender sa che tutto quello che mette nel canale finirà a destinazione, a patto che ci sia una destinazione. Se per qualche motivo la destinazione cessa di esistere, perché viene fatto il drop, ciò che c'è già nel canale, va perduto (perché nessuno è più in grado di leggerlo) e non possono più aggiungere nulla. Ogni eventuale tentativo di aggiunta ulteriore dà origine ad un **SendError** che mi informa che non c'è più un receiver.

Il tratto **IntoIterator** per **Receiver<T>** fornisce 3 metodi per ottenere un iteratore:

- **into\_iter(self)**: consuma il Receiver. Quando viene chiamato `next()`, viene bloccato il thread chiamante fino a quando non è disponibile un nuovo messaggio, oppure fino a quando tutti i Sender sono stati droppati.
- **iter(&self)**: prende un riferimento al Receiver che non viene consumato. Anche questo iteratore blocca il thread chiamante in attesa di nuovi messaggi e restituisce `None` quando tutti i Sender sono stati droppati.
- **try\_iter(&self)**: iteratore che non blocca. Restituisce `None` se il canale è vuoto o se non ci sono più sender

La struct **Receiver<T>** offre il metodo **recv\_timeout(d: Duration)**. Esso consente di limitare il tempo massimo di attesa alla durata passata come parametro. Questo metodo può restituire due tipi di errore:

- **Timeout**: per indicare che non è stato ricevuto alcun messaggio entro il tempo indicato
- **Disconnected**: per indicare che il canale è vuoto e non sono più disponibili **Sender** in grado di spedire messaggi

Quando occorre non bloccarsi nell'attesa di nuovi messaggi, è possibile utilizzare il metodo **try\_recv()**. Se nessun messaggio è presente, il metodo torna subito con un errore di tipo **Empty**. Se non sono più disponibili **Sender**, il metodo restituisce un errore di tipo **Disconnected**

La funzione **sync\_channel<T>(bound: usize)** restituisce invece una coppia di valori di tipo **(SyncSender<T>, Receiver<T>)**. A differenza di un canale semplice, questo è limitato: se il numero di messaggi giacenti nel canale raggiunge il limite definito (**bound**) le invocazioni del metodo **send()** diventano **bloccanti** fino a che non si libera un posto eseguendo una lettura con successo. Se viene costruito un canale sincrono di dimensione **0**, diventa un canale di tipo **rendezvous**: ogni operazione di lettura deve sovrapporsi temporalmente ad una di scrittura. Le restanti operazioni offerte da **SyncSender<T>** hanno semantica simile alle corrispondenti offerte da **Sender<T>**

La libreria **Crossbeam** è una variegata libreria ben documentata e attivamente mantenuta che offre una serie di costrutti a supporto dell'elaborazione concorrente:

- **Costrutti atomici**: la struct **AtomicCell<T>** estende il concetto di mutabilità interna offerto da **Cell<T>** a contesti **multithread**, appoggiandosi a **primitive atomiche** là dove possibile, oppure ricorrendo all'uso di un lock interno per strutture dati più articolate.
- **Strutture dati concorrenti**: le struct del crate (**Injector**, **Stealer** e **Worker**) offrono un meccanismo strutturato per la creazione di schedulatori basati sul frutto di

attività da eseguire; le struct (**ArrayQueue**, **SeqQueue**) implementano code di messaggi (limitate o illimitate) basate sul paradigma **multiple-producer-multiple-consumer**

- **Canali MPMC**: le funzioni **bounded()** e **unbounded()** creano canali unidirezionali con capacità limitata o illimitata basati sul paradigma MPMC i cui estremi possono essere condivisi per semplice clonazione. Le funzioni **after()**, **tick()** creano il solo estremo di ricezione che consegnerà un messaggio dopo il tempo indicato o periodicamente

**AtomicCell** è una primitiva utile quando si ha bisogno di scambiare valori tra thread in modo atomico. A differenza di **Atomic\***, che lavora solo su tipi interi o booleani, questa può contenere qualsiasi tipo che implementi il tratto **Send**

Il paradigma **MultipleProducersMultipleConsumers** offre un meccanismo potente per l'implementazione di pattern concorrenti in Rust. Il messaggio è sempre consumato una volta sola, se ci sono 2 thread che cercano di leggerlo, il messaggio viene letto solo da 1 dei 2. Va bene, dunque, se non mi interessa sapere chi andrà effettivamente a leggere il messaggio

**Fan-out/Fan-in** permette di distribuire attività a più thread indipendenti e raccogliere i risultati prodotti in un singolo punto; usa una coppia di canali per distribuire e raccogliere i dati. **Pipeline** crea una serie di fasi di lavorazione, ciascuna delle quali è eseguita da un singolo thread e utilizza un canale per inoltrare i semilavorati tra due fasi successive. **Producer/Consumer** consente ad uno o più thread produttori di generare valori che saranno elaborati dal primo thread consumatore disponibile; usa un singolo canale per la comunicazione

La funzione **after(duration)** crea un ricevitore che acquisisce un singolo messaggio (**n()**) dopo una certa durata di tempo (**duration**). La funzione **tick(duration)** crea un ricevitore che invia un messaggio (**un()**) ad una determinata frequenza di trasmissione (tempo pari a **duration**)

Prima di introdurre concorrenza, chiedersi sempre:

- I thread devono **condividere uno stato** o **scambiarsi dei messaggi**?
- Serve **mutua esclusione**, **lettura condivisa** o **sincronizzazione su un evento**?
- Il programma è corretto solo per **alcuni** modi con cui i thread possono alternarsi o per **tutti**?
- Ci sono rischi di **deadlock**, **livelock** o **starvation**?
- Esiste una soluzione **più semplice** con meno condivisione?

**crossbeam::queue** è un modulo della libreria `Crossbeam` che fornisce 2 tipi di code che permettono accessi concorrenti (thread-safe) in modalità MPMC:

- **SegQueue<T>**: coda non bloccante FIFO che offre i seguenti principali metodi:
  - **push(item)**: inserisce un elemento in coda
  - **pop()**: rimuove un elemento dalla testa della coda. Ritorna `Err` se vuota
  - **is\_empty()**: verifica se la coda è vuota
- **ArrayQueue<T>**: coda circolare a capacità fissa:
  - **push(item)**: inserisce un elemento nella coda e ritorna `Err` se la coda è piena (non blocca)
  - **pop()**: rimuove un elemento dalla testa della coda. Ritorna `None` se vuota (non blocca)
  - **is\_empty()**: verifica se la coda è vuota
  - **len()**: ritorna il numero degli elementi presenti

**crossbeam::deque** ci consente di utilizzare il **work stealing**, una tecnica di bilanciamento del carico utilizzata in sistemi di calcolo parallelo per migliorare l'efficienza e l'utilizzo dei processori. Il work stealing è un metodo in cui i thread che rimangono inattivi rubano lavoro dai thread che sono ancora occupati. Questo meccanismo aiuta a bilanciare il carico di lavoro tra i thread, assicurando che tutti i processori disponibili siano utilizzati in modo efficiente. Come funziona:

1. Ogni thread riceve una coda di lavoro contenente task ed esegue i task dalla propria coda
2. Se un thread esaurisce i task nella propria coda, diventa inattivo e tenta di rubare task dalle code di altri thread
3. Un thread inattivo sceglie casualmente un altro thread e cerca di rubare un task dalla coda di quest'ultimo. Il furto di solito avviene dalla coda opposta rispetto alla direzione normale di estrazione del task, per minimizzare il conflitto (ad esempio FIFO per il thread proprietario e LIFO per il ladro)

Il crate **deque** permette di creare una serie di code:

- **Worker<T>**: una coda deque (double-ended queue) locale ad un thread
- **Stealer<T>**: una handle che consente ad altri thread di rubare il lavoro dalla parte iniziale della coda Worker
- **Injector<T>**: una coda globale condivisa usata per iniettare nuovi task, accessibile da tutti i thread

La libreria **Rayon** è una libreria di Rust che semplifica enormemente la programmazione parallela, in particolare la parallelizzazione di dati (data parallelism). Uno degli strumenti che offre per il parallelismo è la funzione **join()**, che permette di eseguire due task in parallelo e aspettare che entrambi finiscano prima di procedere. La funzione `join` prende due chiusure e le esegue in

parallelo. Attende che entrambe finiscano e restituisce una tupla con i risultati di entrambe le chiusure. `join` fornisce un modo semplice per eseguire due task in parallelo senza dover gestire esplicitamente i thread. Rayon ottimizza automaticamente l'uso delle risorse hardware, rendendo l'esecuzione parallela efficiente.

Rayon usa un modello di **fork-join**: se ha un compito grande, lo divide in due compiti più piccoli (fork). Questi due compiti possono essere eseguiti in parallelo. Se un compito è ancora troppo grande, viene diviso di nuovo, e così via. Rayon gestisce un suo **pool di thread** (un gruppo di thread pronti a lavorare). Quando un compito viene diviso, una parte viene eseguita dal thread corrente e l'altra parte può essere messa nella coda di lavoro del pool per essere presa da un altro thread. Il metodo `par_iter()` è un iteratore che distribuisce automaticamente il lavoro su più thread, utilizzando il pool di thread gestito da Rayon. Il motore nascosto di Rayon è il **Work-Stealing** e usa **crossbeam::deque**. Se un thread finisce i suoi compiti e altri thread sono ancora occupati, il thread va a rubare compiti dalla coda di un altro thread. Questo assicura che tutti i core siano sempre occupati, bilanciando il carico di lavoro in modo dinamico e molto efficiente.

Si può creare e configurare un pool di thread personalizzato e usarlo per eseguire operazioni parallele:

- **build()** : costruisce il pool di thread con le impostazioni specificate
- **install()** : esegue un blocco di codice nel contesto del pool di thread



## Programmazione Asincrona

Se la necessità di eseguire più operazioni, indipendenti tra loro, **in parallelo** porta alla creazione di più thread, altre esigenze di programmazione possono beneficiare di un approccio differente. L'uso dei thread trova piena giustificazione quando occorre eseguire algoritmi complessi, basati principalmente sull'uso intenso della CPU e si dispone di un hardware multicore. In queste situazioni, infatti, è possibile ridurre il tempo complessivo di elaborazione sfruttando il fatto che più computazioni procedano parallelamente. I costi legati alla programmazione multithread sono costituiti, da un lato, dalla complessità legata ai meccanismi di sincronizzazione e dall'altro, dalla necessità di allocare preventivamente lo stack di esecuzione di ciascun thread. In caso di creazione di molti thread, tale costo diventa significativo e, in certe situazioni, proibitivo.

I casi in cui l'approccio multi-thread può non essere ottimale sono quelli in cui la computazione richieda di ricevere informazioni da un sottosistema separato (che non è sotto il controllo del processo in esecuzione). In queste situazioni l'esecuzione non può continuare e occorre attendere che il sottosistema in questione fornisca le informazioni attese. Normalmente, il sistema operativo rileva la situazione (implementa system call bloccanti) e sosta il thread corrente nello stato **NotRunnable**, sospendendo l'esecuzione fino a che non si verifica la condizione di attesa. **Se il programma deve svolgere altri compiti**, oltre a quello che ha generato l'attesa, si creano tre possibilità:

- **Attendere:** gli altri compiti saranno eseguiti successivamente, dal thread corrente
- **Multithread:** si creano più thread secondari per eseguirli, accollandosi la complessità legata alla loro sincronizzazione
- **Programmazione Asincrona:** si organizza il codice in modo tale da separare la richiesta di eseguire l'operazione, dalle operazioni che dovranno essere fatte quando arriverà la risposta, così da non bloccare l'esecuzione del thread corrente, ammesso che sia altro da fare

Si usano i **thread** quando occorre **elaborare in parallelo**, mentre si usa **l'esecuzione asincrona**, quando occorre **attendere in parallelo**. La programmazione asincrona è molto efficiente per programmi **I/O Bound**.

Il modo più diretto di implementare la terza strategia richiede che le azioni conseguenti ad una data operazione bloccante siano racchiuse in un'apposita funzione (**callback**). Tale funzione riceve come parametro il risultato dell'operazione bloccante e sarà invocata quando il sottosistema che è stato interrogato avrà fornito la propria risposta.

Un tipico modo per implementare operazioni asincrone è basarsi su API ad eventi, ovvero funzioni che permettono di richiedere ad uno **strato soggiacente** (come il SO o una piattaforma di

esecuzione) l'operazione che si intende eseguire, passando una **callback** che dovrà essere invocata nel momento in cui lo strato soggiacente avrà terminato la sua parte di esecuzione.

Questo modo di scrivere il codice, tuttavia, crea grossi problemi quando l'azione che si sta compiendo richiede, **in cascata**, una seconda azione asincrona. Occorre infatti che la callback indicata provveda ad invocare un'ulteriore operazione passando al relativa callback. Le cose si complicano se tale operazione è ciclica e se occorre gestire eventuali errori. Occorre trasformare il codice in una macchina a stati finiti. Gli errori possono originarsi in momenti molto diversi, all'atto dell'invocazione della funzione asincrona o come conseguenza dell'elaborazione asincrona.

Un primo passo che aiuta a semplificare il problema è provare a riscrivere la forma **annidata** delle callback in una forma **lineare** (riscriviamo le callback **una a fianco all'altra**). Appoggiandosi ad una struttura dati che mantenga le informazioni di stato (**future**) che **incapsula il risultato che verrà**.

Questa forma aiuta a vedere come le operazioni che si intendono eseguire possano essere viste come **una macchina a stati finiti**, che evolve **a strappi**. Quando raggiunge uno stato intermedio ritorna e, per continuare, sarà necessario riprenderne l'esecuzione passando come parametro l'esito dell'operazione asincrona che ha causato la sospensione.

La macchina a stati finiti può essere implementata da una chiusura. Essa racchiude il proprio **stato** e tutte le **variabili locali** di cui l'esecuzione ha bisogno. Quando viene eseguita ritorna un valore che indica se ha raggiunto uno degli stati finali o se si trova ancora in uno stato intermedio. Questo permette di non bloccare il thread corrente e procedere con l'esecuzione di altri task. I punti di blocco sono tutti in corrispondenza degli stati intermedi. Questi sono immediatamente preceduti dall'invocazione di una funzione asincrona. Quando uno stato intermedio viene raggiunto, la chiusura ritorna. Perché la macchina a stati possa progredire occorrono due condizioni:

- Che la funzione asincrona invocata scateni qualche attività che possa portare all'evoluzione dello stato
- Che ci sia qualcuno che indichi che l'attesa è terminata e ci sono le condizioni affinché lo stato possa evolvere

Il compilatore Rust supporta esplicitamente la programmazione asincrona grazie all'introduzione di due parole chiave nel linguaggio (**async** e **await**) ed alla presenza di un tipo specifico (**Future**) nella libreria core. Se una funzione o un blocco di codice sono preceduti dalla parola chiave **async**, il compilatore ne analizza il contenuto e lo trasforma in una macchina a stati. La funzione o il blocco vengono ridotti all'inizializzazione di tale macchina a stati. Il tipo restituito (`impl Trait`) viene trasformato da **T** in un **tipo anonimo** (ossia non ne conosciamo il tipo, ma

sappiamo solo che implementa il tratto `Future`) che implementa il tratto **Future** al cui interno è implementata la macchina a stati precedentemente sintetizzata.

Se all'interno del codice della funzione è presente una chiamata ad un'altra funzione asincrona, per poter accedere al suo risultato occorre esplicitamente attenderlo, attraverso l'operatore `.await`. Questo introduce un nuovo stato all'interno della macchina a stati associata alla funzione

**Pin<T>** è un particolare smart pointer che impedisce che la struttura venga mossa, permettendo l'uso di riferimenti relativi e autoreferenziali. **Context** incapsula un oggetto di tipo **Waker**, mediante il quale è possibile notificare all'esecutore quando il metodo `poll()` potrà essere richiamato

Il tratto **Future** viene implementato dagli oggetti che descrivono una computazione asincrona. Tale computazione può essere ancora in corso o essere terminata. Il tratto offre un singolo metodo, `poll()`, che implementa la logica della macchina a stati associata alla funzione. L'oggetto che implementa il tratto mantiene al suo interno lo stato corrente. Il metodo `poll()` restituisce `enum` che può assumere due soli valori:

- **Pending**: per indicare che la computazione è ancora in corso
- **Ready (val)**: per indicare che la computazione è terminata e ha come risultato **val**

Un oggetto che implementa questo tratto è inerte (**lazy**). Affinché la computazione proceda, occorre che qualcuno ne invochi il metodo `poll()`.

Per ciascuno stato individuato, il compilatore sintetizza una struct in cui memorizzare le variabili locali necessarie per consentire l'esecuzione. Inoltre genera una enum che racchiude i possibili stati e implementa il tratto `Future`. Il codice generato dal compilatore per la funzione si riduce all'inizializzazione della macchina a stati.

**Chi invoca una funzione asincrona, ottiene come risultato un Future nel proprio stato iniziale e** affinché possa capitare qualcosa, occorre che ne venga invocato il metodo `poll()`. Chiamando la funzione con il metodo `.await()` ci garantisce che viene chiamato `poll()`. Se la funzione asincrona è chiamata all'interno di un'altra funzione asincrona, diventa automaticamente parte della macchina a stati del chiamante. Se si invoca una funzione asincrona all'interno di una funzione **normale**, occorre gestire il risultato di tipo **Future** in modo esplicito.

Il compilatore supporta la generazione automatica dei tipi che implementano la macchina a stati associata ad una funzione asincrona e la riscrittura delle funzioni in modo opportuno, ma nessun

supporto è invece offerto dal linguaggio per la gestione dell'esecuzione. Lascia il compito al programmatore di richiamare `poll()` quando è ora per fare in modo che il programmatore adotti una strategia che si adatta al tipo di contesto. Rust fornisce delle librerie di supporto per svolgere il compito di scrivere un **Executor**, codice responsabile di capire quando è ora di chiamare la `poll()`:

- **Tokio**: l'ambiente più diffuso, con supporto per connessioni di rete, database. Molto potente e versatile, ma consuma molta memoria
- **smol**: un ambiente semplificato, a basso impatto sulle risorse adatto a sistemi embedded
- **async-std**: ambiente che offre la controparte asincrona delle librerie standard bloccanti. Livello intermedio di prestazioni e costo

La programmazione asincrona in Rust consente di scrivere codice che può gestire molte operazioni simultanee in modo efficiente senza bloccarsi su operazioni I/O lente. Ciò significa che il programma può eseguire più operazioni in parallelo senza dover attendere che ogni operazione si completi prima di passare alla successiva. La programmazione asincrona in Rust è basata sui seguenti concetti:

- **Future**: una `Future` rappresenta un valore che potrebbe non essere ancora disponibile. Una future in Rust è un oggetto che implementa il trait `Future`, il quale ha un unico metodo `poll` che viene chiamato per tentare di avanzare l'esecuzione della `Future`.
- **Async/Await**: le keyword `async` e `await` semplificano l'uso delle `Future`. Una funzione definita come `async fn` restituisce automaticamente una `Future`. La keyword `await` viene usata per attendere il completamento di una `Future` all'interno di una funzione `async`. Quando una funzione asincrona è sospesa, il runtime può passare a eseguire altre operazioni asincrone.
- **Executor**: un `Executor` è responsabile di orchestrare e coordinare i vari `Future` e le funzioni asincrone. Il crate **tokio** è uno degli executor più popolari in Rust per la gestione della programmazione asincrona

I diversi ambienti di esecuzione non sono equivalenti. **tokio** implementa un ciclo reattivo proprio, basato sul crate ad alte prestazioni **mio (Metal I/O)**, che non è compatibile con i tratti usati dagli altri due. **tokio** è multi-thread: costruisce tanti thread (thread-pool) quanti sono i core effettivi della CPU e ripartisce l'esecuzione dei vari blocchi dei `Future` sui vari thread. Un runtime può basarsi su un singolo ciclo reattivo e/o utilizzare un thread-pool cui delegare l'esecuzione di più **Future** in parallelo. In questo caso, è possibile che l'elaborazione di una funzione asincrona inizi in un thread ma sia continuata in un thread differente. Questo implica che tutti i valori utilizzati nella funzione asincrona il cui uso si estende in più stati devono implementare il tratto **Send**, mentre i riferimenti devono implementare il tratto **Sync**

tokio opera utilizzando un certo numero di code di esecuzione (default: numero di core). Ogni coda è gestita da un ciclo reattivo (**Processor**) che estrae ed esegue i task presenti. Quando un Processor esaurisce i task della propria coda, prova a rubarne alcuni ad altre code (**work stealing**), su base euristica. L'intero algoritmo è ottimizzato per ridurre al minimo la sincronizzazione, prevalentemente usando oggetti atomici.

Oltre ad aggiungere l'opportuna dipendenza nel file `Cargo.toml`, indicando anche quali funzionalità si intendono attivare per il crate, occorre strutturare il punto di ingresso del programma in modo opportuno. **Una funzione `async` può essere chiamata solo dentro una funzione `async`.**

La funzione **`spawn`** definisce un **task**, eseguito in modo concorrente con altri **task**, la cui esecuzione inizia subito (a differenza di un **Future** di cui occorre invocare l'operatore **`.await`**). L'oggetto passato come parametro può essere il risultato dell'invocazione di una funzione `async`, o essere un blocco `async` passato per valore. Fa partire la funzione/blocco `async`, va avanti finché non si blocca, quando si blocca passa a far altro e riprende quando è nelle condizioni di continuare.

La macro **`join!`** serve per coordinare il comportamento dei task. Può essere invocata all'interno di una funzione/blocco `async` e forza l'attesa fino a che tutti i suoi parametri non sono completati. Restituisce **una tupla** contenente i risultati associati ai suoi parametri. Una versione specializzata, **`try_join!`**, può essere usata quando le espressioni passate come parametro hanno come valore di ritorno `Result<T, E>`. In questo caso viene restituito un oggetto `Result` che contiene, se tutti i Future hanno avuto successo, una tupla con i relativi risultati, oppure, **in corrispondenza del primo fallimento**, l'errore corrispondente.

La macro **`select!`** permette di attendere su più rami asincroni, eseguiti nell'ambito dello stesso thread, quello che **termina per primo**, cancellando l'esecuzione dei restanti. Può essere usata solo all'interno di funzioni/blocchi asincroni.

La funzione **`sleep()`** sospende l'esecuzione del task corrente per un tempo pari alla durata indicata. Durante l'attesa non viene consumata nessuna risorsa, se non la memoria necessaria a descrivere l'oggetto Future corrispondente. Allo scadere del tempo, l'esecuzione procede normalmente. La funzione **`timeout()`** attende per un tempo massimo pari alla durata indicata che il Future passato come secondo parametro si completi e restituisce un valore di tipo **`Result<T, Elapsed>`**. Se l'esecuzione si completa in tempo, il risultato è positivo e contiene il valore restituito dal Future, altrimenti riporta un errore di tipo `Elapsed`.

Se, in una funzione asincrona, occorre eseguire un compito computazionalmente intenso, la cui durata possa superare il centinaio di microsecondi, è bene richiedere che venga eseguito in un thread apposito, cos' da evitare di introdurre latenza nell'elaborazione degli altri task. Si utilizza la funzione **spawn\_blocking()**. Occorre evitare di lanciare troppi task di questo tipo. Durante la loro esistenza, infatti, tenderebbero a richiedere l'uso di CPU introducendo contesa con i thread deputati a elaborare le code dei messaggi, aumentando la latenza complessiva del sistema. Può essere opportuno condizionare l'esecuzione alla disponibilità di risorse globali o usare esecutori ad hoc, come quelli offerti dalla libreria **Rayon**

Potremmo aver bisogno dei punti di sincronizzazione durante un task e non solo alla fine di un task. Se due task hanno bisogno di condividere una struttura dati, questa deve essere opportunamente protetta. A seguito del meccanismo di work stealing adottato dallo scheduler, è infatti possibile che l'esecuzione avvenga in thread differenti. Occorre pertanto adottare le stesse precauzioni e strategie già viste con la programmazione multithread. tokio mette a disposizione una ricca serie di primitive asincrone, alcune basate sulla condivisione dello stato (**Mutex**, **Semaphore**, **Notify**, **RwLock**, **Barrier**) altre sulla comunicazione dei messaggi (**canali oneshot**, **mpsc**, **broadcast**, **watch**)

**Notify** è un modulo di sincronizzazione che permette ad un task di notificare ad altri task che un certo evento è avvenuto. **Notify** può essere usato per implementare meccanismi di notifica tra task asincroni senza la necessità di un **Mutex** o altre primitive di sincronizzazione più pesanti.

Un canale **watch** è un canale multi-producer e multi-consumer che conserva solo l'ultimo valore inviato. Il channel è creato con un valore iniziale. Per accedere al valore corrente memorizzato nel canale e contrassegnarlo come visto da un determinato ricevitore si utilizza **borrow\_and\_update()**. Per accedere al valore corrente senza contrassegnarlo come visto, si utilizza **borrow()**